CITYU STUDYNOTES

CS5222 · 计算机网络

课程讲义与练习

中文讲解 · English terminology · 双语题答

A4 单栏 · 根据已下载教师材料整理

本资料用于课程学习与复习，保留原材料、历史题源与教学补充的身份。

模板：ElegantBook 4.7；版式与构建由 CityU-StudyNotes 维护。

2026-10-16

目录

Chapter 1 · Introduction ｜一条消息怎样穿过互联网 **5**

1. What is the Internet? ｜先分清“设备视角”和“服务视角” 5
2. What is a protocol? ｜格式、顺序、动作缺一不可 6
3. Network edge and access ｜接入网把你带到第一个入口 7
4. Circuit versus packet switching ｜预留座位，还是来了再排队 9
5. Forwarding, routing and a network of networks ｜路口动作与整条路线 12
6. Delay and loss ｜慢在哪一段，公式才选得对 13
7. Real delays and throughput ｜测得的 RTT 不是某一条线的长度 16
8. Layers and encapsulation ｜把一件复杂的事拆成清楚的职责 17
9. Security and history ｜知道威胁，也知道哪些数字属于当年 18
10. Chapter 1 Q&A ｜直接对照老师的五道课堂题 19
11. 回顾、任务与来源索引 22

Chapter 2 · Application Layer ｜让两端的程序真正说上话 **24**

1. Applications, processes and sockets ｜先找到真正对话的双方 24
2. Service requirements ｜ “可靠”不等于“准时” 25
3. Web and HTTP ｜拿到 HTML 不等于拿齐整页 26
4. Cookies and caches ｜识别会话，与复用内容是两件事 28
5. Electronic mail and SMTP ｜把“送到邮箱”和“打开邮箱”拆开 28
6. POP3, IMAP and webmail ｜下载之后要不要留在服务器? 29
7. DNS services and hierarchy ｜通讯录不是只放在一台电脑上 30
8. Recursive versus iterative DNS ｜是“替我办完”，还是“告诉我下一站” 30
9. DNS records and messages ｜名字、值、类型、有效时间要一起读 31
10. P2P file distribution ｜新加入的人也可以带来上传能力 32
11. BitTorrent ｜先找邻居，再交换稀缺块 33
12. Video, DASH and CDN ｜边下边播，需要的不只是平均带宽 34
13. Socket programming ｜把课堂的大写转换例子真正跑通 36
14. Back to the course ｜用一条请求链把本章连起来 37

课堂 Q&A：按原题判断并说明条件 37

15. Source coverage and cautions ｜每一页有去处 39

Chapter 3 · Transport Layer（当前 part 1–2） **41**

数据到了电脑，还要交对进程、补齐缺口 41

1. Transport service ｜ IP 找到主机，传输层找到应用端点 41
2. Multiplexing and demultiplexing ｜同一栋楼有许多门 42
3. UDP ｜少做承诺，也少保存状态 42
4. Internet checksum ｜算得对不等于能发现一切错误 43

5. rdt1.0 到 rdt2.2 ｜每多一种故障，就多一项必要机制 43
6. rdt3.0 ｜坏消息会来，丢掉的消息可能一直不来 46
7. Stop-and-wait performance ｜忙了 8 微秒，等了 30 毫秒 47
8. GBN 与 Selective Repeat ｜后面的包到了，要不要先收好？ 48
9. TCP 序号｜从“第几个包”切换到“下一个字节” 50
10. RTT 与重传超时｜等多久才有理由怀疑包丢了？ 51
11. 回到老师的题目 52
来源覆盖索引 53

Tutorial 1 · Switching, circuits and sharing ｜把链路模型算清楚 **54**
Q1. Store-and-forward ｜先等整包到，再转发 55
Q2. Circuit capacity ｜数连接，也要数它消耗了几条边 57
Q3. FDM and setup ｜总速率不等于你的那一份 58
Q4. Statistical multiplexing ｜算人数前，先说是否独立 59
回查与巩固 60

Tutorial 2 · Delay and traceroute ｜先定义计时事件，再按顺序相加 **61**
Q1. VoIP ｜语音先攒成一包，还要算这段等待 62
Q2. Three store-and-forward links ｜每一跳的发送与传播都要数 63
Q3. Immediate bit forwarding ｜不用每个中间节点再等整包 64
Q4. Queueing ｜只算轮到你之前的等待 65
Q5. Traceroute ｜把真实测量和推断分开写 65
回查与巩固 66

Tutorial 3 · HTTP and DNS ｜从报文证据走到时间计算 **67**
Problem 1. Read an HTTP request ｜不知道的就明确说不知道 67
Problem 2. DNS resolution ｜一对箭头才是一次往返 69
Problem 3. A page with eight objects ｜先取目录，再知道要取哪八件东西 70
Problem 4. Nslookup and direct-IP browsing ｜解析成功，不保证 IP 网址等于原站 72
来源与复习 72

Tutorial 4 · File Distribution, BitTorrent and DASH **73**
先找谁供不动，再算怎样分得完 73
Problem 1. Construct a distribution scheme 73
Problem 2. 15 Gbit 分给 100 人 74
Problem 3. 不上传的人，是否仍可能下载完成? 75
Problem 4. DASH 到底存 N 份，还是 N^2 份? 75
复习与来源索引 76

Tutorial 5 · Demultiplexing, Checksums and Reliable Transfer **77**
写清端点，算清位，再用反例检查协议假设 77
Q1. How is a socket identified? 77
Q2. Server replies ｜回程要把两端一起交换 78
Q3. One’s-complement checksum ｜八位是题目的简化 79

Q4. Alternating bit ｜编号又回到 0，旧包也可能叫 0 80
来源与复习 81

Assignment 1 · Chapters 1 and 2 ｜计算、事务与技术证据 **82**
P6. Transmission and propagation ｜最后一位刚出发，第一位可能已经到达 82
P31. Message segmentation ｜像流水线一样，让不同链路同时工作 84
P17. POP3 ｜完成整次事务，不只补两行 86
Zoom. Apply protocol ideas with sources ｜先说研究的是哪一种 Zoom 连接 89
覆盖、复算与验收 90

Research Report Guide ｜把一个真实问题解释清楚 **91**
1. 选题从一件事开始，不从一串热门缩写开始 91
2. 把问题、机制、效果连成一条可核查的解释 92
3. 找来源时，为每一句重要话准备证据 92
4. 图要解释困难，不是给文字化妆 93
5. 英文报告怎样组织才连贯 93
6. Reflection log ｜写你真的经历过的理解变化 93
7. 交付前的自查与来源覆盖 94

Chapter 1 · Introduction ｜一条消息怎样穿过互联网

课程目录 · Tutorial 1（本册 p.54）· Tutorial 2（本册 p.61）· Assignment 1（本册 p.82）· 下一章（本册 p.24）

你在宿舍点开课程网页，屏幕很快显示内容。中间不是一条从电脑直达学校服务器的专属电线，而是许多设备、链路和网络共同协作。本章先建立这幅全景：谁负责收发、数据怎样共享道路、为什么会等待或丢失、复杂系统如何分工。后面每章会放大其中一部分。

依据当前完整 PPT（78 页）及课堂 Q&A（6 页）。保留老师 Chapter 命名与顺序。原 PPT 说明其改编自 Kurose/Ross 教材课件、由 Man Hon Cheung 修改；原图版权属于原作者。补充算例在正文中标注。

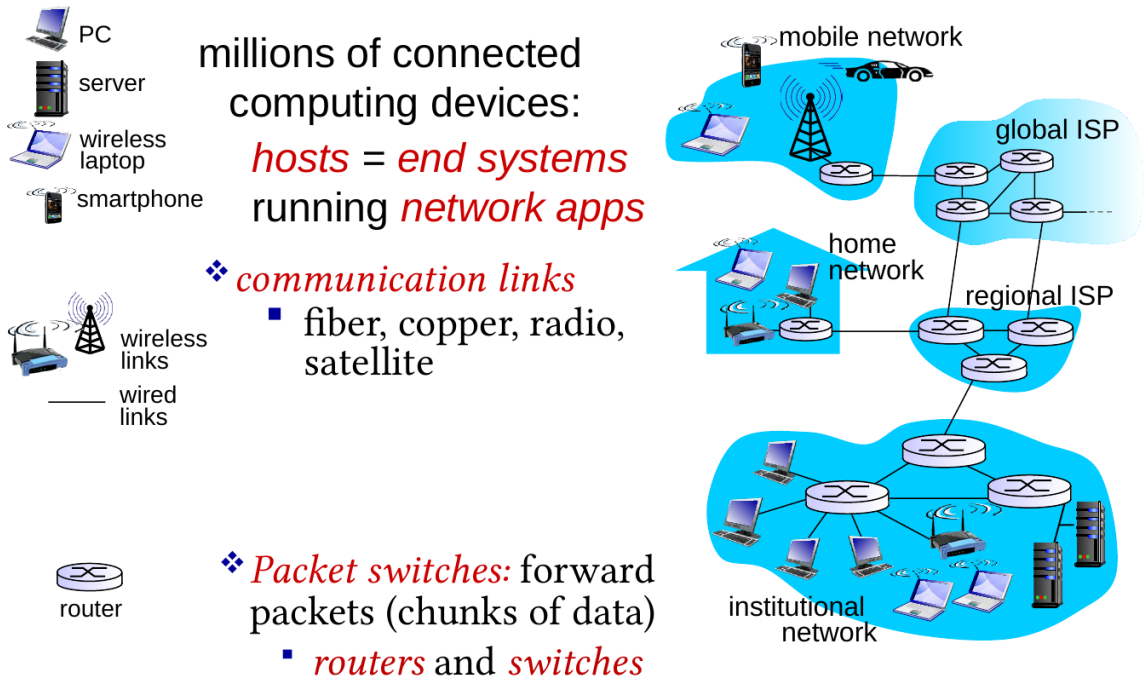
基础复习（可跳过）：先自查 1500 B÷2 Mbps 会不会算；若卡住读单位与时间轴（基础册 p.37）。多人同时活跃的概率看组合概率（基础册 p.28）。

主题	优先级	难度与卡点	掌握要求；依据
Internet、protocol、edge/core	核心必会	低至中；设备和功能	用英文解释与举例；Chapter 1 完整课件，第 4–12 张幻灯片、Q&A
交换、复用、流水线	核心必会	中；容量与时序	画图、手算、说明假设；Chapter 1 完整课件，第 20–32 张幻灯片、Tutorial 1、Assignment 1
delay/loss/throughput	核心必会	中；四类时延和瓶颈	带单位计算与解释观察；Chapter 1 完整课件，第 43–55 张幻灯片、Tutorial 2、Assignment 1
协议分层与封装	核心必会	中；服务方向/数据路径	跟踪消息跨层变化；Chapter 1 完整课件，第 56–65 张幻灯片、Q&A
接入、介质、ISP 结构	常规掌握	中；共享与独占	比较基本机制；Chapter 1 完整课件，第 11–19、33–42 张幻灯片
安全概念与历史	常规掌握	低；定义/时代背景	识别威胁、解释演进；Chapter 1 完整课件，第 66–78 张幻灯片；当前课件内容

优先级与难度独立。当前 Tutorial、Assignment 和课堂题是直接依据；历史复习笔记只作辅助，不按单次出现宣称香港本学期高频或 QE 范围。

1. What is the Internet? ｜先分清“设备视角”和“服务视角”

先从一条网页请求会经过什么来认识互联网。你的电脑和学校服务器是主机，也叫端系统（host / end system）；铜缆、光纤或无线链路（link）让信号在设备之间传播；交换机和路由器负责把数据转到下一段。互联网服务提供商（Internet Service Provider，ISP）把这些网络连接起来。这就是老师所说的“设备组成视角”（nuts and bolts）。



看图从 home network 沿连接走到 regional/global ISP，再到别的网络。手机、PC、服务器属于应用所在端系统；交换机和路由器服务中间转发。原 Chapter 1 完整课件，第 5 张幻灯片的联网相框、冰箱等例子说明“host 不等于台式电脑”。

服务视角（Chapter 1 完整课件，第 7 张幻灯片）则问应用能获得什么：程序通过接口请求把数据送到远端程序，好比寄件时按规则写地址、交给邮政系统。应用不必亲自安排沿途每一个路口；但选用的传输服务决定它获得哪些保证。RFC（Request for Comments）是互联网技术文档系列；IETF（Internet Engineering Task Force，互联网工程任务组）制定许多互联网标准（Chapter 1 完整课件，第 6 张幻灯片）；并非每一份 RFC 都是正式标准。

English takeaway: The Internet connects networks of hosts and packet switches, and provides communication services to applications through interfaces.

来源：Chapter 1 完整课件，第 4 张幻灯片。

2. What is a protocol? ｜格式、顺序、动作缺一不可

两个人问时间，要先提出问题，再给出回答；听不清时还可能重问。网络中的协议（protocol）同样需要约定交换消息的**格式、顺序，以及发送/收到消息时的动作**。只有“说英文”不够，还要知道先问还是先答、没收到怎么办。网络协议把这类约定变成机器可执行规则。来源：Chapter 1 完整课件，第 8–9 张幻灯片。

课堂图先 TCP 连接请求/响应，再 HTTP 请求/文件。它是教学时序，不意味着所有互联网应用都先用 TCP；下一章会比较 TCP 与 UDP。发送方与接收方遵守同一套约定，才能把字节解释成相同的内容。

自查 / Check: 协议是否只规定报文长什么样？/ Does a protocol specify only the format of a message?

答案 / Answer

不，还规定消息顺序和在发送/接收等事件发生时执行的动作。/ No; it also specifies message ordering and actions taken on relevant events.

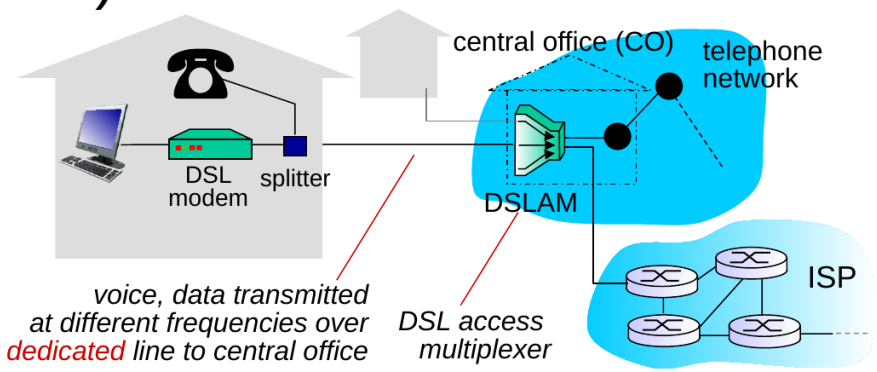
3. Network edge and access ｜接入网把你带到第一个入口

接入网把家里或学校的端系统连到互联网的第一个入口，常见方式有家庭接入、学校/公司接入、移动接入。每次比较都问两件事：速率多少（bit/s）？资源是共享还是专用？仅看到“带宽很大”还不能推断每个用户都能持续独占它。

以家里的电话线接入为例：打电话和上网共用一根线，却占用不同频段，像在同一条路上划出不同车道。这种方式叫数字用户线路（Digital Subscriber Line，DSL）。运营商一端用 DSLAM 设备汇集许多用户的线路，再接入 ISP 网络。家到局端的这一小段可以专用，后面的道路仍可能与别人共享。

家里常见的一只“路由器盒子”也可能同时做几份工作：modem 把数据转换为线路上的信号，路由/NAT 负责转发和地址转换，防火墙按规则过滤流量，Wi-Fi 接入点（access point，AP）提供无线连接。装在同一个盒子里，不等于这些功能相同。

Home access net: digital subscriber line (DSL)



- DSL modem: takes digital data & translates it to high frequency tones for transmission over telephone wires.
- use *existing* telephone line by frequency division multiplexing
  - voice over DSL phone line goes to telephone net
  - data over DSL phone line goes to Internet
- DSL access multiplexer (DSLAM): Translate analog signals from many such houses into digital format.

Introduction 1-13

企业以太网把主机连到交换机，再由机构路由器连 ISP。无线 LAN 经 AP 共享无线介质，蜂窝接入经运营商基站。课件列出的 54Mbps、3G/4G 等速率是原版本的示例，不作为 2026 年的产品上限或购买指标；标称速率也不同于应用可得吞吐。

介质（Chapter 1 完整课件，第 17–19 张幻灯片）	信号怎样走	要理解的差别
Twisted pair 双绞线	铜导线中的电信号	成本、距离、布线与干扰
Coaxial cable 同轴	同轴导体	可承载多个频带；HFC 结合光纤与同轴
Fiber 光纤	光信号	高容量、抗电磁干扰；“每个光脉冲一个 bit”是入门简化
Radio 无线	电磁波传播	反射、遮挡、干扰，共享频谱
Satellite 卫星链路	经卫星中继	传播距离与轨道决定时延；不能将一个示例时延套到所有轨道

English: Access capacity, sharing, and physical propagation are separate considerations. A dedicated access segment does not imply an end-to-end dedicated path.

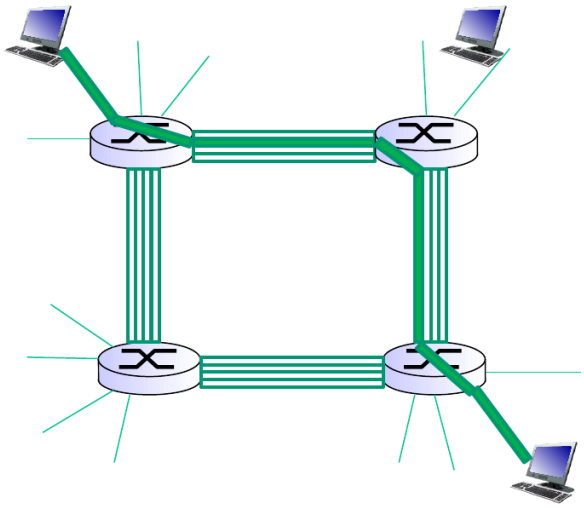
来源：Chapter 1 完整课件，第 11–19 张幻灯片。

4. Circuit versus packet switching ｜预留座位，还是来了再排队

Circuit switching（电路交换）先预留路径上的资源，即使使用者暂时不发送，别人通常也不能借用这份预留。Chapter 1 完整课件，第 22 张幻灯片原图每条边有 4 条电路，一次连接在经过的每段占一份。频分复用（frequency-division multiplexing，FDM）让各连接使用不同频带；时分复用（time-division multiplexing，TDM）则轮流分配重复出现的时隙。同一用户在不同链路使用哪一个编号电路不必相同。

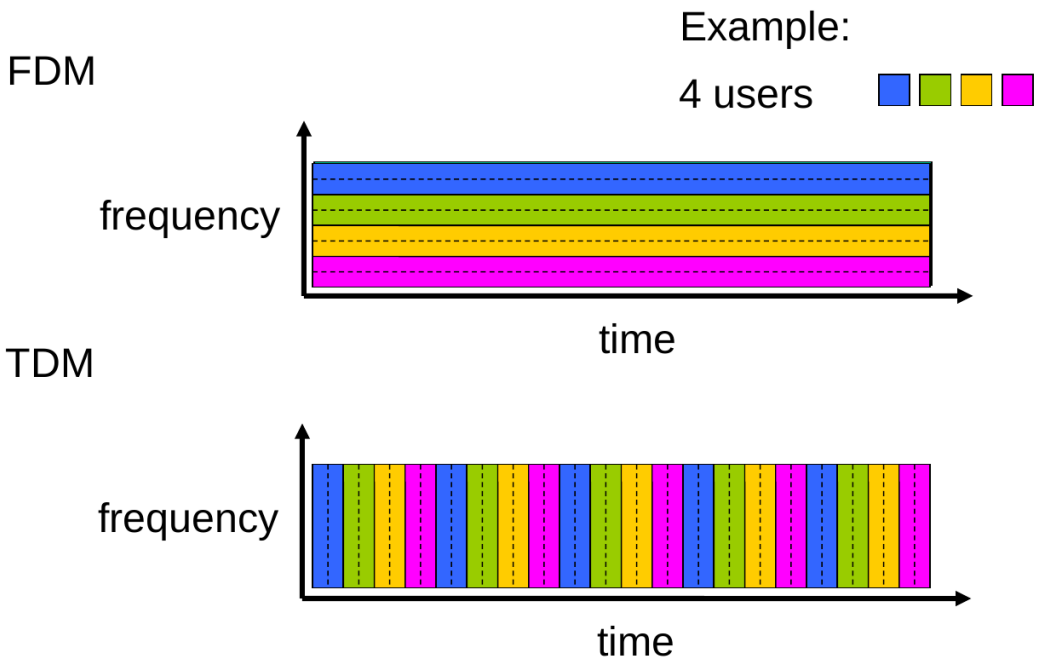
Circuit switching

Circuit switching: end-end resources are reserved for communication between source & dest:
In diagram, each link has four circuits.
call gets 2nd circuit in top link and 1st circuit in right link.
circuit segment idle if not used by call *(no sharing)*
Commonly used in traditional telephone networks



Introduction 1-22

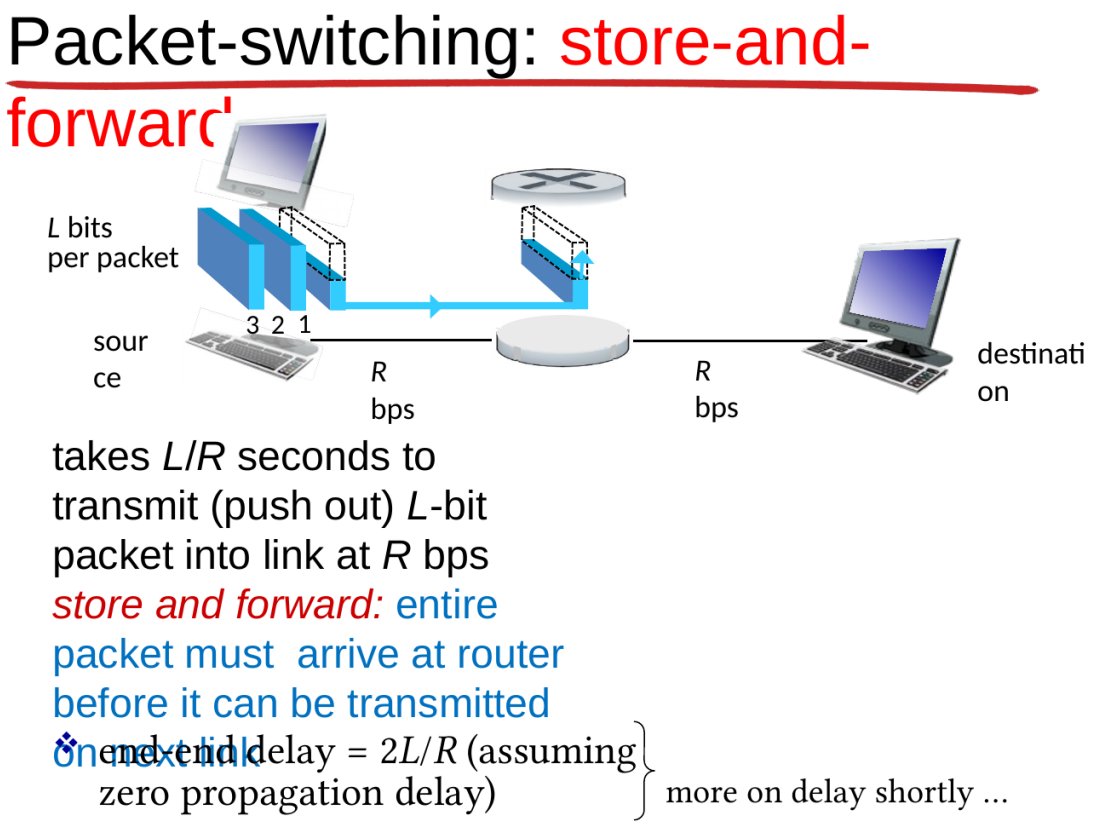
Circuit switching: FDM versus TDM



Introduction 1-23

Packet switching（分组交换）把应用消息拆成包，按需使用链路。一包发送时可用链路发送速率 R，竞争者可能排队；这不表示每个并发用户都永久获得 R。像食堂公共座位提高利用率，但午饭同时来的人多了还是要等，不能只记“共享比较好”。

Store-and-forward（存储转发）要求收完整个包再向下一跳发送。包长 L bit、速率 R bit/s，一跳发送耗时 L/R。在忽略传播、排队、处理的 Chapter 1 完整课件，第 26 张幻灯片模型中，一个包过两条链路耗时 2L/R。注意“一跳”不自动包含整条路径。



三个包过两跳不是 6L/R，因为后一包可在前一包进入第二链路时占用第一链路。完整时序见基础时间轴（基础册 p.40）与 Tutorial1 Q1（本册 p.55）。等长包、等速率、连续发送且没有额外时延时，P 个包、N 条链路总耗时 $(P+N-1)L/R$。

共享带来的容量收益，有什么条件？

原例：1Mbps 链路，每用户活跃时 100kbps、活跃概率 0.1。电路交换最多固定支持 10 人；35 人做分组共享时，假设用户独立，活跃人数 K 服从 Binomial(35,0.1)。

$$P(K>10)=\sum_{k=11}^{35}\binom{35}{k}0.1^k0.9^{35-k}\approx 0.0004243. \tag{1.1}$$

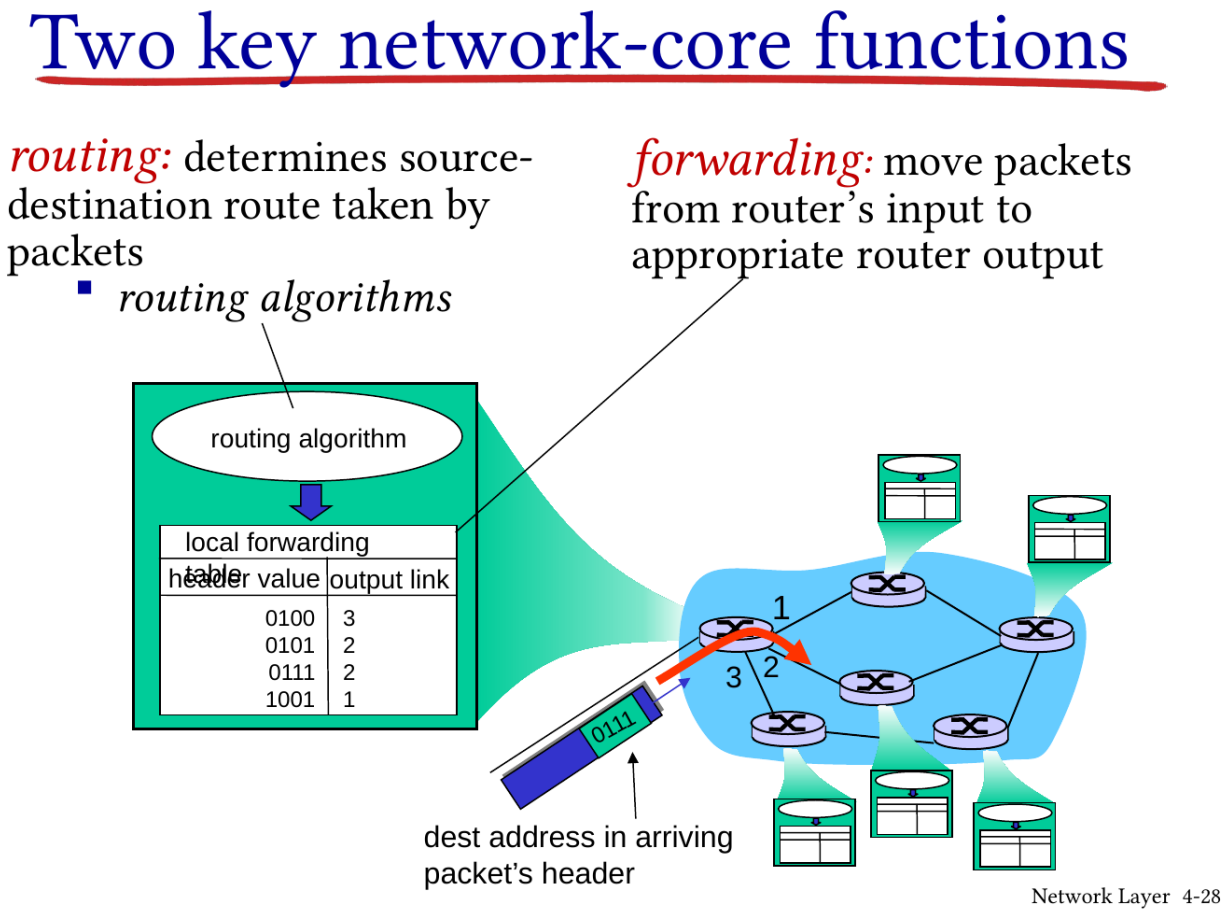
原页写 “less than .0004” 不够精确；独立模型重算约 0.0004243，约 0.04243%，是略大于 0.0004。Chapter 1 完整课件，第 30 张幻灯片只有 Binomial 标题，补课给出从列举到公式的台阶。

这说明超额需求较少，不是保证零排队、零丢包或人人始终满速；短时到达突发、包调度和缓冲也影响等待。用户相关性改变时需要重新建模。分组交换适合突发需求，电路预留则提供更可预测资源，选择取决于服务需求。

来源：Chapter 1 完整课件，第 29 张幻灯片。

5. Forwarding, routing and a network of networks ｜路口动作与整条路线

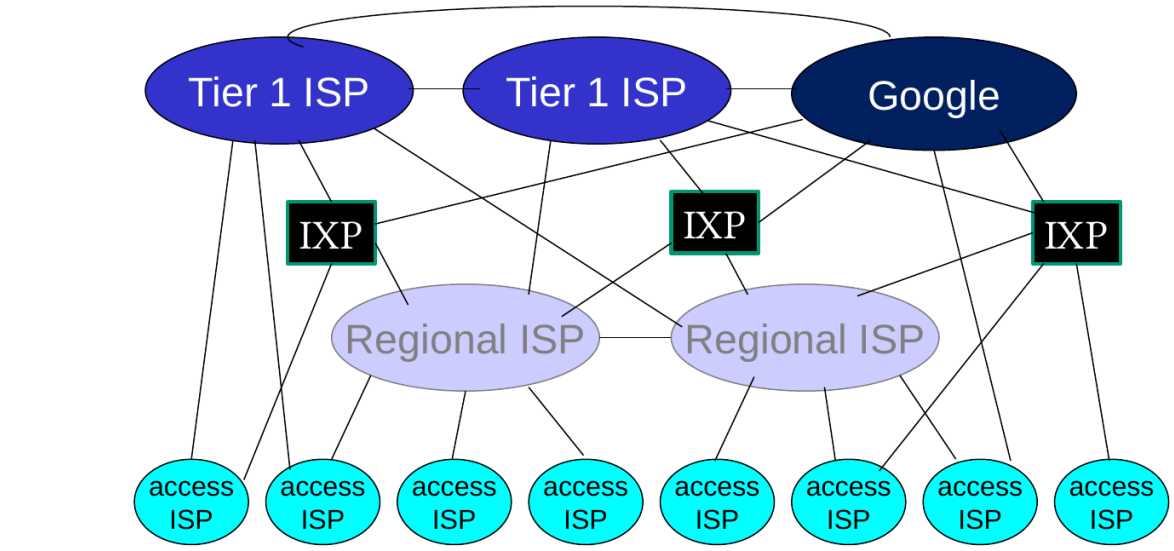
课堂的 forwarding 是当前设备查表，把到来的包从合适出口送走；routing 是确定路径并形成所需转发信息的过程。导航规划与开到路口选出口的类比有帮助，但真实路由可能分布式更新，不必有一个中央导航员。



沿图做一次实际查表：到达包的目的字段是`0111`，设备找到表中`0111 → 2`这一行，把包交给 2 号输出链路。若该出口正在发另一个包，本包先进入相应队列；查到出口不等于已经发出。另一个包的目的字段若为`1001`，按同一张表应走 1 号出口。这里的四位字段是课件示意，不是完整 IPv4 地址。

路由过程负责产生、更新这张表所需的信息；转发过程用当前表处理眼前的包。因此一个包到来时，设备不必为它重新计算整张互联网的路线。后面讲 CDN 把副本放近用户时，改变的是请求可能经过哪些网络；沿途各设备仍各自完成这样的局部转发。

不同网络还需要相互连接。一步步构造互联网：若 N 个接入 ISP 两两直连，需 $N(N-1)/2$ 条连接，扩展困难；接入 ISP 可以向上游购买传输服务（transit），让上游替它把流量送往更多网络；也可以与另一网络直接互联（peering）。互联网交换点（Internet Exchange Point，IXP）提供集中互联的位置。区域网络（regional network）连接一片地区，内容提供商网络（content-provider network）连接自己的服务节点。内容提供商将服务放近用户、建立自有网络，会减少对某些上游路径的依赖，从而可能缩短传输路径。原公司的例名保留为图中时代背景。



English: Forwarding is a local per-packet action; routing determines paths. The Internet consists of interconnected networks with both technical and economic relationships.

来源：Chapter 1 完整课件，第 28 张幻灯片；第 33–42 张幻灯片。

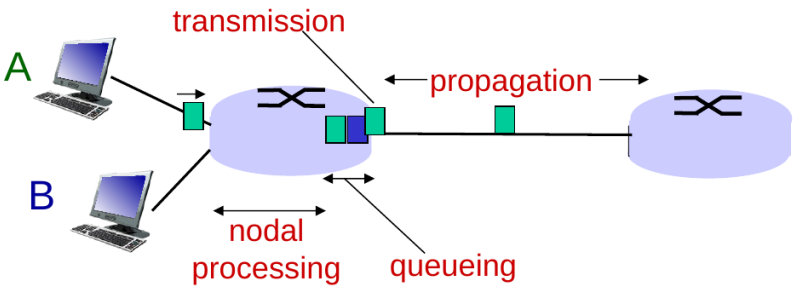
6. Delay and loss ｜慢在哪一段，公式才选得对

区分四类时延：

$$d_{nodal} = d_{proc} + d_{queue} + d_{trans} + d_{prop}. \tag{1.2}$$

项	发生什么	计算/影响
Processing	检错、读头、决定出口	题中给定或忽略
Queueing	等前面的包使用出口	取决于到达与队列状态
Transmission	将整包推入链路	L/R；L bit、R bit/s
Propagation	已进入介质的信号往前走	d/s；d m、s m/s

Four sources of packet delay



$$d_{nodal} = d_{proc} + d_{queue} + d_{trans} + d_{prop}$$

d_{proc}: nodal processing

- check bit errors
- determine output link
- typically < msec

d_{queue}: queueing delay

- time waiting at output link for transmission
- depends on congestion level of router

Introduction 1-45

发送时延看一整包需要多久进入链路，传播时延看已经出发的信号多久走到另一端。下面用老师的车队例子，把这两个计时起点分开。

课堂的车队原例：10 辆车，每辆收费 12 秒，最后一辆过第一个收费亭需 120 秒；100km 以 100km/h 行驶需 1 小时，全部在第二亭前到齐需 62 分钟。Chapter 1 完整课件，第 48 张幻灯片改为每车 1 分钟、车速 1000km/h；首车 1 分钟后出发，再走 6 分钟，t=7 分钟到第二亭，早于第 10 辆 t=10 分钟离开第一亭。两题都从第一辆开始服务计时，没让第二亭再处理一遍。

队列的 traffic intensity 为 $\rho = La/R$，a 为平均包到达率（packets/s），L 为本模型固定包长。rho 接近 1 时在常见随机排队模型中等待显著增长；rho>1 长期输入超过服务能力，无限缓冲理想模型不稳定，现实有限缓冲会丢包。仅平均 rho<1 不能保证任意流量下没有大突发或所有等待性质都良好，课件曲线是模型直觉。

从一次突发看见排队与丢包

设输出链路 8 Mbps，每包 1000 B，发送一个包需 $8000/(8\times10^6)=1$ ms。链路初始空闲，**等待队列只有两个位置，正在发送的包另占服务位置**。A、B、C、D 在 t=0 同时到达，按这个顺序处理到达事件，A 立即开始发送；忽略处理时延，只观察此出口，不计下一跳传播。这是教学补充例子。

包	到达后的处理	开始发送	发完末位	排队时延
A	立即使用空闲链路	0 ms	1 ms	0 ms
B	占第一个等待位置	1 ms	2 ms	1 ms

包	到达后的处理	开始发送	发完末位	排队时延
C	占第二个等待位置	2 ms	3 ms	2 ms
D	两个等待位置均满，丢弃	—	—	不进入队列

等候 B、C 时，链路正在把前面的包送出去；已经发出的比特仍可在介质中传播。三个过程可以重叠，但对于同一个包，自己的等待结束后才开始发送。丢弃的 D 不能记为 “排队时延 0”，因为它没有获得服务。

如果每 10 ms 才来这样一批四包，平均输入为 400 包/s，$\rho = 8000 \times 400/(8 \times 10^6) = 0.4$，仍然每批丢 D。平均供给足够，并不能让两个等待位置容纳一次到来的全部突发。

独立变式 / Transfer： 链路 2 Mbps、每包 500 B，初始空闲，只有一个等待位置，不含服务位置。A、B、C 在 t=0 按此顺序处理，A 立即发送，忽略处理与传播。谁被丢弃？其余包何时开始、何时发完？/ An idle 2-Mbps link receives three 500-byte packets A, B and C at t=0 in that order. A starts immediately. There is one waiting slot, excluding the packet in service. Ignore processing and propagation. Identify the dropped packet and each accepted packet’s transmission interval.

答案 / Answer

每包发送 2 ms。A 在 0–2 ms 发送，B 等 2 ms 后在 2–4 ms 发送，C 被丢弃。/ A transmits during 0–2 ms, B during 2–4 ms, and C is dropped. B’s queueing delay is 2 ms.

来源：Chapter 1 完整课件，第 27、44、49、52 张幻灯片的队列与有限缓冲模型；事件表与变式为补充推导。

把四项接成一次完整计时（教学示范）：某节点从开始处理一个 500 B 的包计时。处理 1 ms、排队 2 ms，随后以 2 Mbps 发送；链路长 1000 km，传播速度 2×10^8 m/s。忽略其他开销。

发送部分先把 500 B 换成 4000 bit，得到 $4000/(2\times10^6)$=2 ms；传播部分把 1000 km 换成 10^6 m，得到 5 ms。从开始处理到末位到达下一节点，总共 1+2+2+5=10 ms。首位到达时包可能还没收齐，所以这里明确计到末位。

独立变式 / Transfer: 另一个包 1000 B，R=4 Mbps，d=3000 km，s=2×10^8 m/s，处理 1 ms、排队 3 ms。从开始处理到整包到达下一节点需多久？/ A 1000-byte packet has a 4-Mbps link, 3000-km distance, propagation speed 2×10^8 m/s, 1-ms processing and 3-ms queueing. Find the time from processing start to complete reception at the next node.

答案 / Answer

发送 2ms、传播 15ms，合计 1+3+2+15=21ms。/ Transmission is 2 ms, propagation is 15 ms; total delay is 21 ms.

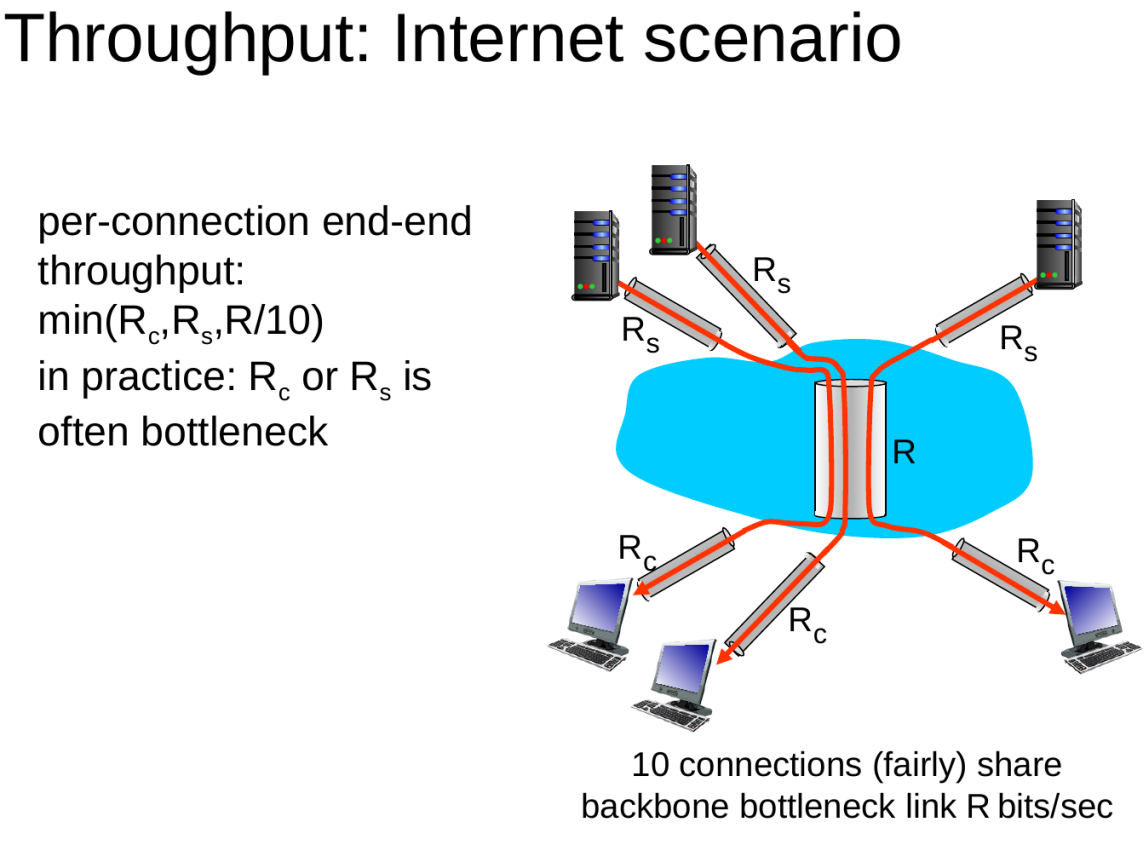
来源：Chapter 1 完整课件，第 44–49 张幻灯片；第 47 张幻灯片。

7. Real delays and throughput ｜测得的 RTT 不是某一条线的长度

用 traceroute 逐渐增加 TTL 探查沿途响应，每行列出应答地址与往返时间 RTT。后跳比前跳小并不矛盾：探测发生时间、排队、回程路径和应答处理可能不同；不能简单逐行相减当作单条链路时延。星号表示规定时间内未收到响应，不足以单独证明那台路由器坏了或一定被防火墙挡住。带日期的本机观察放在 Tutorial2 Q5（本册 p.65）。

课堂的丢包来自缓冲满等机制；包是否重传、由谁重传取决于协议，不是所有丢包都被网络自动恢复。吞吐 throughput 是单位时间实际交付多少 bit，容量 capacity 是链路能力；应用有效载荷的 goodput 还会扣去协议与重传等开销。

简单持续流、无竞争时，多链路瓶颈近似 $\min(R_s, R_c)$；10 条连接公平分享骨干 R 且其他条件简化时，每连接为 $\min(R_s, R_c, R/10)$。公平分享是题设，不是每个真实网络自动保证的定理。



自测 / Check: 接入 10Mbps、服务器 20Mbps、5 条流公平共享 30Mbps 骨干，每流瓶颈吞吐？/ Each flow has 10-Mbps access and a 20-Mbps server link; five flows fairly share a 30-Mbps backbone. Find the per-flow bottleneck rate.

答案 / Answer

min(10,20,30/5)=6Mbps。/ 6 Mbps under the stated sharing model. 容量不要相加成 60Mbps。

来源：Chapter 1 完整课件，第 50–51 张幻灯片；第 52 张幻灯片。

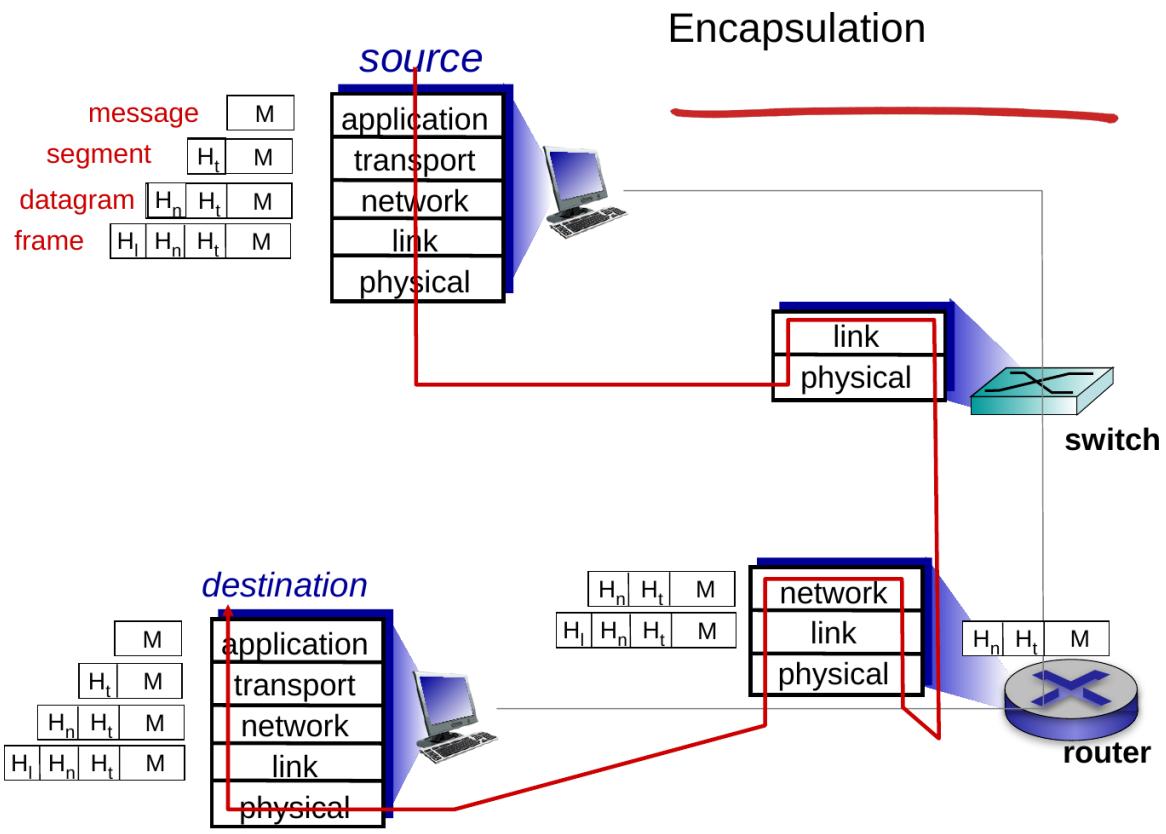
8. Layers and encapsulation ｜把一件复杂的事拆成清楚的职责

老师用航空旅行解释分工：买票、托运行李、登机、起飞、航线各有职责。某一层使用下层服务，同时向上层提供服务；合理接口让内部变化不必牵动所有部分。网络分层借助清楚的责任和接口组织复杂任务。来源：Chapter 1 完整课件，第 58–60 张幻灯片。

Internet 五层（从上向下）	核心职责	课堂例子
Application	应用间协议与内容含义	HTTP、SMTP、FTP
Transport	进程到进程传输	TCP、UDP；保证不同
Network	主机间数据报与路由	IP
Link	相邻节点间传输	Ethernet、Wi-Fi
Physical	介质上的比特信号	铜线/光/无线信号

从底向上编号时 transport 是第 4 层。OSI 另列 presentation/session，Internet 应用仍可能需要编码、加密、会话等功能，只是不独立命名为这两层；“没有这一层”不等于“不需要这个功能”。

封装（encapsulation）将上一层信息作为本层 payload，加上本层头部，某些链路协议还加尾部。例如浏览器的 HTTP 请求先是一段应用消息（message），TCP 给它加传输头成为报文段（segment），IP 再加网络头成为数据报（datagram），链路层包成帧（frame），最后变成介质上的比特信号。接收端按相反顺序解释并移去各层信息。UDP 在传输层通常也称 datagram，读这个词时要确认所在层。



Introduction 1-64

沿原图看：端系统走完整协议栈；典型转发路由器处理到网络层，以太交换机主要处理链路层。不要由这个教学模型推断所有现实设备绝不检查更高层。每跳链路帧可更换，应用消息不因此被每个路由器当成网页程序执行。

把图中的同一条消息 M 跟到底。设一条 HTTP 请求装入一个 TCP 段和一个 IP 数据报，路径为源主机 → 交换机 → 路由器 → 目的主机；本例没有 NAT、隧道、IP 分片或应用代理。H_t、H_n、H_l 分别表示传输、网络和链路层首部，实际首部大小无需在这里猜定。

经过哪里	对这份数据做什么	哪些内容继续向前
源主机	应用产生 M，TCP 加入 H_t，IP 加入 H_n，链路层加入当前链路的 H_l	帧承载整个 IP 数据报，内部仍有 TCP 段与 M
交换机	按链路层信息选择出口，转发链路帧	不把 TCP 段当作本机网页请求处理
路由器	从入站帧取出 IP 数据报，读取目的 IP 并查转发表，处理必要 IP 字段，再装入出站链路的帧	新链路首部服务下一段传送；本例 TCP 段和 M 仍发往原目的主机
目的主机	逐层解释首部，IP 交给 TCP，TCP 按连接信息交给相应应用	HTTP 程序最终读取 M 的请求内容

同层协议描述的是双方如何理解报文，例如两个端点的 TCP 怎样确认字节；实际比特仍要经过本机下层、介质和对端下层。图中两端应用“逻辑上对话”，不表示应用层之间存在一根绕过其余各层的物理线。

自测 / Check：在上述无 NAT、无代理模型中，路由器改用一个出站链路帧，是否就产生了一条新的 HTTP 请求或把 TCP 目的端口改成路由器的端口？/ In this model without NAT or proxies, does forwarding in a new outgoing link frame create a new HTTP request or replace the TCP destination port with a router port?

答 / Answer：都不会。变化发生在逐跳链路封装，HTTP 请求与 TCP 端点仍属于原两端。/ No. The outgoing link encapsulation serves the next hop; the HTTP request and TCP endpoints remain those of the original end hosts.

来源：Chapter 1 完整课件，第 61–65 张幻灯片；逐设备表按第 64 张图展开。

English: Each layer offers services to the layer above and uses services below. Encapsulation adds the information needed by the current layer.

9. Security and history ｜知道威胁，也知道哪些数字属于当年

介绍恶意软件、DoS、嗅探与地址伪造。病毒常依附宿主内容并需执行，蠕虫可自动传播；并非“被动收到任意包就必然感染”，通常还需漏洞或执行机制。Botnet 由受控设备组成，DDoS 是分布式资源耗尽；packet sniffing 能看到可访问的流量，但加密与网络结构限制能读到的明文。伪造源 IP 不等于已经能接收发回那个源地址的响应。

按时间讲分组交换研究、ARPANET、互联思想、TCP/IP 与 DNS、Web 商业化、移动与云。应记住问题如何推动结构演变：让不同网络自治互联、尽力而为、分散控制；“stateless routers”在这里指不为每个端到端连接预留电路状态，不是路由器没有路由表或任何状态。

保留原页历史顺序，同时澄清两处可核查的简写：Chapter 1 完整课件，第 74 张幻灯片将 ALOHAnet 标为 1970 卫星网络；夏威夷大学记录其岛际 packet-radio 服务于 1971 年 6 月开始，卫星连接属于后续发展。UH 项目史 Chapter 1 完整课件，第 76 张幻灯片的 Mosaic 年份应注意版本语境，NCSA 记录其最早普及阶段在 1993 年。NCSA 历史两处查阅于 2026-09-23。

课堂的 2016 设备数量、早期社交网络用户数只作为课件当时背景，不称当前规模；“instantaneous”是宣传式简写，跨网络通信仍有非零时延。

来源：Chapter 1 完整课件，第 67–71 张幻灯片；第 73–77 张幻灯片；第 77 张幻灯片。

10. Chapter 1 Q&A ｜直接对照老师的五道课堂题

以下为原题条件的双语整理，答案为依据本章的推导，原 Q&A 文件没有附独立教师答案页。原图可在末尾展开。

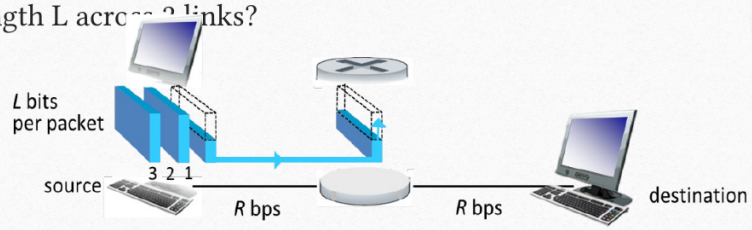
1. **Which is not an edge device: PC, smartphone, server, router? / 哪个不属于本章端系统意义的网络边缘设备？**答 D，router；这里 edge 按运行应用的 end systems 定义，不否认现实术语也有“edge router”。/ D, a router, using the chapter’s end-system definition of edge.
2. **Three L-bit packets cross two R-bit/s store-and-forward links. Ignore propagation, queueing and processing. Find completion time. / 3 个等长包过 2 条等速存储转发链路，忽略其他时延，多久？**答 B，4L/R；流水线时间表见 §4。/ B, 4L/R, because the links can work on different packets concurrently.
3. **Which delay is affected by congestion in the chapter’s model? / 四项时延中哪项随拥塞变化？**答 B，queueing；其他给定包长/速率/距离条件不变。真实设备处理负载也可能变化，但原题考四类简化模型。/ B, queueing delay in the stated model.
4. **A 100Mbps modem connects a phone through a 54Mbps router, upgraded to 1Gbps. What is ideal throughput after upgrading? / 100Mbps 外网口，54Mbps 路由器换 1Gbps 后理想瓶颈？**答 A，min(100,1000)=100Mbps；还假设手机及其他段无更低瓶颈。/ A, 100 Mbps under the simplified bottleneck assumptions.
5. **Which transport-layer statements are true? / 传输层哪项正确？** A 总可靠：错，UDP 不保证；B 是第 4 层：对（底层起数）；C 为下方网络层提供服务：错，服务向上；D 给 segment 加头再上送 application：错，发送方向给应用消息加传输头后向下。/ Only B is correct under bottom-up layer numbering.

原 Q&A 页图 / Original question slides

- Which of this is **not** a network edge device?
  A) PC
  B) Smartphones
  C) Servers
  D) Router

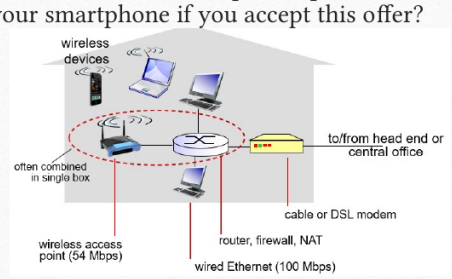
- (**Store-and-forward** transmission) Ignore queuing, propagation delay, and processing delay. What will be the end-to-end delay for sending 3 packets each of length L acrc[illegible]nks?

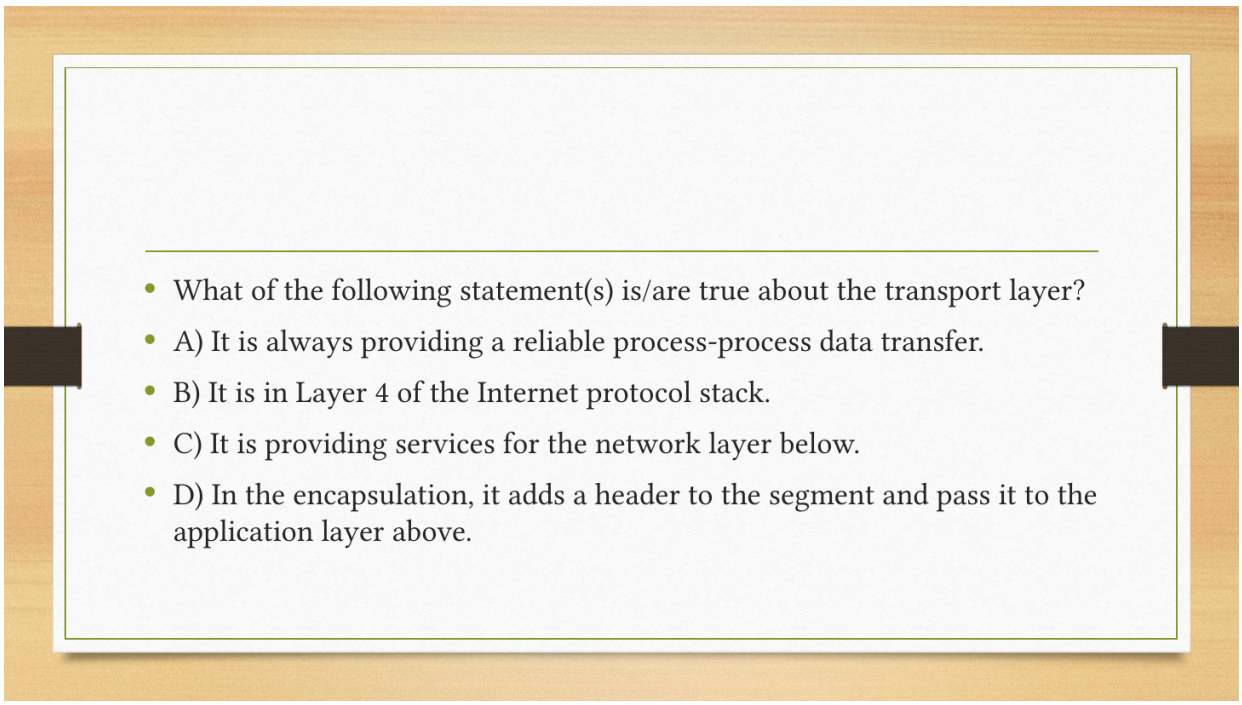
A. 3L/R
B. 4L/R
C. 5L/R
D. 6L/R



- Which of the following packet delay(s) is/are affected by network congestion?
- A) nodal processing delay
- B) queueing delay
- C) transmission delay
- D) propagation delay

- Suppose you are using a modem with speed 100 Mbps for your wireless router with speed 54 Mbps. They are connecting your smarthphone to the Internet. During contract renewal, PCCW offers to give you a new Wi-Fi router of 1 Gbps to replace the old one. What is the end-to-end throughput of your smartphone if you accept this offer?
- A) 100 Mbps
- B) 54 Mbps
- C) 154 Mbps
- D) 1 Gbps





11. 回顾、任务与来源索引

顺着主线复述：端系统产生消息 → 接入网进入互联网 → 分组在链路与交换设备间前进 → 共享导致排队/丢失，路径容量决定瓶颈 → 分层协议规定每一部分的格式与动作。接着做 Tutorial1（本册 p.54）的交换与容量题、Tutorial2（本册 p.61）的时延与测量，再做 Assignment1 P6/P31（本册 p.82）。

英文独立表达：**Explain why increasing a link's rate does not reduce its propagation delay. / 解释为什么增加速率不减少传播时延**。要点：L/R 改变，d/s 由距离与传播速度决定。/ Link rate changes transmission time; propagation depends on distance and signal speed.

首轮可选 **N015（分组交换）**、**N025（四类时延）**、**N033（流量强度）**、**N072（流水线）**，再在 Markji 按卡号复习。微课入口：NET02。

原完整 PPT 页码	本文位置	覆盖内容
1–3	开篇	来源、目标、路线；目录页不重复计主题
4–9	§1–2	设备/服务/API/RFC、协议与原对话
10–19	§3	接入、DSL、家庭/企业/无线、物理介质
20–28	§4–5	电路、FDM/TDM、分组、存储转发、转发/路由
29–32	§4	原 35 用户例、二项补足、交换比较
33–42	§5	ISP 逐步构造、peering/IXP/内容网络
43–49	§6	四类时延、两道车队题、rho
50–55	§7	traceroute、丢包、吞吐、共享瓶颈
56–65	§8	航空类比、五层/OSI、封装与名称
66–71	§9	安全目标与四类威胁

原完整 PPT 页码	本文位置	覆盖内容
72–78	§9、§11	演进、时代数字、总结
Q&A 1–6	§10	标题及全部 5 道题

part1 的 49 页对应完整 1–49；part2 的 29 页对应完整 50–78，页码重编不算新增教学内容。原图在相关段落展示，计算步骤见正文。

Chapter 2 · Application Layer ｜让两端的程序真正说上话

课程目录 · 上一章（本册 p.5）· Tutorial 3（本册 p.67）· Assignment 1（本册 p.82）· Research Report（本册 p.91）

上一章知道消息可以穿过网络，这一章把镜头移到两端：浏览器怎样开口要网页，邮件怎样送到邮箱，名字怎样换成地址，多个用户怎样互相传文件，视频为什么可以边下边播。最后亲手让两个本机程序交换一句话。

依据当前完整 Chapter2（107 页），保持**应用原则 → HTTP → 邮件 → DNS → P2P → 视频/CDN → socket** 的实际正文顺序。早期目录列有 FTP，但当前完整文件未提供独立 FTP 教学段，后半目录为视频/CDN。课件来源为 Kurose/Ross、Man Hon Cheung 修改，原图版权保留。

可选基础： 报文与字节（基础册 p.39）、时间轴（基础册 p.40）。运行 Python 先读 Lecture1 的函数与文件（CS5489 册 p.14），按需要复习即可。

主题	优先级	难度与卡点	掌握要求与依据
架构、进程、socket、TCP/UDP	核心必会	中；对象/服务边界	解释并运行通信；Chapter 2 完整课件，第 4–17、94–105 张幻灯片
HTTP 消息与时延	核心必会	中；连接、流水线、对象数	解读完整请求、画 RTT；Chapter 2 完整课件，第 19–37 张幻灯片、Tutorial 3
SMTP、POP3、IMAP	核心必会	中；投递和访问分开	写出完整事务与状态；Chapter 2 完整课件，第 39–51 张幻灯片、Assignment 1
DNS、递归/迭代、缓存	核心必会	中；谁向谁问	跟踪消息与算 RTT；Chapter 2 完整课件，第 53–69 张幻灯片、Tutorial 3
P2P、BitTorrent	常规掌握	中；资源下界与实际性能	推导三项下界，解释分块；Chapter 2 完整课件，第 71–80 张幻灯片
视频、DASH、CDN	常规掌握	中；码率/吞吐/缓冲	解释机制、算小例；Chapter 2 完整课件，第 82–92 张幻灯片

优先级依据当前课件与配套任务；考试范围以教师公布的要求为准。

1. Applications, processes and sockets ｜先找到真正对话的双方

浏览器、邮件、游戏、会议和视频程序通常运行在端系统上，开发一个网络应用不需要重新编写沿途每台路由器。进程（process）是运行中的程序；一个主机能同时有多个进程，因此只写主机 IP 不够，还需端口等传输端点信息。

Client/server 架构中，一方提供相对稳定可达的服务，客户端发起请求；P2P 中节点既请求也提供资源。纯 P2P 不依赖永远在线的中心数据服务器，但实际系统可有 tracker、登录或协调服务，不能由“使用 P2P”推断完全没有服务器。客户端/服务器也可以是某一次交互的角色，不是某台机器终身的职业。

假设浏览器要向服务器索取网页。浏览器把要发出的字节交给操作系统提供的接口，这个接口叫 socket（套接字）；操作系统再通过传输协议发送。IP 地址帮助找到目标主机，端口号帮助找到主机上的相应服务。这样，访问网页和收邮件就能在同一台电脑上同时进行。

对照课件时留意：32-bit 地址指 IPv4；第 10 张幻灯片的`connect` 是客户端主动连接，附近的“accept incoming”注释与该代码不符。

自查 / Check: 同一 IP 上有 Web 和邮件服务，只给 IP 能否指出收消息的进程？/ Can an IP address alone distinguish a web process from a mail process on the same host?

答案 / Answer

不够，还要传输协议与端口等端点信息。/ No; transport protocol and port information are needed. 课堂 HTTP 80、SMTP 25 是相应传统服务示例，不是全部现代配置。

来源：Chapter 2 完整课件，第 3–4 张幻灯片。

2. Service requirements ｜ “可靠”不等于“准时”

应用协议规定消息类型、语法、语义和交互规则。开放标准便于互操作；专有协议由特定实体控制，但不意味着一定没有任何公开文档。选择传输服务前，先问应用需要什么：完整性、及时性、持续吞吐、安全性。

文件少一段不能算完成，实时通话晚到的音节可能已没有价值。TCP 提供可靠、有序字节流，并有流控（保护接收端）与拥塞控制（适应网络）；它不承诺固定时限、最小吞吐，也不自动加密。UDP 提供数据报接口，没有这些内建可靠性/顺序保证；应用可在其上实现所需机制。TCP 可靠性也不是“网络永久断开仍能保证交付”，失败会暴露给应用。

用 SSL 称呼安全层，是旧术语；今天应区分 TLS 与旧 SSL。教学要点是加密、完整性和端点认证需要额外机制，不能将“用 TCP”当作已经安全。服务表里的码率与应用例名是课堂时代背景，核心是需求比较，不是当前产品参数清单。

English: Reliability, ordering, latency, throughput and security are different service properties. UDP's minimal service lets applications choose additional mechanisms, but does not remove responsibility for congestion and reliability design.

来源：Chapter 2 完整课件，第 12 张幻灯片；第 17 张幻灯片。

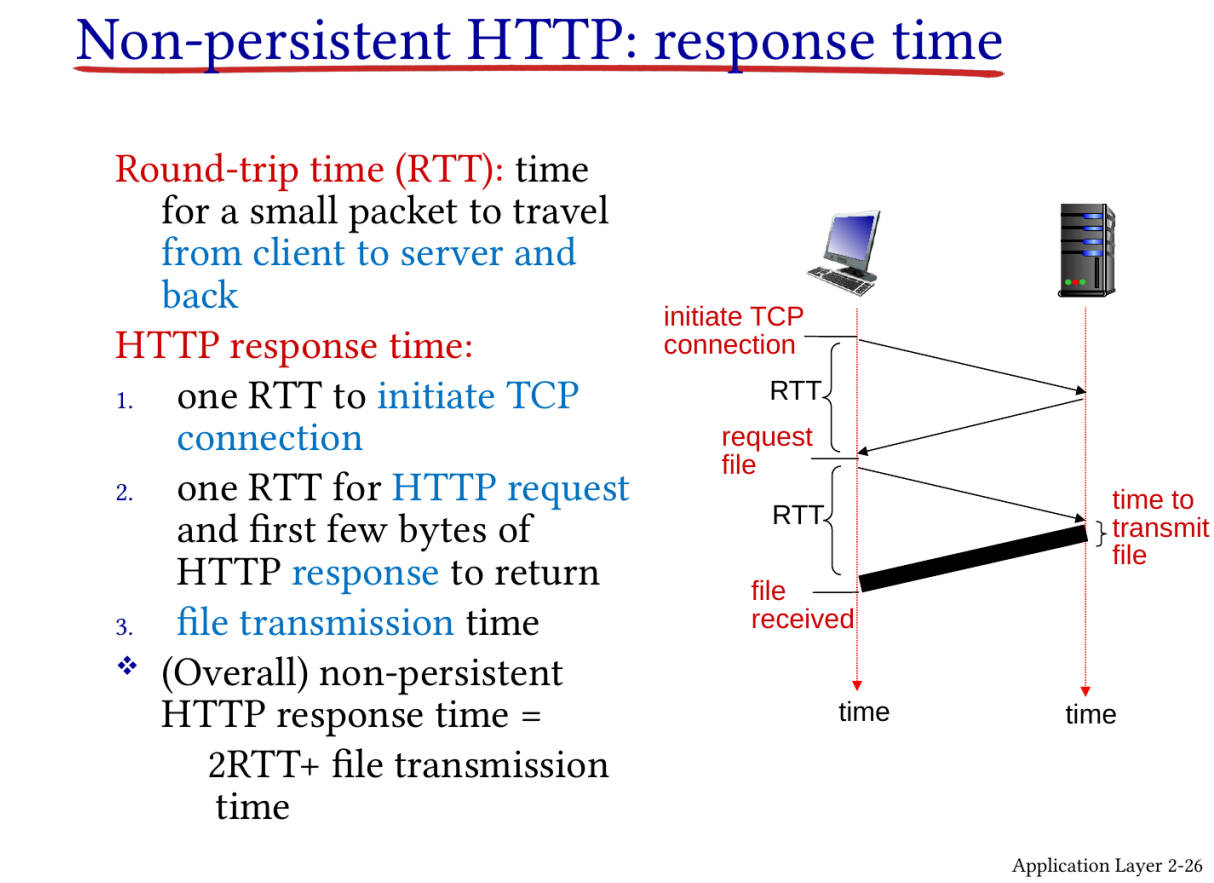
3. Web and HTTP ｜拿到 HTML 不等于拿齐整页

分 base HTML 和 referenced objects：HTML 告诉浏览器还需要哪些图片、脚本等，每个对象有 URL。URL 不只是主机名，还包含路径、可能的协议方案、端口等。本章和 Tutorial 使用传统 HTTP/1.x over TCP 模型；不能把“HTTP 必然使用 TCP”推广到 HTTP/3，后者基于 QUIC。RFC9114

浏览器是 HTTP 客户端，Web 服务器应请求返回资源（Chapter 2 完整课件，第 20 张幻灯片）。HTTP 无状态是说核心请求语义不要求服务器记住先前请求，不等于 TCP 连接没有状态，也不等于网站不能存购物车；cookie 等机制随后会补上会话关联。

连接可以复用，请求是否并行还要另问

Non-persistent HTTP：每个对象新建 TCP 连接。简化为握手 1RTT、请求到首个响应 1RTT，再加对象发送时间；忽略 DNS 等其他部分时 $2RTT + L/R$。它不是包含 TLS、慢启动、服务处理的全现实上网公式。



Persistent HTTP：复用连接，后续对象省去重新握手。**持久连接不等于流水线（pipelining）**；无流水线时每次等上一请求响应后才发下一次。多个并行 TCP 连接又是第三个概念，可让不同对象同时等待，但占更多资源，吞吐也受共享瓶颈限制。

补充小例：base HTML 加 2 个小对象，都在同服务器，忽略发送时间和 DNS。串行非持久共 3×2=6RTT；单个持久无流水线是 1 握手 +3 请求 =4RTT；拿到 HTML 后一次发两个后续请求的理想

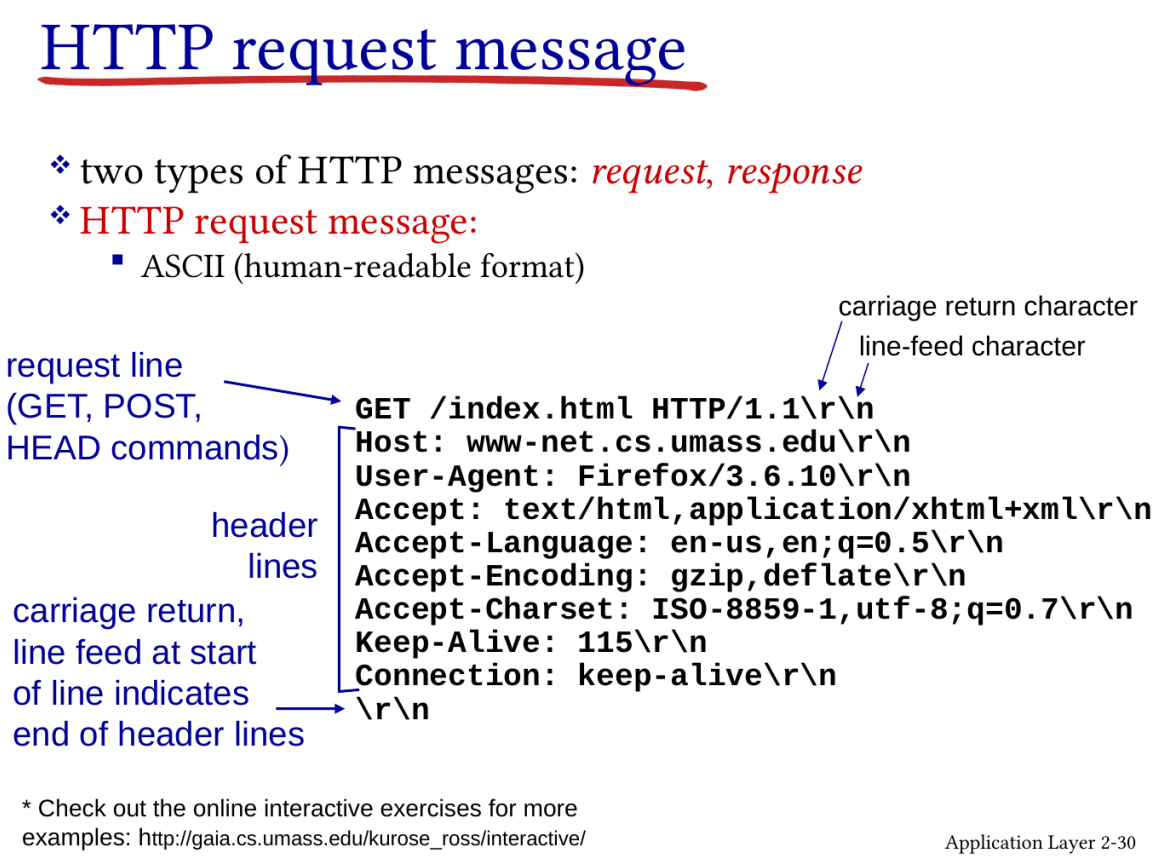
流水线模型为 3RTT。先问清协议/浏览器约定，才有唯一算式。原 Chapter 2 完整课件，第 27 张幻灯片画的是顺序复用，Tutorial3c 据此读。

变式 / Transfer：同一服务器上的 HTML 还引用 4 个很小的对象。初始无连接，先收到 HTML，忽略 DNS、发送、TLS、丢包和处理时延。串行非持久与单条无流水线持久连接各需多少 RTT? / A base HTML file references four small objects on the same server. Start without connections and fetch HTML first. Ignore DNS, transmission, TLS, loss and processing. Compare serial non-persistent HTTP with one non-pipelined persistent connection.

答 / Answer：非持久有 5 次“握手＋请求”，共 10 RTT；持久只握手一次，加 5 次请求，共 6 RTT。/ Ten RTTs versus six RTTs. 复用连接省的是握手，请求本身仍需往返。

读报文：把每一行与问题对应

请求行是 method、request target、version；头字段包括 Host、User-Agent、Accept 等；CRLF 分行，空行结束头部。GET 通常索取表示；HEAD 只请求相应响应头；POST 提交内容供资源处理，不能仅凭方法名保证请求是否“安全”或是否一定有某种 body。



原 HTTP/1.1 文本适合观察 ASCII 结构；不要据此认为所有 HTTP 版本在传输中都用相同明文文本布局。Tutorial3 将用真实题干逐项判断，IP 地址未出现时应明确说无法仅从这些头字段得到，不能从 Host 误认浏览器地址。

English: A page may require multiple object requests. Connection persistence, pipelining and parallel connections are separate choices; name the timing model before counting RTTs.

来源：Chapter 2 完整课件，第 19 张幻灯片；第 30–31 张幻灯片。

4. Cookies and caches ｜识别会话，与复用内容是两件事

课堂的 Susan 购物例子：网站生成标识，响应`Set-Cookie`交给浏览器，浏览器后续发`Cookie`，服务器用它找到后端会话/购物车。四部分是响应头、浏览器保存、服务端记录、后续请求头。Cookie 不是自动等于用户名密码，也不保证一个 ID 就足以安全认证；它也带来跨请求关联与隐私问题。

课堂的 cache 对浏览器是服务器，对 origin 又是客户端。命中且可用时直接回应，未命中向源获取，减少响应时延与接入流量。需要考虑新鲜度、验证及不应共享的个人内容；不是“见到任意响应就永久缓存”。这和 cookie 保存会话身份作用不同。

变式 / Transfer: 一个代理向源服务器请求未命中的对象时，它是 client 还是 server？对原浏览器呢？/ What roles does a proxy play toward the origin and toward the browser?

答案 / Answer

对源是 client，对浏览器是 server。/ It is a client to the origin and a server to the browser. 角色按具体交互定义。

来源：Chapter 2 完整课件，第 32–35 张幻灯片；第 36–37 张幻灯片。

5. Electronic mail and SMTP ｜把“送到邮箱”和“打开邮箱”拆开

Alice 的邮件客户端（user agent）先把信交给发送邮件服务器，放入 outgoing queue；服务器经 SMTP 投递到 Bob 的邮件服务器，进入 mailbox；Bob 再用客户端访问。发件/收件服务器可能在不同交互中分别担当 client/server。

SMTP 通过命令/响应完成 greeting、邮件传送、结束。课堂 Chapter 2 完整课件，第 42 张幻灯片原对话顺序是`HELO` → `MAIL FROM` → `RCPT TO` → `DATA` → 邮件内容 → 单独一行`.` → `QUIT`。只有服务器给出 354 后才发 DATA 内容；250 代表该阶段成功，但不是“Bob 已阅读”。同一持久 TCP 连接可投递多封邮件。

Sample SMTP interaction

```
S: 220 hamburger.edu (server's host name)
C: HELO crepes.fr     (client's host name)
S: 250  Hello crepes.fr, pleased to meet you
C: MAIL FROM: <alice@crepes.fr> (beginning of message)
S: 250 alice@crepes.fr... Sender ok
C: RCPT TO: <bob@hamburger.edu>
S: 250 bob@hamburger.edu ... Recipient ok
C: DATA
S: 354 Enter mail, end with "." on a line by itself
C: Do you like ketchup?          (two-line message)
C: How about pickles?
C: .                                  (end of message)
S: 250 Message accepted for delivery
C: QUIT                 (all messages have been sent)
S: 221 hamburger.edu closing connection
```

Application Layer 2-42

将 HTTP 称 pull、SMTP 称 push，是典型课堂使用方式；HTTP 也支持客户端上传，别把类比读成协议完全禁止别的方向。邮件内容头`From/To/Subject`与 SMTP 信封命令作用不同；头部与正文之间为空行。Chapter 2 完整课件，第 41/45 张幻灯片的 7-bit ASCII 是基础历史模型，附件与国际化需编码/扩展，不能说现代电子邮件只能写英文。

English: SMTP transfers mail to a receiving server. Message headers are distinct from SMTP envelope commands, and server acceptance does not mean the recipient has read the message.

来源：Chapter 2 完整课件，第 39–43 张幻灯片；第 44 张幻灯片。

6. POP3, IMAP and webmail ｜下载之后要不要留在服务器？

邮件到达服务器后，收件人还要访问邮箱。POP3 的三阶段是 Authorization、Transaction、Update。客户端认证后用 LIST 看大小、RETR 取内容、DELE 标记删除，成功 QUIT 进入 Update 提交删除。**RETR 本身不会自动删除**，DELE 也不是立刻不可撤销地移除；在 Transaction 中 RSET 可清除删除标记，非正常断开不进入正常 Update 流程。RFC1939 §§5–6

Download-and-delete 与 download-and-keep 由客户端是否标记删除并成功结束会话等行为形成，不是“POP3 只能删”。跨会话不维护 IMAP 那样的文件夹/已读同步状态，不等于服务器不保存邮箱或不能提供持久 UIDL 识别；客户端可记住已下载 UID 来避免重复下载。

IMAP 把邮件和文件夹等状态留在服务器，支持多设备一致访问。Webmail 中浏览器与网站通常使用 HTTP(S)，而邮件服务器之间仍可 SMTP；不要因为你在网页按发送，就断言浏览器直接向所有收件服务器开 SMTP 连接。完整两封邮件事务见 Assignment1 P17（本册 p.86）。

来源：Chapter 2 完整课件，第 46–51 张幻灯片。

7. DNS services and hierarchy ｜通讯录不是只放在一台电脑上

课堂的 Domain Name System 既是分布式数据库，也是应用层协议。它解决主机名到地址、别名、邮件服务器定位与负载分布等问题。一个名字可以返回多个地址，同一台服务器也可服务多个名字。

以查询`www.example.com` 为例，本机通常把问题交给一个递归解析器（recursive resolver，课件称 local DNS）。如果没有可用缓存，它先问根服务器（root），得知该向哪些.com 顶级域服务器询问；再问顶级域（top-level domain，TLD）服务器，找到负责 example.com 的权威服务器（authoritative server）；最后向权威服务器取得所需记录。每一步可能只给出“下一站问谁”，这叫委派（delegation）。

root、TLD 和权威服务器形成管理层次；本地解析器负责替你沿这条线查询及缓存结果。一个权威服务器负责的一片记录叫区域（zone）。域名树的一部分还可以交给其他服务器管理，因此 zone 的边界不一定与某个 domain 的全部下级名称重合。

把 root 写成“自己联系权威并取回最终映射”不代表通常迭代解析的根服务行为；按随后 Chapter 2 完整课件，第 60–61 张幻灯片的常见图，递归解析器接收 referral 后自己继续查询。RFC1034 §4.3 原页根服务器实例数量是当时快照，不作为当前数量。不要把根服务标识数、实例数、物理机器数混成同一个概念。

来源：Chapter 2 完整课件，第 53–59 张幻灯片；第 57 张幻灯片。

8. Recursive versus iterative DNS ｜是“替我办完”，还是“告诉我下一站”

请求主机向本地 resolver 发递归请求：“请给最终答案或错误”。resolver 向 root、TLD、权威发迭代请求：“若不知道，告诉我下一站”。收到最终答案再返还主机。Chapter 2 完整课件，第 62 张幻灯片展示全递归作为另一种可能模式，不能默认每一层都替下一层递归。

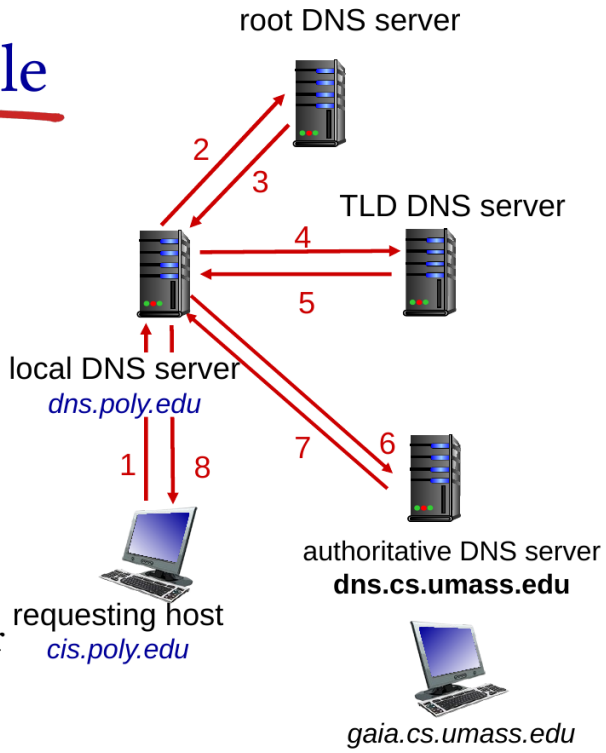
DNS name resolution example

Iterative query:

- contacted server replies with name of server to contact
- "I don't know this name, but ask this server"
- E.g., (2,3), (4,5), (6,7)

Typical query pattern:

- query from the requesting host to the local DNS server is recursive
- remaining queries are iterative.



Application Layer 2-61

看图先跟数字 1–8，不要把每根箭头当一个 RTT：一次往返有去和回。无缓存、顺序访问 root/TLD/权威时，若 host↔local 为 RTT_L，每个 local↔ 远端为 RTT_r，总 RTT_L+3RTT_r。命中 local 缓存、无需再验证且忽略处理时，只 RTT_L。原图也用于 Tutorial3。若 RTT_L=2 ms、RTT_r=10 ms，无缓存时为 2+3×10=32 ms；本地命中时为 2 ms。

独立题 / Check：仍按图中顺序，忽略处理与发送时延。RTT_L 改为 3 ms、RTT_r 为 15 ms，无缓存与本地有效缓存命中分别多久？/ Under the same sequential DNS flow and negligible processing/serialization, use RTT_L=3 ms and RTT_r=15 ms. Find the uncached and valid local-cache-hit times.

答 / Answer： 3+3×15=48 ms 与 3 ms。/ 48 ms and 3 ms.

缓存有 TTL（time to live，有效存活时间），回答可以非权威但仍来自缓存的权威资料链。过期需要重新查询；记录变化不会自动让所有旧缓存同时消失。DNS TTL 是缓存时间概念，和 IP TTL 的跳数控制不是同一字段。

9. DNS records and messages ｜名字、值、类型、有效时间要一起读

类型	name→value	用途
A	主机名 →IPv4 地址	地址映射；IPv6 另用 AAAA（补充区分）
NS	区域/域名 → 权威服务器主机名	委派；不是直接给全部终端 IP

类型	name→value	用途
CNAME	别名 → 规范名	后续仍需得到地址记录
MX	邮件域 → 邮件交换服务器名	实际还含优先级字段；不是发往 Web 服务器的 A 记录

例如解析器要找`video.example`的 IPv4 地址，先得到`video.example CNAME edge.example`，再得到`edge.example A 192.0.2.20`。第一个回答给出另一个名字，第二个才给出可连接的地址。它们可能一起出现在回答中，也可能需要继续查询；CNAME 不会把视频文件一起带回来。这些名称和地址仅作教学示例。

缓存变式 / Transfer： 解析器在 t=0 同时缓存上述两条记录，CNAME 的 TTL 为 300 s，A 的 TTL 为 60 s。忽略其他缓存与提前刷新，t=90 s 再次询问相同地址时，哪条记录仍有效、还缺什么？/ At t=0, a resolver caches the CNAME with TTL 300 s and the A record with TTL 60 s. With no other cache or refresh, what can it reuse at t=90 s and what must it obtain again?

答 / Answer： 别名关系还剩 210 s；A 记录已过期，需要重新取得`edge.example`的有效地址，不能假定旧 IP 仍可直接用。/ The CNAME has 210 s left. The A record has expired, so a valid address for edge.example must be obtained again.

课堂的查询/响应共用结构：ID 匹配事务；flags 说明请求/回答、是否希望递归、是否支持递归、是否权威等；counts 说明后续 questions、answers、authority、additional 的数量。附加区可以携带相关地址，减少后续查询，但不是任意附加记录都自动可信。

新公司例子：向注册商登记域和权威服务器，父区设置委派 NS，必要时有 glue 地址；在自己权威区添加 www 的地址记录。课件同页混用`networkuptopia/networkutopia`且 www 与 apex 不一致，是拼写/名称示意问题；真正配置时它们是不同 DNS 名字，必须统一，不能指望解析器猜出来。

课堂的 DDoS、缓存投毒与反射放大用来理解可靠性/信任边界。原 “Not successful to date” 描述的是旧课件编写时的情况。缓存降低某些上游访问压力，但不能证明 DNS 不可能遭攻击。

English: Recursive resolution delegates completion; iterative resolution returns referrals. DNS records have distinct types and TTLs; a non-authoritative cached answer is not automatically an incorrect answer.

来源：Chapter 2 完整课件，第 63–69 张幻灯片；别名链与 TTL 数字为教学补充。

10. P2P file distribution ｜新加入的人也可以带来上传能力

给理想模型：服务器有 F bit 文件，需分发到 N 个 peer，服务器上传 u_s bit/s，peer i 上传 u_i、下载 d_i，核心网络不额外限速，$d_{min} = \min_i d_i$。

Client-server 的服务器共需上传 NF bit，最慢客户端下载 F bit：

$$D_{cs} \geq \max\left(\frac{NF}{u_s}, \frac{F}{d_{min}}\right). \tag{2.1}$$

服务器不必物理 “顺序开完一个连接才开下一个”，NF/u_s 来自总上传工作量，哪怕并行也绕不过这个容量下界。

P2P 中的节点可以互助上传，但最初至少要有一份文件从源头出去；每个节点仍须下载 F，所有节点总共需 NF 的接收量：

$$D_{p2p} \geq \max\left(\frac{F}{u_s}, \frac{F}{d_{min}}, \frac{NF}{u_s + \sum_i u_i}\right). \tag{2.2}$$

图和公式是理想容量下界，实际调度、块稀缺、离线、协议开销会增加耗时。Chapter 2 完整课件，第 75 张幻灯片“always less”应读为在理想能力比较中不劣且有时更好，可能相等，不能保证任意实际 P2P 更快。

回到 Chapter 2 完整课件，第 75 张幻灯片原例：每个 peer 上传 u，F/u=1 小时，服务器 u_s=10u，所有下载速率至少 u_s。于是 CS 下界为 N/10 小时；P2P 下界为 max(0.1,N/(10+N)) 小时。N=10 时为 1 小时对 0.5 小时，N=100 时为 10 小时对约 0.909 小时；N=1 时两者均 0.1 小时。用户增加也带来上传资源，这解释了原图的扩展性趋势。

完整补算： F=100Mbit、N=4、u_s=10Mbps、每个 d_i=5Mbps、u_i=2Mbps。CS 下界 max(40,20)=40s；P2P 下界 max(10,20,400/18)=22.22s。分母总上传 18Mbps，不把下载速率也当上传资源加进去。

变式 / Transfer: 同条件但各 peer 上传 0，P2P 下界？/ Set every peer upload rate to zero; keep all other parameters.

答案 / Answer

max(10,20,40)=40s，与 CS 相同。/ 40 s, equal to the client-server bound. 这是“P2P 一定严格更快”的反例。

来源：Chapter 2 完整课件，第 71–75 张幻灯片。

11. BitTorrent ｜先找邻居，再交换稀缺块

一个 torrent 是交换同一文件分块的节点集合，tracker 帮助发现参与者，不必亲自发送全部文件。peer 加入时无块，逐渐下载并向别人上传；成员离开/加入称 churn。

请求块采用 rarest-first，优先扩散目前邻居中少见的块，降低最后只缺同一块的风险。发送方面，课堂用“偏向近期给自己较高上传速率的四个伙伴，每 10 秒重评、每 30 秒随机尝试一个新伙伴”的 tit-for-tat/optimistic unchoking 例子。这里的 4/10/30 属于课件所述算法版本，客户端实现可能采用不同参数。随机试新伙伴避免只锁在既有关系里。

具体看 Alice 刚加入、缺少 A/B/C 三个等大块的时刻。她从 tracker 得到邻居列表后，询问各邻居持有哪些块；假设 P1 持有 A、B，P2 持有 A、B，P3 持有 A、C。统计可见副本数：A 有 3 份、B 有 2 份、C 只有 1 份，因此先向持有 C 且愿意供给的 P3 请求 C。拿到后 Alice 也可以向别人上传 C，使稀缺块多一个来源。tracker 在这个过程中帮助找人，文件块来自 peer。

“先请求哪个缺失块”与“把自己的上传机会分给谁”是两项决定。rarest-first 回答前者，tit-for-tat 与 optimistic unchoking 回答后者。Bob 即使暂时没有给 Alice 上传，也可能被随机选中获得一段服务；这解释了 Tutorial 4 Q3（本册 p.75）为什么讨论 free-riding 的可能性，而不保证任意不上传节点都能下载完成。

独立题 / Check：保持上例，Alice 仍缺 A/B/C。P2 离开后，A、B、C 在剩余邻居中各有几份，下一请求是否唯一？若 P3 也离开，C 是否仍可立即下载？/ Alice still lacks A, B and C. After P2 leaves, count the visible copies and identify the rarest candidates. If P3 also leaves, is C immediately available from the remaining neighbors?

答 / Answer： P1 与 P3 共有 A 两份、B 一份、C 一份，B/C 并列稀缺，题设没有指定并列选择规则。P3 再离开后，剩余 P1 没有 C，须寻找其他供给或等待。/ The counts are 2, 1 and 1. B and C tie; no tie-breaking rule is given. Without P3, C is unavailable from the remaining neighbor, so Alice needs another source or must wait.

原图中的`256Kb chunks`表示一个分块大小的示例（Chapter 2 完整课件，第 76 张幻灯片）；实际分块长度随 torrent 选择，b/B 也需辨别；理解单位见基础页，不要求把这个旧示例当标准固定块长。

来源：Chapter 2 完整课件，第 76–80 张幻灯片。

12. Video, DASH and CDN ｜边下边播，需要的不只是平均带宽

解释视频由连续图像组成；空间冗余可压缩同一帧重复信息，时间冗余利用帧间变化。不同编码码率对应不同质量/数据量，但码率不是用户网络吞吐。“4Mbps 视频”指播放每秒需要约 4Mbit 编码数据，而不是网络必然提供 4Mbps。

从单个 HTTP 文件到 Dynamic Adaptive Streaming over HTTP（DASH）：服务器把视频切成时间块，同一块有多个码率版本，manifest 给出访问位置；客户端估计吞吐与缓冲状况，决定何时请求、用哪档、从哪里取。不能只用刚测到的瞬时峰值去选最高清，否则可能很快耗尽缓冲。

补算：4 秒片段、编码 2Mbps，块大小 8Mbit；可用吞吐 4Mbps 时下载 2 秒，若同时正常播放，缓冲时长净增加约 2 秒。若只有 1Mbps 则下载 8 秒，需要足够初始缓冲，否则会停顿。这里忽略请求/变码率等开销，按固定编码率计算；实际块大小可以变化。

自己算 / Try：一个 6 秒视频块按 3 Mbps 编码，可用吞吐 6 Mbps，初始缓冲充足且边下边播。求大小、下载时间和缓冲时长净增量。/ A six-second chunk is encoded at 3 Mbps and downloaded at 6 Mbps. With enough initial buffer and simultaneous playback, find its size, download time and net buffer gain.

答 / Answer： 18 Mbit、3 秒、净增 6−3=3 秒，忽略请求和解码开销。/ 18 Mbit, 3 seconds to download, and 3 seconds of net buffer gain, ignoring request and decoding overhead.

内容分发网络（content delivery network，CDN）把副本放在多个地理/网络位置，降低单站压力与长路径问题。Enter deep 靠近接入网络、地点多；bring home 集中于较少大型 IXP 附近集群。选择“近”往往看网络路径/负载，不只地图公里数。DASH 控制表示与请求，CDN 提供分发位置，二者可配合但不是同义词。原用户数、流量占比和部署地点数均为原页时期的案例。

一次播放请求怎样找到视频块

课堂末图把用户导向一份合适副本，但没有展开全部消息。下面把前面学过的 HTTP、DNS 与 manifest 接起来，给出一**种采用 DNS 导向的教学流程**。假设播放器已经取得视频页面，所需视频块已放在 CDN 节点上；示例域名不是实际服务，流程不要求所有网站都这样部署。

顺序	谁向谁发送什么	这一步得到什么
1	播放器向页面指定的服务请求 manifest	得到不同码率、时间块及其 URL 的描述；清单本身不是视频数据
2	播放器选中下一块的一档表示，解析该 URL 中的主机名	把“要哪个内容”接到“应连接哪个地址”
3	本地 DNS 解析器完成查询，CDN 的 DNS 系统为该名字给出合适的服务地址	CDN 可结合网络位置、负载等信息导向一个节点；也可先返回别名，再解析地址
4	播放器连接所获地址，按块 URL 发送 HTTP GET	选中的 CDN 节点返回这一时间块、这一档码率的数据
5	播放器将块加入缓冲，同时观察下载耗时与剩余缓冲	为下一块调整请求时机与码率；只有可用 URL 或导向机制支持时，才改取其他节点

第 3 步的 DNS 服务器只回答名字/地址信息，第 4 步的内容服务器才发送视频字节。一个地址还可以对应许多视频，具体对象由 HTTP 请求中的目标路径等信息区分。若后续块使用同一主机名、缓存地址仍有效且连接可复用，就不必每块重查完整 DNS 链、重建 TCP 连接。

换码率也不必换服务器：同一 CDN 节点可以同时保存低、中、高质量的对应块。反过来，换到另一个副本也可以继续请求同一码率。CDN 部署位置解决“从哪里供给”，DASH 的客户端决策解决“下一块何时请求、选哪档”等播放问题。若部署通过 HTTP 重定向导向节点，客户端会先收到新 URL，再向该目标请求；这是另一种实现，不应在上表中强行把两种机制都算成每次必经步骤。

来源：Chapter 2 完整课件，第 87–92 张幻灯片；DNS 与 HTTP 导向的机制补充参见 RFC 7336《Framework for Content Distribution Network Interconnection》§2.1，上述单次播放流程为教学整理。

保留服务器，只调码率，能否避免停顿？

独立变式 / Transfer：同一 CDN 节点提供 4 s 长的对齐视频块，码率可选 0.5、1、2 Mbps。下一请求开始时已有 3 s 可播放缓冲，实际下载吞吐稳定为 1 Mbps；边下边播，整块到齐才加入缓冲，无其他开销，也不切换节点。三档各下载多久、停顿多久？/ One CDN node offers aligned four-second chunks at 0.5, 1 and 2 Mbps. Start the next request with three seconds of playable buffer and a constant 1-Mbps download rate. Playback continues during download; each complete chunk becomes playable only on arrival. Ignore other overhead and keep the same node. Find download and stall times for each representation.

答案 / Answer

三档大小分别为 2、4、8 Mbit，所以下载 2、4、8 s。初始 3 s 缓冲能覆盖的等待有限：停顿分别为 0、1、5 s。选择 0.5 Mbps 档时下载结束前播掉 2 s，余 1 s，再加入 4 s 块，缓冲变为 5 s。选择 1 Mbps 虽然长期编码率等于吞吐，本题等待整块到达的条件下仍会先停 1 s。

Download times are 2, 4 and 8 s; stalls are 0, 1 and 5 s. At 0.5 Mbps the post-download buffer is 5 s. Matching the long-run encoding and download rates alone does not prevent the first stall with only 3 s of initial buffer.

音频与视频表示是否合并保存，另看 Tutorial 4 Q4（本册 p.75）的一一配对条件；表示数量、时间块数量和 CDN 副本数量是不同的计数对象。

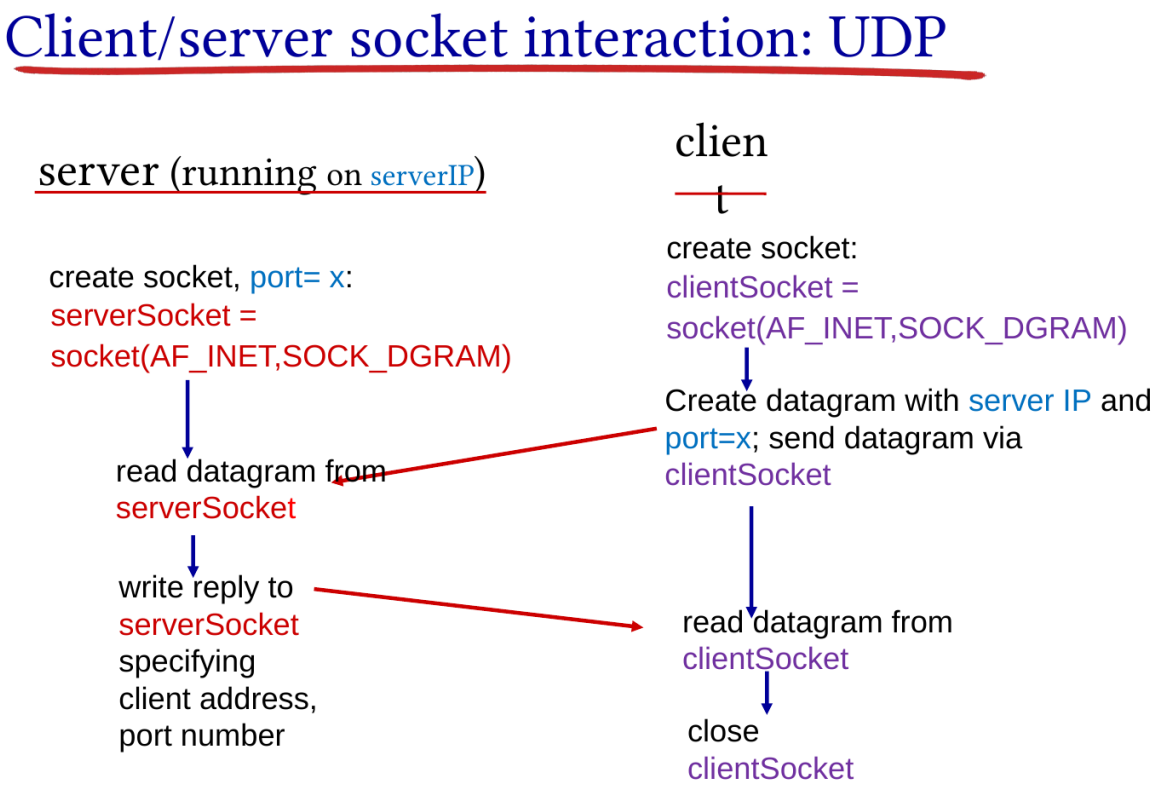
English: Encoding rate describes media demand, while throughput describes delivery capacity. DASH adapts chunk requests; CDNs distribute replicas across serving locations.

来源：Chapter 2 完整课件，第 82–84 张幻灯片；第 85–88 张幻灯片；第 89–92 张幻灯片。

13. Socket programming ｜把课堂的大写转换例子真正跑通

课堂的原任务：client 读一句文字 → server 转大写 → 回给 client。当前原代码含 Python2 `raw_input` /print 写法，以及`from socket import *`与`socket.AF_INET`混用。配套 socket_demo.py 使用 Python3、模块导入、显式 UTF-8 编码和超时，只绑定本机 127.0.0.1。

UDP 路径：`socket(AF_INET,SOCK_DGRAM)` → 服务端 bind → `client.sendto(data,address)` → `server.recvfrom`得到 bytes 和 client 地址 → sendto 回复 → client.recvfrom。它不做连接握手；收到回包不证明所有未来数据报都可靠。



TCP 路径：server 创建监听 socket、bind、listen；client 创建 socket 再 connect；server.accept 返回**新的连接 socket**；双方用该连接收发；结束关闭连接 socket，监听 socket 可继续接新连接。Chapter 2 完整课件，第 102 张幻灯片用源端口帮助区分连接，完整连接识别还涉及双方地址/端口与协议，不能只看一个源端口。

```
# 在本目录运行。两个终端；服务端只处理一次请求便退出。
python labs/socket_demo.py --transport udp --role server --port 12000
```

```
python labs/socket_demo.py --transport udp --role client --port 12000 --message 'hello
↪  network'
# TCP 只需把两边的 udp 同时改成 tcp。
python labs/socket_demo.py --self-test
```

TCP 没有消息边界：本例规定每条文本以换行结束，`sendall`发送全部 bytes，接收循环直到换行，不能假设一次 recv(1024) 一定收到整句。UDP 保留报文边界，但要保证例题报文在接收缓冲范围内。

运行`--self-test`可检查 UDP、TCP 和超过一次读取大小的 TCP 消息，并在本地`observations/socket-self-test.json`保存本次结果。它验证本机接口与消息分帧，不测量跨互联网可靠性或时延。

独立任务 / Try: 将客户端文字改成`course practice`，两种协议各运行一次；再解释 TCP 为什么有监听与连接两个 socket。/ Run both transports with this message, then explain the two server-side TCP sockets.

预期与解释 / Expected result

输出`COURSE PRACTICE`。监听 socket 接受新连接，连接 socket 处理当前客户端的数据，职责不同。/ The reply is COURSE PRACTICE; the listening socket accepts connections while the connected socket exchanges this client's data.

来源：Chapter 2 完整课件，第 94–105 张幻灯片。

14. Back to the course ｜用一条请求链把本章连起来

打开网页时：DNS 得到服务地址 → socket 取得所选传输服务 → HTTP 表达所需资源 → 响应中的 cookie 可关联状态、cache 可复用内容。邮件用 SMTP 投递、POP3/IMAP/HTTP 访问；大文件与视频则进一步考虑 peer 上传能力、分块、编码与分发位置。共同的问题始终是：**双方是谁、消息怎么写、何时发送、需要哪些服务、状态由谁保存**。

接着做 Tutorial3（本册 p.67）的报文/DNS/HTTP 题和 Assignment1（本册 p.86）的 POP3 与 Zoom 分析。首轮可选 **N100（连接复用）**、**N118（Webmail 路径）**、**N124（DNS 查询模式）**、**N233（HTTP 时序）**，在 Markji 按卡号定位。相关微课 NET05 继续可用。

英文自测：**Explain why persistent HTTP does not necessarily imply pipelining. / 为什么持久 HTTP 不等于流水线？**关键答：复用的是连接，流水线另决定是否在上一响应完成前发后续请求。/ Persistence reuses a connection; pipelining additionally overlaps outstanding requests.

课堂 Q&A：按原题判断并说明条件

Q&A_cha2 学生版第 2–7 张幻灯片有六题，未附教师解答。下面保留完整选项，答案根据本章机制整理。

1. Transport service ｜为应用选择传输服务

EN: Which services should use UDP? A) File transfer. B) Video streaming. C) Email. D) Online games.

中文：哪些服务适合使用 UDP？A 文件传输；B 视频流；C 邮件；D 在线游戏。

答 / Answer：课堂对时延敏感服务的简化模型选 B、D；实际协议还要结合应用设计，例如视频也可能使用 HTTP/TCP。/ B and D under the classroom model of latency-sensitive services; real applications may choose other transports, including HTTP/TCP for video.

2. HTTP timing ｜从 HTML 到两张图片

EN: A browser obtains an HTML page and two very small JPEG images from the same server using non-persistent HTTP. What is the total response time? A) 2 RTTs. B) 3 RTTs. C) 6 RTTs. D) 8 RTTs.

中文：浏览器从同一服务器获取 HTML 和两张很小的 JPEG，采用非持久 HTTP，总响应时间是 A 2、B 3、C 6 还是 D 8 个 RTT?

答 / Answer：按课件顺序获取、忽略 DNS 与对象传输时延的约定，三个对象各需 2RTT，共 6RTT，选 C。原题未明确并行性；并行连接时答案要重算。/ C: six RTTs for serial requests, ignoring DNS and object transmission time. Parallel connections would change the timing.

3. Mail addressing ｜目标主机与服务端口

EN: What addressing information does a client process need to communicate with the mail server at abc.com? A) Its IP address, usually obtained through DNS. B) Its IP address, which the client usually already knows. C) Its port number, usually obtained through DNS. D) Port number 25.

中文：客户进程联系 abc.com 的邮件服务器，需要哪些地址信息？A 通常经 DNS 得到 IP；B 客户端通常已知道 IP；C 通常经 DNS 得到端口；D 端口 25。

答 / Answer：按本题 SMTP 投递情境选 A、D：IP 通过 DNS 确定，端口 25 来自协议与配置。它不代表所有邮件客户端操作都用 25；提交、收信另有服务。/ A and D for SMTP delivery: resolve the server's IP address and use port 25. Submission and mailbox access are different services.

4. DNS records ｜一个名字怎样对应多台服务器

EN: Which DNS record type can enable load balancing in this question? A) A. B) NS. C) CNAME. D) MX.

中文：哪类 DNS 记录可实现本题所指的负载分配？A A 记录；B NS；C CNAME；D MX。

答 / Answer：选 A。同一名字可返回多个 IPv4 地址，由客户端或返回顺序影响选择；它本身不保证各服务器负载精确相同。/ A: multiple addresses can be returned for one name, although this alone does not ensure perfectly balanced load.

5. BitTorrent ｜找到谁、向谁下载

EN: Which statement about BitTorrent is true? A) It uses a client-server architecture. B) A user needs the IP addresses of peers. C) Missing chunks with the largest size are requested first. D) A user always communicates with the same peers throughout the download.

中文：哪句正确？A 采用 client-server 架构；B 需要 peer 的 IP 地址；C 优先请求最大的缺失块；D 下载全程始终联系同一批 peer。

答 / Answer：选 B。BitTorrent 的数据分发是 P2P，常用稀缺块优先，连接的 peer 可以变化；发现 peer 可以借助 tracker 等机制。/ B. Data distribution is peer-to-peer, chunk selection commonly uses rarest-first, and peers may change during a download.

6. Socket services ｜接口提供了什么

EN: Which statements are true? A) TCP guarantees that bytes arrive in order. B) UDP performs a handshake before sending data. C) With UDP, the sender explicitly supplies the destination IP address and port for each packet. D) Socket programming selects the network-layer protocol.

中文：哪些说法正确？A TCP 提供可靠且有序的字节传输；B UDP 发数据前握手；C UDP 发送时逐包给目的 IP 和端口；D socket 编程选择网络层协议。

答 / Answer：课件基本服务与`sendto`接口模型选 A、C。TCP 向应用交付有序字节，但连接失败时不能保证最终成功；C 描述本章未连接 UDP 的发送接口。D 不能理解为应用通过本章接口任意指定沿途网络协议。/ A and C in the taught service/API model. TCP delivers ordered bytes but can report connection failure; C describes the unconnected UDP sendto interface used here.

来源：Chapter 2 课堂 Q&A，第 2–7 张幻灯片，英文按原题整理。

迁移 / Transfer：把 Chapter 2 课堂 Q&A，第 3 张幻灯片改为 HTML 后可并行取两张图，非持久连接且理想无竞争，总需几 RTT？/ After receiving the HTML, fetch its two small images through parallel non-persistent connections. Ignore DNS and transmission time, assume no contention, and express the total in RTTs.

答案 / Answer

HTML 两 RTT，图片同一批两 RTT，共 4RTT；这是补充变式，不替换原题顺序获取约定。/ Four RTTs under the stated idealized parallel model.

Tutorial4（本册 p.73）继续练本章 P2P/DASH；Tutorial5（本册 p.77）属于下一章传输层。Q&A 页面并入本章覆盖索引：Chapter 2 课堂 Q&A，第 1 张幻灯片为封面，Chapter 2 课堂 Q&A，第 2–7 张幻灯片对应以上六题。

15. Source coverage and cautions ｜每一页有去处

完整 PPT 页	对应正文	核对重点
1–3	开篇	来源、目标、应用例
4–12	§1–2	架构、进程、socket、寻址、语法语义
13–17	§2	服务需求、TCP/UDP、旧 SSL 表述
18–22	§3	目录、对象、HTTP/TCP 和无状态

完整 PPT 页	对应正文	核对重点
23–29	§3	非持久/持久/并行、RTT 约定
30–31	§3	完整请求与字段
32–37	§4	cookie 四部件、proxy/cache
38–45	§5	目录、邮件结构、SMTP 原事务、信封/内容
46–51	§6	POP3 三阶段、keep/delete、IMAP、webmail
52–59	§7	目录、DNS 服务、层级与 local resolver
60–63	§8	递归/迭代的两种图、TTL/cache
64–69	§9	RR、消息、注册/委派、攻击概念
70–75	§10	目录、P2P 架构、分发下界与原例
76–80	§11	tracker、块、churn、rarest-first、tit-for-tat
81–88	§12	目录、视频编码、HTTP 流媒体、DASH
89–92	§12	CDN 两种布局、ISP/IXP 回接
93–105	§13	目录、任务、UDP/TCP 流程与全部原代码含义
106–107	§14–15	协议共同主题、控制/数据、状态、可靠性、边缘复杂性

part1 对应完整 1–22，part2 对应 23–65，part3 对应 66–107；内容核对忽略明确的页脚重编号。课件中的旧 RFC、客户端与数字按原例保留；协议版本差异在正文就近说明。FTP 独立正文和后续未下载章节列为待补材料。

Chapter 3 · Transport Layer（当前 part 1–2）

数据到了电脑，还要交对进程、补齐缺口

课程目录 · 上一章：应用层（本册 p.24）· Tutorial 5（本册 p.77）

同一台电脑可以同时开浏览器、邮件和游戏。网络把数据送到这台电脑之后，还要知道交给哪一个程序。即使地址写对，途中也可能损坏、丢失或重复；收到半段文件不能直接说“差不多齐了”。这一章从进程寻址开始，逐步增加可靠传输需要的机制。

依据当前 part1 的 44 页和 part2 的 26 页。两份课件的幻灯片序号各自从 1 开始。目前整理至 TCP 的 RTT 与超时；流控、连接管理及拥塞控制待后续材料补充。

基础补课（可跳过）： bit、byte 与速率（基础册 p.37）、二进制（基础册 p.38）、时间轴（基础册 p.40）。阅读时留意“第几个包”和“第几个字节”的区别，§9 会具体比较。

主题	优先级	难度与卡点	掌握要求及依据
传输服务、复用/解复用	核心必会	中，主机/进程与端点	读 UDP/TCP 标识与回程端口；Chapter 3 part 1，第 4–15 张幻灯片、Tutorial 5 Q1–2
UDP 与校验和	核心必会	中，回卷进位	手算、解释检测边界；Chapter 3 part 1，第 17–20 张幻灯片、Tutorial 5 Q3
rdt1.0–3.0	核心必会	高，双方状态与坏 ACK	逐事件解释、画乱序反例；Chapter 3 part 1，第 22–44 张幻灯片、Tutorial 5 Q4
停等与流水线	核心必会	中，忙碌时间/周期	计算利用率与吞吐；Chapter 3 part 2，第 1–6 张幻灯片
GBN 与 SR	核心必会	高，ACK 与窗口	画一次丢包后的行为；Chapter 3 part 2，第 7–15 张幻灯片
TCP 序号与 RTO	核心必会	中，字节与估计量	算下一序号、累计 ACK、超时；Chapter 3 part 2，第 17–26 张幻灯片
完整握手/流控/拥塞控制	后续待当前材料	不以“尚缺”推断不考	暂留历史预习，不补成教师已讲

1. Transport service ｜ IP 找到主机，传输层找到应用端点

先区分网络层的主机间逻辑通信与传输层的进程间逻辑通信。以发送一段应用数据为例：应用层称它为 message（消息）；传输层封装后常称 segment（报文段）；再封装进网络层的 datagram（数据报），通过链路层 frame（帧）送到下一站。各层名称强调不同的封装，日常说 packet 时则需要看上下文。

TCP 提供连接上的可靠有序字节流以及流控、拥塞控制；UDP 提供较简单的数据报服务。可靠不表示网络永久中断也能永远交付，不表示有固定到达期限，也不等于加密。

自测 / Check：一段数据已经到达正确 IP 地址，是否已确定交给哪一个浏览器连接？/ Does arrival at the correct host IP identify a specific browser connection?

答案 / Answer

没有。还需传输协议与端口等信息；TCP 连接区分还涉及双方 IP 和端口。/ No. Transport endpoint information is still needed; established TCP connections use both address-port pairs.

来源：Chapter 3 part 1，第 4–7 张幻灯片。

2. Multiplexing and demultiplexing ｜同一栋楼有许多门

发送端复用，把多个 socket 的数据交给网络；接收端解复用，看首部把数据送到相应 socket。端口是传输端点标识的一部分，要与 IP 地址和协议一起理解。

先看一个普通、未连接的 UDP 接收 socket：目的 IP 与目的端口标识接收 socket，不同来源的数据报可以到同一个 UDP socket。Chapter 3 part 1，第 13 张幻灯片的已建立 TCP 连接按四元组区分：源 IP、源端口、目的 IP、目的端口。因此服务器端同一个 80 端口可同时服务多个连接，只要四元组不同。

跟做 / Worked：沿用 Tutorial 5 Q2 的端口：C:7532→B:80、C:26145→B:80、A:26145→B:80 是三条不同 TCP 连接。回程分别 B:80→C:7532、B:80→C:26145、B:80→A:26145。后两者客户端端口相同，客户端 IP 仍区分连接。/ Using the ports in Tutorial 5 Q2, reverse both address-port pairs for the return direction.

实现补注：这里比较的是课堂的普通 UDP 接收与已建立 TCP 连接。监听 socket、已接受连接及 connected UDP 的接口细节，可在实际编程时进一步区分。

变式 / Transfer：两客户端在不同 IP 上都使用端口 50000，连接同一个服务器 443 端口，是否冲突？/ Two clients with different IP addresses both use source port 50000 to connect to the same server IP and port 443. Do the established TCP connections conflict?

答案 / Answer

四元组不同，不因客户端端口数值相同而冲突。/ No: their source IP addresses distinguish the four-tuples.

来源：Chapter 3 part 1，第 9–15 张幻灯片；端口数值沿用 Tutorial 5 Q2，不与课件第 14 张的另一组端口混用。

3. UDP ｜少做承诺，也少保存状态

UDP 不先进行 TCP 式握手，也不内建重传、排序和拥塞控制。应用可以在其上增加所需功能。需要及时性的应用可能愿意舍弃过时数据，但“视频”这个标签不能唯一确定它使用 UDP，HTTP 自适应视频就是不同情境。

UDP 首部四个 16bit 字段为 source port、destination port、length、checksum，合计 8byte。length 包含 UDP 首部与载荷，不包含外层 IP 首部。100byte 数据的 UDP length 是 108byte。

English takeaway: UDP preserves datagram boundaries but does not provide TCP's built-in reliable ordered byte-stream service. Its length field counts the UDP header and payload.

来源：Chapter 3 part 1，第 17–18 张幻灯片。

4. Internet checksum ｜算得对不等于能发现一切错误

校验和采用 16bit 反码加法。先把校验字段置 0，把保护的数据组成定长字，逐字相加；超出 16bit 的进位卷回最低位，再按位取反。接收端把含 checksum 的字也相加，正确结果应为全 1。

跟做 / Worked： `0xFFFF+0x0001=0x10000`。保留低 16 位 0，最高进位 1 卷回，得到 1；取反得到 `0xFFFE`。接收端相加卷回后为`0xFFFF`。普通“溢出直接丢掉”会得到另一个错误答案。

Tutorial5（本册 p.79）改用 8bit 以便手算，这是题目简化，不能把真实 UDP 首部缩成 8bit。单比特翻转可被此校验发现；两个适当位置的变化可能相互抵消。检验通过不证明数据来源可信，也不等于密码学完整性认证。

自测 / Check： 8bit 简化模型中，`11111111+00000001`的反码校验和是多少？/ In a toy 8-bit model, compute the one's-complement checksum for words 11111111 and 00000001, including end-around carry.

答案 / Answer

回卷后 00000001 , 再取反 11111110。/ End-around carry yields 00000001, then complement to 11111110.

来源：Chapter 3 part 1，第 19–20 张幻灯片。

5. rdt1.0 到 rdt2.2 ｜每多一种故障，就多一项必要机制

假设发送应用要依次交付数据 A、B。这里先只传一个方向的应用数据，确认消息仍会反向流动。发送端看不到接收端内存，只能从收到的反馈推测对方进度；接收端也不知道自己发出的 ACK 有没有被正确收到。协议必须在这种信息不对称下避免漏交、错交和重复交付。

FSM 的事件、条件与动作

应用调用`rdt_send(data)`把数据交给发送协议；协议用`udt_send(packet)`把包装好的包交给底层通道。某个包到达时，产生`rdt_rcv(packet)`事件；接收协议只有确定它是正确的新数据，才调用`deliver_data(data)`交给上层。

有限状态机（finite-state machine，FSM）的圆圈记录“现在等什么”。每条箭头读成：**当前状态下发生上方事件、满足相应条件，就执行下方动作，再进入箭头终点状态**。&&表示条件同时成立，||表示至少一个成立。箭头绕回自身表示状态不变，仍可有发送反馈或重传等动作；Λ 表示不执行额外动作，不表示忽略了所有状态转移。

rdt1.0：信道可靠，双方各一个状态

底层不损坏、不丢失，按课堂的正常交付顺序传送。发送端始终“等上层调用”：收到 A 就`make_pkt(A)`、`udt_send`，随后仍等上层，可接 B。接收端始终“等下层到包”：收到包就提取数据并交付，随后继续等包。没有需要恢复的故障，因此不用 ACK、序号或定时器。

rdt2.0：先解决数据损坏，再看反馈损坏

加入比特损坏后，接收端用 checksum 检测，正确时交付并回 ACK，损坏时不交付并回 NAK。发送端必须保存本轮的`sndpkt`，否则收到 NAK 也没有原包可重传。这个版本暂假定反馈能被正确识别，且包不丢失。

当前状态	收到的事件/条件	动作	下一状态
发送端：等上层	上层给新数据	封装并保存、发送	等 ACK 或 NAK
发送端：等反馈	收到 ACK	本轮完成，允许下一份数据	等上层
发送端：等反馈	收到 NAK	重发保存的原包	仍等反馈
接收端：等下层	数据包校验通过	提取、交付、回 ACK	仍等下层
接收端：等下层	数据包损坏	不交付、回 NAK	仍等下层

正常时，A 到达、交付、ACK 返回，发送者才接 B。异常时，损坏的 A 到达、NAK 返回，发送者重发 A；这次校验通过后才交付 A 并确认。checksum 只负责识别损坏，真正恢复靠反馈和重传。

问题出在 ACK/NAK 也可能损坏。发送者看到一条读不懂的反馈，无法分清“接收端交付 A 后，ACK 坏了”和“接收端拒绝坏 A 后，NAK 坏了”。不重发可能永远漏交；直接重发又可能把已经交付的 A 交第二次。接收端不能仅看内容是否相同，因为应用本来就可能合法地连续发送两份相同数据。

rdt2.1：序号记录的是这一轮数据身份

给数据包加入交替的 0/1 序号。重传 A 仍用同一个编号，只有确认 A 完成，下一份新数据 B 才换号。发送端有四个状态：“等上层提供 0 号数据”“等 0 号包反馈”“等上层提供 1 号数据”“等 1 号包反馈”。接收端有两个状态：“等 0”“等 1”。

发送端在等某号包反馈时，收到正确 ACK 就进入另一编号的等上层状态；收到 NAK 或损坏反馈，就重发保存的原包，状态不变。原 rdt2.1 图的 ACK/NAK 消息尚未显式携带被确认的包序号，状态名中的 0/1 表示发送者正在等待哪一轮。

接收端当前状态	到达的数据包	是否交付	发什么反馈、下一状态
等 0	正确的 0	交付一次	ACK，改等 1
等 0	正确的 1（上一轮副本）	不交付	ACK，仍等 0
等 0	损坏	不交付	NAK，仍等 0
等 1	正确的 1	交付一次	ACK，改等 0
等 1	正确的 0（上一轮副本）	不交付	ACK，仍等 1

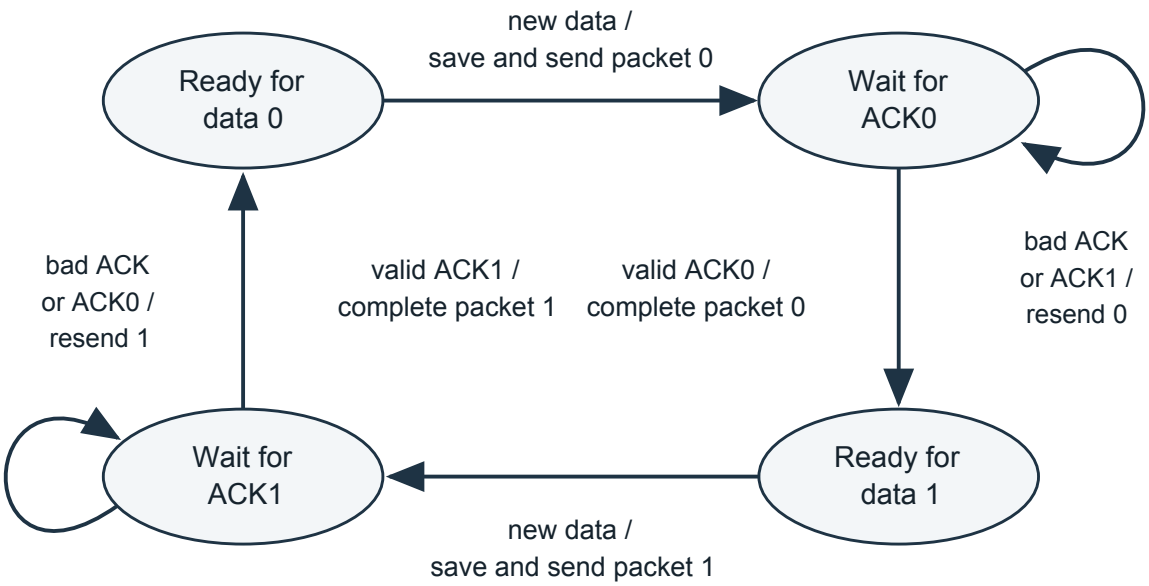
接收端当前状态	到达的数据包	是否交付	发什么反馈、下一状态
等 1	损坏	不交付	NAK，仍等 1

走一遍“ACK 损坏”：A(0) 正确到达，接收端交付 A 并改等 1；ACK 途中损坏，发送者仍等 0 号反馈，重发 A(0)。接收者此时等 1，认出 0 是上一轮副本，只再发 ACK，既不交付也不切回等 0。第二个 ACK 成功到达后，发送者才接收并发送 B(1)。双方在中途分别等不同编号，正是反馈尚未同步的正常情况。

rdt2.2：让 ACK 带编号，便可去掉 NAK

接收者改成“确认最后一个正确收到的包”。等待 0 时，正确 0 到达就交付、回 ACK0、改等 1；损坏包或重复 1 到达，就回 ACK1、仍等 0。等待 1 时完全对称：正确 1 触发交付与 ACK1，其他情况回 ACK0。初始等 0 时，可把保存的上一确认初始化为 ACK1，表示 0 尚未被接受，并不声称应用已经收到过一份真实的 1 号数据。

发送者等 ACK0 时，只把校验通过的 ACK0 视为完成；ACK1 或损坏 ACK 都使它重发 0，继续等 ACK0。等 ACK1 时交换 0/1 即可。图中从“等 ACK0”到“等上层 1”的箭头只允许接下一份数据，不会在应用尚未提供数据时凭空发出 1 号包。



rdt2.2 sender: event / action; successful ACK enables the next application call

按 Chapter 3 part 1 第 38–39 张整理的完整发送端图。箭头写“事件 / 动作”；Ready 状态等应用，Wait for ACK 状态等反馈。异常反馈沿自环重传，成功反馈才切换数据编号。接收端的两状态规则见上文。

例如双方正在处理 B(1)，但它到达时损坏：接收者仍等 1，回 ACK0；发送者正在等 ACK1，看见 ACK0 便重发 B(1)。ACK0 说的是“上一次正确收到的是 0”，并没有对损坏的 1 作成功确认。这种单个重复 ACK 触发的重传属于 rdt2.2 模型，不是后续 TCP 的重复 ACK 算法。

独立题 / Check： rdt2.2 中 A(0) 已正确交付，接收者改等 1，但 ACK0 在途中损坏。随后重传的数据与反馈都正确到达，且无丢失或重排。依次写出发送者动作、接收者动作及双方最终状态。/ In rdt2.2, A(0) is delivered and the receiver waits for 1, but ACK0 is corrupted. The retransmitted data and its reply then arrive intact, with no loss or reordering. State both endpoints' actions and final states.

答案 / Answer

发送者读不懂 ACK，重发 A(0)，仍等 ACK0。接收者等 1，丢弃重复数据但补发 ACK0，仍等 1。发送者收到正确 ACK0 后进入“等上层 1”。A 只交付一次。/ The sender retransmits A(0) and keeps waiting for ACK0. The receiver does not redeliver it, sends ACK0 and keeps waiting for 1. The valid ACK0 moves the sender to ready-for-data-1; A is delivered once.

English takeaway: Checksums detect certain damage, acknowledgments report receiver state, and sequence numbers distinguish new data from retransmissions. Each mechanism addresses a different ambiguity.

来源：Chapter 3 part 1，第 22–28 张幻灯片（接口与 rdt1.0）、第 29–34 张（rdt2.0 及反馈歧义）、第 35–37 张（rdt2.1）、第 38–39 张（rdt2.2）。第 39 张接收端只画片段，指向“等 0”的正确 1 号包箭头来自省略的“等 1”状态，不能读成在等 0 时交付 1。

6. rdt3.0 ｜坏消息会来，丢掉的消息可能一直不来

现在让模型再面对一种故障：数据或 ACK 可能丢失。没有反馈时不能永远等，因此发送方启动 timer，超时重发当前包。收到当前包的正确 ACK 后停 timer 并进入下一编号；损坏或错误编号的 ACK 不能错误推进。

与 rdt2.2 相比，课堂 rdt3.0 的**接收端沿用原来的 ACK 编号规则，发送端改用超时控制重传**。以正在等 ACK0 为例：

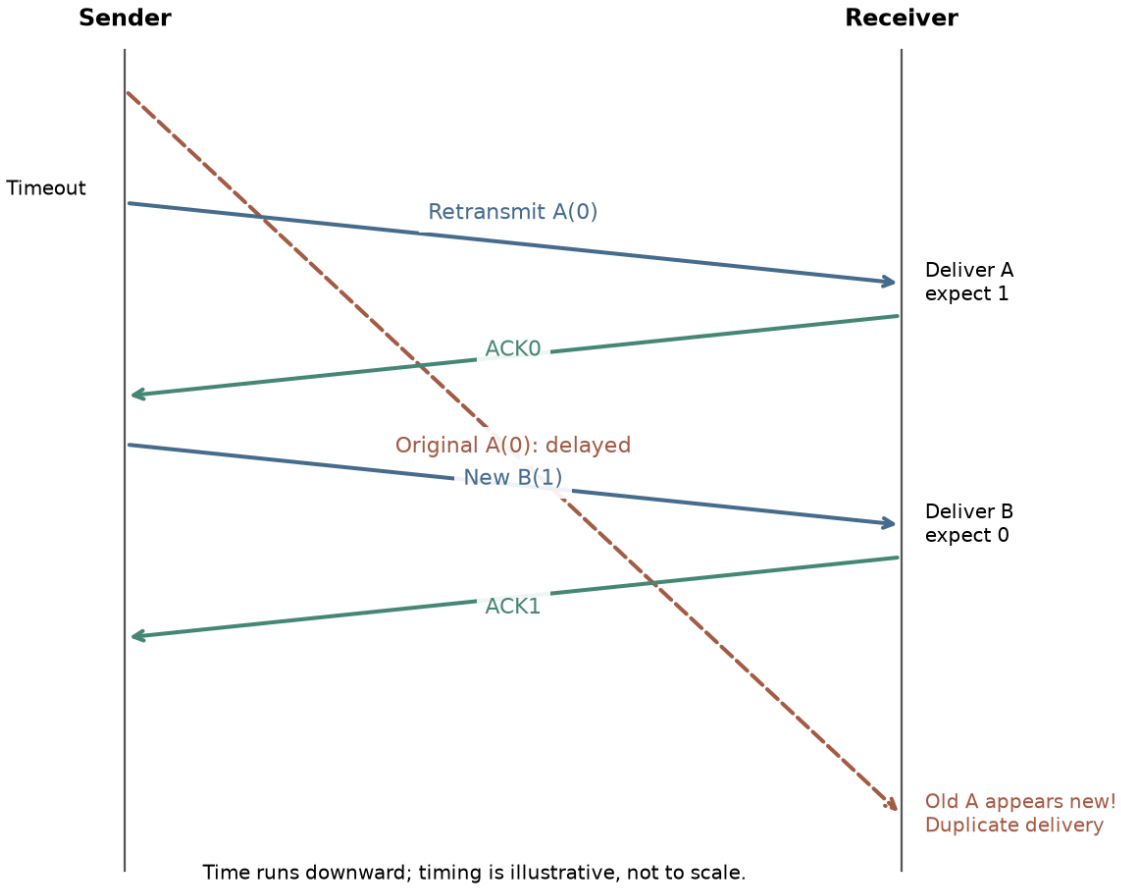
事件	发送端动作	状态与定时器
正确 ACK0 到达	停止定时器，本轮完成	改为等上层提供 1 号数据
ACK 损坏或收到 ACK1	不重传、不推进编号	仍等 ACK0，原定时器继续运行
定时器超时	重传保存的 0 号包，重新启动定时器	仍等 ACK0

所以“收到坏 ACK 就立即重传”不能从 rdt2.2 直接抄到本节。发送 0 号包时启动定时器，即使数据或 ACK 完全消失，没有任何到包事件，超时事件仍能打破等待。

发生什么	发送者看见什么	必要行为
数据包丢失	没有对应 ACK	超时重传
ACK 丢失	也没有对应 ACK	超时重传；接收端辨重复，不重复交付
ACK 只是很慢	可能提前超时	正确处理重复包和旧 ACK

一次只允许一个未确认包在途，称 stop-and-wait（停等）。例如数据 0 已交付、ACK0 丢了：发送者超时重发 0；接收者此时期待 1，认出它是重复包，只补发 ACK0，不再次交付。这把定时器、序号

和 ACK 三项连成了一次恢复过程。0/1 足以在原可靠性模型中区分相邻轮次，但若旧副本可任意延迟到编号重用之后，接收方可能把它当新包。这个边界正是 Tutorial5 Q4 要求画图说明的，不能把“序号只有两种就够”背成无条件结论。



图为补充推导，时间向下。原 0 包被延迟，其重传先到；随后 1 包也完成。接收方又等 0 时，最早那份旧 0 才到。内容已经过期，但编号恰好符合期望，因此可能被重复交付。

自测 / Check： rdt3.0 发送者等 ACK1 时收到校验正确的 ACK0，定时器尚未超时。应马上重发 1、改发新 0，还是继续等？ / An rdt3.0 sender waiting for ACK1 receives a valid ACK0 before timeout. Should it retransmit 1 immediately, send new data 0, or keep waiting?

答 / Answer： 继续等 ACK1，保持原定时器；超时后才重传 1。/ Keep waiting with the existing timer. Retransmit packet 1 only if that timer expires.

来源：Chapter 3 part 1，第 40–44 张幻灯片。

7. Stop-and-wait performance ｜忙了 8 微秒，等了 30 毫秒

看课堂的停等传输例子：R=1Gbps，L=8000bit，单程传播 15ms，RTT=30ms，忽略 ACK 传输和处理。发送时间 $L/R = 8\mu s = 0.008ms$。一轮从开始发包到 ACK 返回，耗 $RTT + L/R$。

$$U = \frac{L/R}{RTT + L/R} = \frac{0.008}{30.008} \approx 0.000266596. \tag{3.1}$$

发送端忙碌比例约 0.02666%，有效吞吐 $RU \approx 266596$bit/s，即 0.2666Mbps。链路名义上 1Gbps，却大部分时间在等。这是利用率，不是丢包率。

流水线允许 ACK 回来前连续发 N 个包。理想无损模型的利用率为

$$U_N = \min\left(1, \frac{NL/R}{RTT + L/R}\right). \tag{3.2}$$

窗口越大，可填补等待；但不能超过 100%。本例达到满利用需 $N \geq 3751$。这是忽略其他瓶颈与 ACK 成本的模型结果，不是给真实 TCP 直接配置 3751 就必然满速。

迁移 / Transfer： 同样 RTT 与包长，R 改为 100Mbps，停等吞吐是否变为原来的十分之一？ / Recompute after reducing R to 100 Mbps.

答案 / Answer

$L/R = 0.08ms$，$U = 0.08/30.08$，吞吐约 0.2660Mbps，仍主要受等待限制。 / About 0.2660 Mbps; stop-and-wait remains dominated by the RTT.

来源：Chapter 3 part 2，第 1–4 张幻灯片。

8. GBN 与 Selective Repeat ｜后面的包到了，要不要先收好？

流水线传输有两种值得比较的做法。Go-Back-N（GBN，回退 N 步）发送者可有 N 个未确认包，base 表示最早未确认序号，nextseqnum 表示下一新包序号。累计 ACK(n) 表示直到 n 都已收齐。基本模型只给最早未确认包设 timer，超时从 base 重传所有已发未确认包；接收方丢弃乱序包，重复确认最后连续收到的包。

Selective Repeat（SR，选择重传）分别确认正确收到的包，缓存窗口内乱序包，每个未确认包有自己的超时逻辑。某包超时只重传该包；应用仍需按序交付，不能因为缓存里有后段就跳过前段缺口。

窗口怎样随 ACK 向前移动

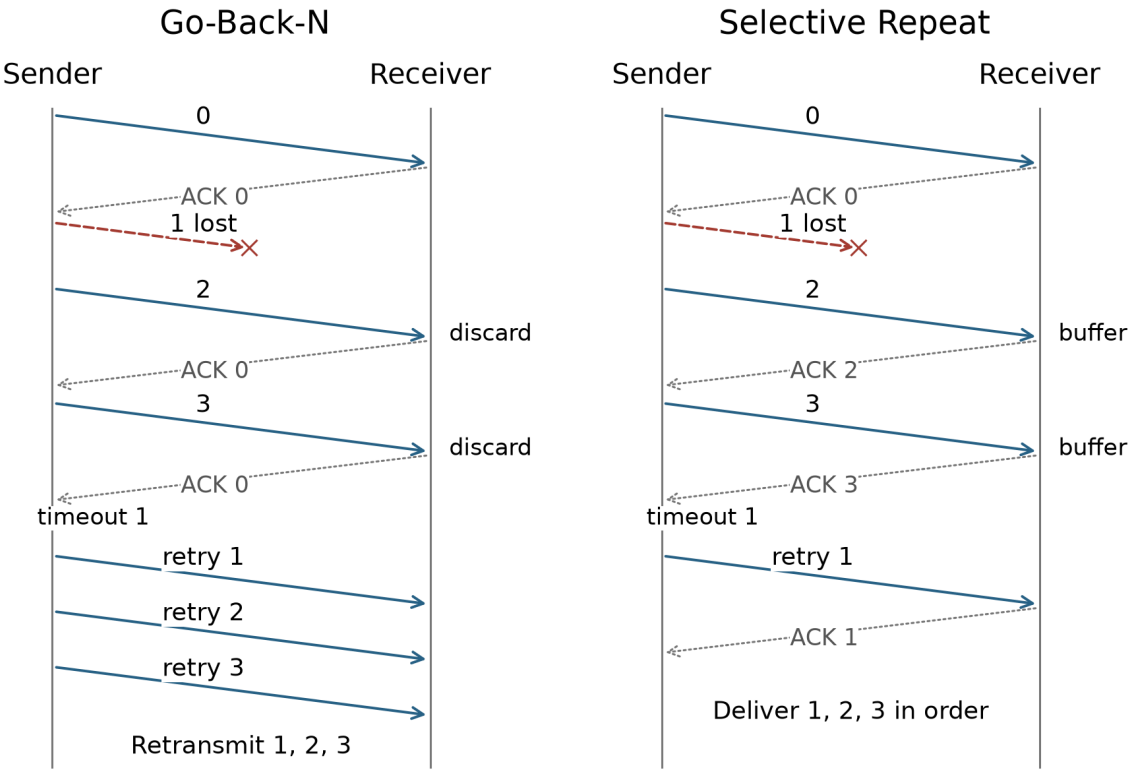
设发送窗口 N=4、序号暂不回绕，发送前 base=nextseqnum=0。允许使用的编号为 0–3；发完四包后 nextseqnum=4。下一新包必须满足`nextseqnum < base+N`，所以此时不能再发 4。这里窗口覆盖 N 个连续编号，不是“任何位置凑齐 N 个未确认包”就可以任意跳过缺口。

GBN 收到 ACK0，便知道截至 0 已连续收齐，base 移到 1，窗口变为 1–4，可以发送 4。再收到 ACK2，意味着 1 和 2 也已收齐，即使 ACK1 没有单独到达，base 也可移到 3；只要没有其他新包，最早未确认的是 3，定时器继续为它服务。

SR 收到 ACK0 后同样可把 base 移到 1。但若接着只收到 ACK2、ACK3，发送者仅把 2、3 标成已确认，base 仍卡在 1，窗口仍为 1–4。等 ACK1 补上，已确认的连续前缀才一次越过 1、2、3，base 移到 4。接收端也有类似边界：2、3 可以先缓存，应用须等缺失的 1 补齐后才能依次收到 1、2、3。

SR 接收端对当前窗口内正确包确认并按需缓存；对上一窗口内的已交付重复包，再发相应 ACK 而不重复交付，帮助发送者从 ACK 丢失中恢复。其他窗口外的包按课堂模型忽略。发送者收到一个较大编号的 SR ACK，只知道那个包收到了，不能把它当成 GBN 式的累计承诺。

同一场景 / Worked：窗口足以容纳四包，序号不回绕；依次发 0、1、2、3，只丢数据 1，没有其他丢失或重排。/ Both protocols have sufficient window space, with no sequence-number wraparound. Send packets 0–3; only packet 1 is lost, and all other data and ACKs arrive without reordering. GBN 收到 0 后 ACK0，收到 2 和 3 均丢弃并 ACK0，超时通常重传 1、2、3。SR 可缓存 2 和 3 并分别 ACK2/ACK3，等 1 补上后连续交付 1、2、3。这是包序号，尚不是 TCP 字节 ACK。



Event order runs downward; not a measured RTT scale

教学时序图：两边都只发送 0–3 这一批，窗口足够大，只有数据 1 丢失，ACK 均成功，未发生序号回绕。纵向表示事件顺序，不按真实 RTT 比例。GBN 丢弃 2 和 3 后重传 1–3；SR 先存好 2 和 3，只补 1。图中省略了 GBN 重传后的正常累计 ACK。

独立题 / Check：第一批发 0–7，只有数据 6 丢失，无重排、ACK 无丢失，窗口足够发完这批，序号不回绕。在任何重传开始前，GBN 与 SR 接收端分别发哪些 ACK？/ In the first batch, send packets 0–7 with a sufficiently large window and no sequence-number wrap. Only data packet 6 is lost; there is no reordering or ACK loss. List each receiver's ACKs before any retransmission.

答案 / Answer

GBN：0,1,2,3,4,5,5。SR：0,1,2,3,4,5,7。总共 7 个，因为丢失包没到达，不产生 ACK。/ Seven ACKs in each case; their final value differs.

序号空间必须足够，避免旧包与新包混淆；一般 SR 窗口限制的完整论证属于补充，可回现有微课。当前 Chapter 3 part 2，第 13–15 张幻灯片侧重窗口、缓存与逐包确认。

来源：Chapter 3 part 2，第 5–15 张幻灯片。

9. TCP 序号｜从“第几个包”切换到“下一个字节”

TCP 首部记录了通信和控制所需的信息：源/目的端口、序号、确认号、接收窗口、校验和、控制标志等。MSS 是每段可携带的数据上限，不包含所有协议首部。

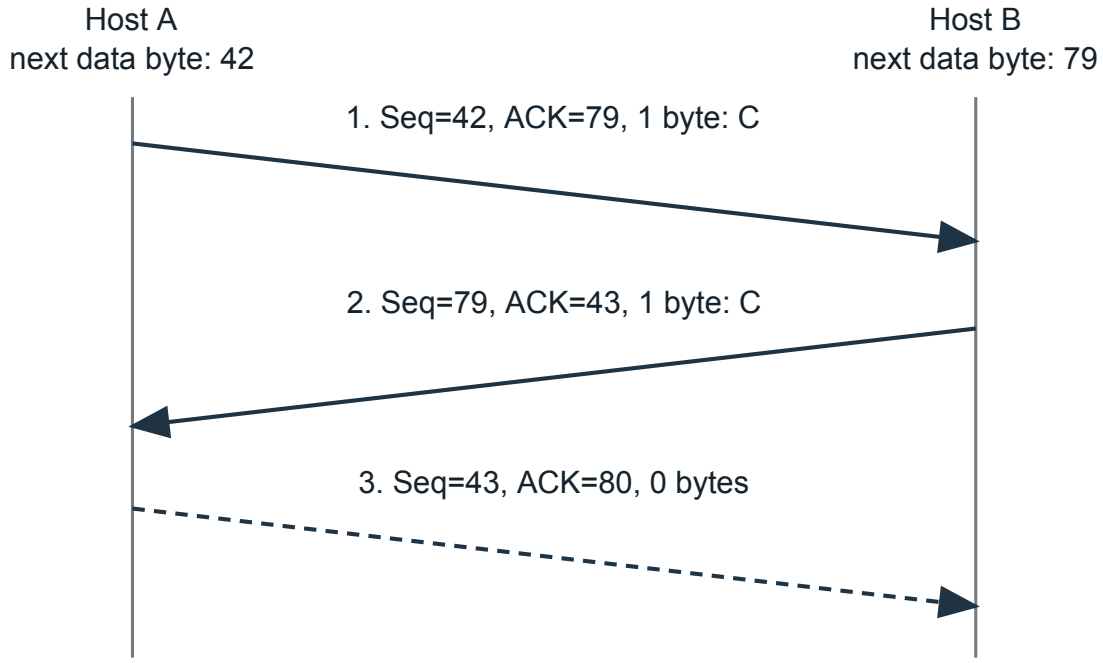
TCP segment 序号是该段第一个数据字节的位置；累计 ACK 是下一期望字节。收到 0–535 后，ACK 为 536，而不是 535。后一段先到不能让连续前缀跨过缺口；基本讨论也不能把 “是否缓存乱序” 与 “是否交给应用” 混为一谈。

跟做 / Worked： seq=1000 且含 500byte，覆盖 1000–1499；若此前均收齐，ACK1500。收到的不是 “1000 号包”，不能简单加 1。

沿课堂 Telnet 图算完三个报文段

在课堂的 Telnet 回显例子中，两端各有自己的字节序列；回显字符属于反方向的数据，一个段可以同时携带本方向数据和对反方向的 ACK。不要用一个共享计数器给整个双向连接编号。

连接已建立，A 下一待发送数据字节编号 42，B 下一待发送数据字节编号 79；A 正期待 B 的 79，B 正期待 A 的 42。用户在 A 输入 ASCII 字符`C`，本例只计这个字符的 1 byte，不另加回车、换行或 Telnet 控制数据。报文按图中顺序正确到达，没有其他数据、SYN 或 FIN。



Already established; no other data, SYN or FIN. Time runs downward.

按 Chapter 3 part 2 第 22–23 张的原数字整理。时间向下，仅表示先后，不按实际 RTT 比例。实线带数据，虚线是无数据的 ACK；灰度也可区分。

1. **A 发`C`：Seq=42，ACK=79，数据 1 byte**。 Seq 属于 A→B 方向，这个字节占 42；A 发送之后下一新字节应从 43 开始。ACK=79 描述 B→A 方向：A 还没收到 B 的新数据，仍等待 79。

2. **B 确认并回显：Seq=79，ACK=43，数据 1 byte**。 B 收到 A 的 42，下一期待 43，所以写 ACK43。回显的C是 B 自己发送的数据，使用 B 的 79；不能因为字符相同就复制 Seq42。这个段把数据和确认放在一起，称捎带确认（piggybacking）。
3. **A 只确认回显：Seq=43，ACK=80，数据 0 byte**。 A 已经收到 B 的 79，下一期待 80。自己的下一个数据编号仍是 43，所以空段写 Seq43；它不发送新数据，也没有 SYN/FIN，不消耗数据序号。若随后输入另一个单字节字符，仍从 Seq43 发出，而非 44。

因此填写一行前，先确定“是谁发送”：Seq 从该端自己的发送进度取，ACK 从该端对另一方向的连续接收进度取。收到对自己数据的确认，不会让自己的发送编号额外再加一次。纯 ACK 不占序号空间的补充依据见 RFC 9293《Transmission Control Protocol》§3.5。

独立变式 / Transfer：连接已建立，A 下一发送字节为 1000 并等待 B 的 5000，B 下一发送字节为 5000 并等待 A 的 1000。A 在一个段中发送 ASCII OK两个字节，B 完整收到后用一个段原样回显，A 最后发纯 ACK。三段依次完整到达，无其他数据、SYN 或 FIN。填写每段的 Seq、ACK、数据长度，以及 A 再发新数据时的 Seq。/ On an established connection, A’s next send byte is 1000 and it expects B’s 5000; B’s next send byte is 5000 and it expects A’s 1000. A sends ASCII OK in one segment, B echoes both bytes in one segment, and A sends a pure ACK. All three arrive in order, with no other data, SYN or FIN. Give each segment’s Seq, ACK and payload length, and A’s next new-data Seq.

答案 / Answer

方向	Seq	ACK	数据长度 / Payload
A→B	1000	5000	2 byte
B→A	5000	1002	2 byte
A→B	1002	5002	0 byte

A 下次新数据仍从 1002 开始。两个方向各增加 2，纯 ACK 不额外增加；OK按 ASCII 是两个字节，不是一个“消息号”。/ A’s next new-data Seq is 1002. Each data direction advances by two bytes, and the pure ACK consumes no sequence space.

English takeaway: A TCP sequence number locates the first payload byte; a cumulative ACK identifies the next expected byte. Each direction has its own byte sequence.

来源：Chapter 3 part 2，第 17–23 张幻灯片。

10. RTT 与重传超时｜等多久才有理由怀疑包丢了？

超时设置需要权衡：过短会误重传，过长会拖慢真正丢包的恢复。SampleRTT 是一次测得的往返时间，EstimatedRTT 是平滑估计：

$$E_{new} = (1-\alpha)E_{old} + \alpha S, \quad \alpha = 0.125\text{为课件常用值.} \tag{3.3}$$

S 是本次样本，E 与 S 都以相同时间单位计。历史样本的影响指数衰减。波动用 DevRTT 近似：$D_{new} = (1-\beta)D_{old} + \beta|S-E|$，课件常用 β=0.25；超时 $RTO = E + 4D$。D 不是“又一份平均 RTT”，而是安全余量相关的波动量。

把两项都算一次（教学例）：旧 E=100 ms、旧 D=10 ms，本次 S=140 ms，取 alpha=0.125、beta=0.25。**本例约定先更新 E，再把新 E 代入 D 的式子**。/ Start with E=100 ms, D=10 ms and S=140 ms, with alpha=0.125 and beta=0.25. In this teaching calculation, update E first and use the new E in the deviation update.

步骤	代入	结果
平滑 RTT	0.875×100+0.125×140	105 ms
与新估计的偏差	abs(140−105)	35 ms
平滑波动	0.75×10+0.25×35	16.25 ms
本例超时	105+4×16.25	170 ms

如果题目规定 D 固定为 10 ms，则是 105+40=145 ms；若用旧 E 更新 D，又会得到另一值。做题时要把采用的更新顺序写明，不能把不同约定的结果直接比较。

变式 / Transfer：从同样的旧 E=100 ms、旧 D=10 ms 重新开始，S 改为 80 ms，仍先更新 E 再更新 D。求 E、D、RTO。/ Restart from E=100 ms and D=10 ms, but use S=80 ms. Keep the same coefficients and update order. Find the new E, D and RTO.

答 / Answer： E=97.5 ms；偏差 17.5 ms，D=11.875 ms；RTO=145 ms。/ E=97.5 ms, D=11.875 ms, RTO=145 ms.

超时样本还涉及重传后无法确定 ACK 对应哪次发送的歧义，原 Chapter 3 part 2，第 24 张幻灯片因此强调不把重传段直接用于 SampleRTT。真实 TCP 的最小超时与退避规则需另参协议规范；本节只做课件给定模型，不把本例算出的毫秒数当所有系统应采用的配置值。

来源：Chapter 3 part 2，第 24–26 张幻灯片。

11. 回到老师的题目

Tutorial5（本册 p.77）前两题检查端点、方向和解复用；Q3 检查反码加法及漏检；Q4 要求你给可靠性结论找模型反例。先完成这些，再复习 N150、N154、N158、N167。

英文自测：Why can different clients share server port 80? What ambiguity do sequence numbers solve? Why is a timer needed? Why can a delayed duplicate defeat alternating-bit assumptions? How do packet ACKs differ from TCP ACKs?

最低要点 / Key points：完整端点不同；分清新数据与重传；丢失时无反馈；旧编号可能被重用；TCP 确认下一个字节。/ Different endpoint tuples; distinguish new data from retransmissions; loss may produce no feedback; old numbers can be reused; TCP acknowledges the next byte.

来源覆盖索引

原页	讲义位置
Chapter 3 part 1，第 1–3、8、16、21 张幻灯片章节范围与目录	开头、各节顺序；后续目录不计为已下载正文
Chapter 3 part 1，第 4–7 张幻灯片传输服务、层间关系	§1
Chapter 3 part 1，第 9–15 张幻灯片复用、UDP/TCP 解复用与图例	§2
Chapter 3 part 1，第 17–20 张幻灯片 UDP 与 checksum	§3–4
Chapter 3 part 1，第 22–27 张幻灯片可靠传输接口与 FSM	§5
Chapter 3 part 1，第 28–39 张幻灯片 rdt1.0、2.0、2.1、2.2	§5
Chapter 3 part 1，第 40–44 张幻灯片 rdt3.0、定时器、正常/数据丢失/ACK 丢失时序	§6
Chapter 3 part 2，第 1–4 张幻灯片停等与流水线利用率	§7
Chapter 3 part 2，第 5–15 张幻灯片 GBN/SR、窗口、状态与丢包	§8
Chapter 3 part 2，第 16 张幻灯片目录	当前范围提示
Chapter 3 part 2，第 17–23 张幻灯片 TCP 概览、首部、seq/ACK、Telnet	§9
Chapter 3 part 2，第 24–26 张幻灯片 RTT 与 timeout	§10

原页序保留，示意图与数值算例标为补充；后续 part 未提供的内容可在历史预习中选读。当前范围见开头说明。

Tutorial 1 · Switching, circuits and sharing | 把链路模型算清楚

Chapter 1（本册 p.9）· 课程目录 · 下一份 Tutorial（本册 p.61）

按当前原题 PDF（3 页）的 Q1–4 逐题讲解，并对照教师解答 PPT（10 页）。下文“教师结论”指该文件；推演、时间表和变式由整理者补充。所有选择题的原选项在原页图中，正文给出推理步骤。

可选先修： bit/byte 与速率（基础册 p.37）、流水线时间轴（基础册 p.40）、组合概率（基础册 p.28）。

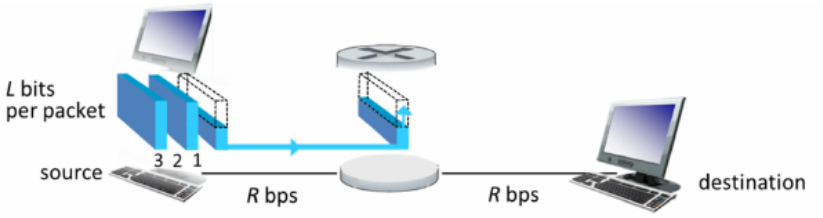
原题	优先级	难度/卡点	本题检验；依据
Q1	核心必会	中；不同包可以重叠	画存储转发时间轴；Tutorial 1 教师解答，第 3–4 张幻灯片
Q2	核心必会	中；每次连接占用哪些边	给出可行路由与容量上界；Tutorial 1 教师解答，第 6–7 张幻灯片
Q3	核心必会	低；单电路速率	单位换算与建立时延；Tutorial 1 教师解答，第 9 张幻灯片
Q4	核心必会	中；独立与组合数	容量和人数分布；Tutorial 1 教师解答，第 10 张幻灯片

学习优先级依据当前原题与教师解答。

Q1. Store-and-forward ｜先等整包到，再转发

Tutorial 1, CS5222, 2026-27

1. As we introduced in the lecture, the packet switching uses **store-and-forward transmission** at the inputs to the links. That means the entire packet must arrive at router before it can be transmitted onto the next link. (Ignore queuing, propagation delay, and processing delay)



An illustrative example of Packet-switching: store-and-forward

a. Suppose there is exactly one packet switch between a sending host and a receiving host. The transmission rates between the sending host and the switch and between the switch and the receiving host are R_1 and R_2, respectively. With store-and-forward switching, what is the total end-to-end delay to send a packet of length L?

A) L / R_1
B) L / R_2
C) $L / R_1 + L / R_2$
D) $2L / R_1 + 2L / R_2$

b. Assuming the transmission rate to be constant at R, what will be the end-to-end delay for sending P=3 packets each of length L across N=2 links (i.e., the packets need to pass two links to reach the destination as shown in the figure above)?

A) L / R
B) 2L / R
C) 3L / R
D) 4L / R

2. Consider the circuit switch network in Figure below. Suppose that there are 4 circuits on each link. Label the four switches A, B, C and D, going in the clockwise direction.

完整条件 / Conditions: 一个源主机、一个分组交换机、一个目的主机；两条链路速率为 R_1、R_2，包长 L bit。存储转发，忽略排队、传播和处理时延。b 问改为相同速率 R，P=3 个 L-bit 包跨 N=2 条链路。/ One switch connects a sender and receiver through links R_1 and R_2. Packets contain L bits. Use store-and-forward and ignore queueing, propagation and processing. In part b, three equal packets cross two equal-rate R links.

a 的推演： t=0 开始发送；$t=L/R_1$ 时整包抵达交换机；交换机此时才开始第二段，又花 L/R_2。因此教师结论为

$$d = L/R_1 + L/R_2 \quad \text{（选 C）}. \tag{T1.1}$$

不是 $L/(R_1 + R_2)$，因为这两条链路是先后经过，不是并行拼成一条更快链路。

b 的推演：令 T=L/R，把每段链路当成一次只能发送一包的工位：

时段	源 → 交换机	交换机 → 目的
0 到 T	包 1	等待整包 1
T 到 2T	包 2	包 1
2T 到 3T	包 3	包 2
3T 到 4T	无	包 3

最后整包到达时间 4T=4L/R，选 D；与教师解答第 4 页 $(P+1)L/R$ 一致。一般等速 N 跳为 $(P+N-1)L/R$，但必须保留本题全部简化条件。

English answer: (a) $L/R_1+L/R_2$. (b) 4L/R. Store-and-forward constrains each packet, but separate links can transmit different packets simultaneously.

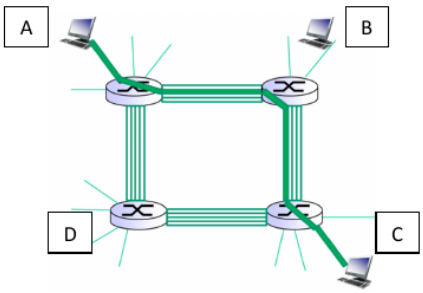
变式 / Transfer: 5 个包、3 条同速链路，同样忽略其他时延，多久？/ Five packets cross three equal-rate links under the same assumptions. Find completion time.

答案 / Answer

7L/R。第一包 3L/R，之后 4 个每隔 L/R 到。/ 7L/R: three intervals for the first packet, then four further intervals.

Q2. Circuit capacity ｜数连接，也要数它消耗了几条边

Tutorial 1, CS5222, 2026-27



a. What is the maximum number of simultaneous connections that can be in progress at any one time in this network?

A) 4
B) 8
C) 16
D) 32

b. Suppose that all connections are between switches A and C. What is the maximum number of simultaneous connections that can be in progress?

A) 4
B) 8
C) 16
D) 32

c. Suppose we want to make four connections between switches A and C, and another four connections between switches B and D. Can we route these calls through the four links to accommodate all eight connections?

A) No
B) Yes

3. Suppose we want to send a file of 160,000 bits from Host A to Host B over a circuit-switched network. Suppose that all links in the network use FDM with 12 frequency bands of the same bandwidth. Assume that the 12 frequency bands together can support a total bit rate of 1.536 Mbps. Also suppose that it takes 600 msec to establish an end-to-end circuit before Host A can

条件 / Conditions: A、B、C、D 顺时针围成四边环，每条边 4 条电路，一次连接在路径每条边占一条电路。/ Four switches A,B,C,D form a clockwise ring; each link supports four circuits. A connection uses one circuit on every traversed link.

a 最大任意连接数 / Maximum arbitrary simultaneous connections: 让每条边只承载相邻两点之间的连接，每边 4 个、4 条边，总 16，选 C。上界也为 16，因为全网总共 16 份“边电路”且每连接至少占一份，所以这个构造达到上界。

b 全部 A 到 C / All connections from A to C: 每个至少两跳，可分别走 A–B–C 与 A–D–C，各 4，总 8，选 B。A 出发仅 8 份电路，给出同样的上界；不是 16÷4，因为每连接没有绕完整一圈。

c 四个 A–C 加四个 B–D / Four A–C plus four B–D: 可行，选 B（Yes）。按教师方案：A–C 中 2 个经 B、2 个经 D；B–D 中 2 个经 A、2 个经 C。

路径	连接数	占用的边
A–B–C	2	AB、BC
A–D–C	2	AD、DC
B–A–D	2	BA、AD
B–C–D	2	BC、CD

逐边相加 AB=4、BC=4、CD=4、DA=4，不超过容量。不要只说“总共 8 小于 16，所以可以”，那忽略了路径可能挤在同一条边。

English answer: (a)16; (b)8; (c)Yes, split each opposite-pair demand evenly between its two two-hop paths. Every link then carries exactly four circuits.

变式 / Transfer: 每边只有 3 条电路，全部 A–C 最多多少？/ With three circuits per link, what is the maximum for A–C connections only?

答案 / Answer

6，两条边不重叠路径各 3。/ Six, with three on each disjoint two-hop path.

Q3. FDM and setup ｜总速率不等于你的那一份

条件 / Conditions: 文件 160000 bit；电路网络用 12 个等宽频带，总速率 1.536 Mbps，建立端到端电路需 600 ms，随后才发文件。求总时间。/ A 160000-bit file uses one of twelve equal-rate FDM circuits sharing 1.536 Mbps. Circuit setup takes 600 ms before transmission. Find total time.

每电路速率 $1.536 \times 10^6/12 = 128000$bit/s。发送用 $160000/128000 = 1.25$s，加 setup0.6s，共 **1.85s，选 A**。教师解答第 9 页相同。题目仅计建立和发送时延。

English answer: Each circuit supplies 128 kbps; 1.25 s transmission plus 0.60 s setup gives 1.85 s.

常见错法：直接除总速率 1.536Mbps；把 600ms 当 600s；把 160000bit 再乘 8（本来就已经是 bit）。

变式 / Transfer: 同电路发送 320000bit、setup 不变，总时间？/ Double the file size to 320000 bits without changing setup or circuit rate.

答案 / Answer

2.5+0.6=3.1s。/ 3.1 s. 文件翻倍只让发送部分翻倍，不把建立时延翻倍。

Q4. Statistical multiplexing ｜算人数前，先说是否独立

Tutorial 1, CS5222, 2026-27

begin to transmit the file. How long does it take to send the file?

A) 1.85 sec
B) 1.95 sec
C) 2.05 sec
D) 2.15 sec

4. Suppose users share a 3 Mbps link. Also suppose each user requires 150 kbps when transmitting, but each user transmits only 10 percent of the time.
   a. When circuit switching is used, how many users can be supported?

A) 10
B) 20
C) 30
D) 40

   b. For the remainder of this problem, suppose packet switching is used. Find the probability that a given user is transmitting.

A) 0.1
B) 0.2
C) 0.3
D) 0.4

   c. Suppose there are 120 users. Find the probability that at any given time, exactly n users are transmitting simultaneously.

A) $\binom{120}{n}(0.1)^{120-n}(0.9)^n$
B) $\binom{120}{n}(0.9)^{120-n}(0.1)^n$
C) $(0.9)^n$
D) $(0.1)^n$

条件 / Conditions: 共享 3Mbps；每人活跃时需 150kbps，每人 10% 时间活跃。a 用电路交换；b/c 用分组交换，c 共有 120 用户。/ Users share 3 Mbps, each requiring 150 kbps when active, with activity fraction 10%. Part a uses circuits; parts b/c use packet switching, with 120 users in c.

a 每用户预留 150kbps，可容纳 $3\times10^6/(150\times10^3)=20$ 人，选 B。不把它再除活跃比例，电路资源已经预留。

b 随机时刻某用户活跃概率 0.1，选 A。把长期活跃比例作为该随机时刻概率，是题目的稳态抽样约定。

c 在**用户独立、每人同为 p=0.1** 的模型下，K~Binomial(120,0.1)，恰 n 人活跃概率

$$P(K=n)=\binom{120}{n}(0.1)^n(0.9)^{120-n},\quad n=0,1,\ldots,120. \tag{T1.2}$$

对应原选项 B（其因子顺序先写 0.9）。组合数选出哪些人在发；0.1 乘 n 次、0.9 乘其余 120−n 次。原题没有清楚写独立，教师二项解答隐含这个假设，不能遗漏。

English answer: (a)20 users; (b)0.1; (c)choose(120,n) · 0.1^{n} · 0.9^{120-n}, assuming independent user activity.

平均活跃人数 Np=12 并不意味着始终 12 人；容量 20 也不保证任何时刻都够用。补充：超额事件为 K>20，应该求从 21 起的尾概率，不是只算 K=20 或 K=21。超额需求可用同一个人数分布继续计算。

变式 / Transfer: 3 个独立用户、p=0.1，容量可同时容纳 2 人，超额概率？/ Three independent users have p=0.1 each; capacity supports two. Find excess-demand probability.

答案 / Answer

三人全活跃，0.1^3=0.001。/ All three must be active, giving 0.001.

回查与巩固

所有数值与环路逐边占用由 network_examples.py 复算。原题 PDF p1=Q1/Q2 引入，p2=Q2/Q3 引入，p3=Q3 继续/Q4；Tutorial 1 教师解答，第 1–2 张幻灯片背景、3–4Q1、5 电路图、6–7Q2、8FDM/TDM、9Q3、10Q4 均已对应。

复习使用现有 Tutorial1 卡，首选 N071/N072（存储转发）、N076（FDM）、N077（人数与容量），以及 Chapter1（本册 p.19）相关题。学习完应能用英文解释 *why*，不只记住 C/D/B 这些选项字母。

Tutorial 2 · Delay and traceroute ｜先定义计时事件，再按顺序相加

Chapter 1（本册 p.13）· 课程目录 · Tutorial 1（本册 p.54）· Assignment 1（本册 p.82）

依据当前原题 PDF（3 页）与教师解答 PPT（13 页），按 Q1–5 讲解。难点是计时边界与转发模型；优先级均为核心必会，Q1/Q3 中等偏难，Q2/Q4 中等，Q5 需实际观察与解释。优先级依据当前 Tutorial。

基础复习（可跳过）：单位（基础册 p.37）、首位与末位时间轴（基础册 p.40）。所有计算约定 Mbps=10^6bit/s、kbps=10^3bit/s。

Q1. VoIP ｜语音先攒成一包，还要算这段等待

Tutorial 2, CS5222, Sem A, 2026-27

1. In this problem, we consider sending real-time voice from Host A to Host B over a packet-switched network (VoIP). Host A converts analog voice to a digital 128 kbps bit stream on the fly. Host A then groups the bits into 64-byte packets. There is one link between Host A and B; its transmission rate is 4 Mbps and its propagation delay is 8 msec. As soon as Host A gathers a packet, it sends it to Host B. As soon as Host B receives an entire packet, it converts the packet's bits to an analog signal. How much time elapses from the time a bit is created (from the original analog signal at Host A) until the bit is decoded (as part of the analog signal at Host B)? (note: 128 kbps = $128*10^3$ bps, 4Mbps = $4*10^6$ bps)

A) 2.08 msec
B) 6.08 msec
C) 9.128 msec
D) 12.128 msec

2. a) Consider a packet of length L which begins at end system A and travels over three links to a destination end system. These three links are connected by two packet switches. Let d_i, s_i, and R_i denote the length, propagation speed, and the transmission rate of link i, for i = 1, 2, 3. The packet switch delays each packet by d_{proc}. Assuming no queuing delays, in terms of d_i, s_i, and R_i, (i = 1,2,3), and L, what is the total end-to-end delay for the packet?

A) $L/(R_1 + R_2 + R_3) + d_1/s_1 + d_2/s_2 + d_3/s_3 + 2d_{proc}$
B) $L/R_1 + L/R_2 + L/R_3 + d_1/s_1 + d_2/s_2 + d_3/s_3 + 2d_{proc}$
C) $LR_1 + LR_2 + LR_3 + d_1s_1 + d_2s_2 + d_3s_3 + 2d_{proc}$
D) $LR_1 + LR_2 + LR_3 + d_1s_1 + d_2s_2 + d_3s_3$

b) Suppose now packet is 1,500 bytes, the propagation speed on all three links is $2.5*10^8$ m/s, the transmission rates of all three links are 2 Mbps, the packet switch processing delay is 3 msec, the length of the first link is 5,000 km, the length of the second link is 4,000 km, and the length of the last link is 1,000 km. For these values, what is the end-to-end delay? (Note that M = 10^6, k = 10^3)

A) 24 msec
B) 64 msec
C) 86 msec
D) 96 msec

完整条件 / Conditions: A 以 128kbps 实时生成语音比特，每 64byte 成一包，收集完成立即发送；链路 4Mbps、传播 8ms，B 收完整包后还原模拟语音。问生成一个 bit 到它被解码的时延。/ A generates voice at 128 kbps, groups it into 64-byte packets and immediately transmits each completed packet over a 4-Mbps link with 8-ms propagation. B decodes after receiving the complete packet. Find generation-to-decoding delay.

先按包首位及连续流体近似计算：包长 512bit；收集一包 512/128000 = 4ms；把包送上链路 512/4000000 = 0.128ms；传播 8ms。首位从生成到可开始播放的延迟为 **12.128ms，选 D**，与 Tutorial 2 教师解答，第 5 张幻灯片结论一致。

这里“任意一位”还要区分读取与播放这两个时刻。一个包内越晚产生的 bit 等待成包越短；若问“该 bit 到达整包缓冲、可被读取”的延迟，数值随位次变化，约从 12.128ms 降到 8.128ms。若接收端

在包到齐后，**按与源相同 128kbps 的节奏顺序恢复/播放语音**，后面的 bit 在接收端也会多等相同位次时间，因此每个 bit 的生成到播放时延相同，都是 12.128ms。教师“all bits”的说法应连着这个播放约定读。

设第 j 位相对首位晚产生 τ_j，包到达时刻 T=12.128ms；可读取延迟为 T−τ_j，顺序播放时刻为 T+τ_j，播放延迟则 (T+τ_j)−τ_j=T。因此，答题前要确定问的是可读取时刻还是播放时刻。

English answer: Under the instructor's packetization and source-rate sequential playout model, the delay is 4+0.128+8=12.128 ms for every bit. Complete-packet availability alone gives a position-dependent delay.

变式 / Transfer: 包改 128byte，其他条件不变，首位/同速播放时延？/ Double packet size to 128 bytes, keeping all rates and propagation unchanged.

答案 / Answer

8+0.256+8=16.256ms。/ 16.256 ms. 较大包减少每份数据的头部比例，但收集等待变长；本题没有给头长，不计算额外百分比。

Q2. Three store-and-forward links ｜每一跳的发送与传播都要数

条件 / Conditions: 长 L bit 的包经三条链路，两台存储转发交换机；第 i 链路长 d_i m、传播速度 s_i m/s、发送速率 R_i bit/s，每台交换机处理时延 dproc，无排队。/ An L-bit packet crosses three store-and-forward links and two switches. Link i has length d_i, propagation speed s_i and rate R_i; each switch adds dproc and there is no queueing.

a 把事件从左到右排：链路 1 发送/传播 → 交换机 1 处理 → 链路 2 发送/传播 → 交换机 2 处理 → 链路 3 发送/传播。

$$d_{end} = \sum_{i=1}^{3} \frac{L}{R_i} + \sum_{i=1}^{3} \frac{d_i}{s_i} + 2d_{proc} \quad \text{（选 B）.} \tag{T2.1}$$

b 给定 L=1500byte=12000bit，各 R=2Mbps，各 s=2.5×10^8m/s，距离 5000/4000/1000km，每交换机 3ms：

部分	计算	结果
三段发送	每段 12000/(2×10^6)	6+6+6ms
三段传播	每段先 km→m 再除 s	20+16+4ms
两次处理	2×3ms	6ms
总计	18+40+6	**64ms，选 B**

English answer: Sum the three serialization times, three propagation times and two processing times. The numerical total is 64 ms.

这里包长 L 在公式中必须是 bit；若直接使用 1500 需在分子明确乘 8。处理是两台交换机，不是三条链路所以乘 3。

Q3. Immediate bit forwarding ｜不用每个中间节点再等整包

Tutorial 2, CS5222, Sem A, 2026-27

3. In the above problem, suppose $R_1 = R_2 = R_3 = R$ and $d_{proc} = 0$. Further suppose the packet switch does not store-and-forward packets but instead immediately transmits each bit it receives before waiting for the entire packet to arrive. What is the end-to-end delay?

A) 16 msec
B) 26 msec
C) 36 msec
D) 46 msec

4. A packet switch receives a packet and determines the outbound link to which the packet should be forwarded. When the packet arrives, one other packet is halfway done being transmitted on this outbound link and four other packets are waiting to be transmitted. Packets are transmitted in order of arrival. Suppose all packets are 1,500 bytes and the link rate is 2 Mbps. What is the queuing delay for the packet? (Note that M = 10^6.)

A) 27 msec
B) 37 msec
C) 47 msec
D) 57 msec

More generally, what is the queuing delay when all packets have length L, the transmission rate is R, x bits of the currently-being-transmitted packet have been transmitted, and n packets are already in the queue?

条件 / Conditions: 沿用 Q2，R_1=R_2=R_3=R、dproc=0，无排队，中间节点每收到一 bit 立即转发，不等整包。/ Use the Q2 path, equal rates and zero processing; each bit is forwarded immediately with no whole-packet waiting or queueing.

首位经过各传播段，末位相对首位只落后源端一次 L/R。故总为 $L/R + \sum_i d_i/s_i$。带 Q2 数字为 6+20+16+4=**46ms，选 D**，Tutorial 2 教师解答，第 8 张幻灯片相同。

这不表示物理链路“传输时间消失”，而是各段的发送过程可以流水重叠，末位到达总时间只额外保留一次包序列长度。不同速率、实际 cut-through 启动/头部检查有其他限制；不要把这个理想模型直接当所有交换机行为。

English answer: Equal-rate immediate bit forwarding gives L/R plus all propagation delays, hence 46 ms. Serialization on successive links overlaps.

变式 / Transfer: 四条相同速率链路，每条传播 2ms，源包发送 3ms，即时转发和整包存储转发分别多久？/ Four equal-rate links each propagate for 2 ms; serialization takes 3 ms. Compare immediate bit forwarding with store-and-forward for one packet.

答案 / Answer

即时 3+4×2=11ms；存储转发 4×3+4×2=20ms，忽略排队/处理。/ 11 ms versus 20 ms, with no queueing or processing.

Q4. Queueing ｜只算轮到你之前的等待

条件 / Conditions: 新包到达时，出口上一个 1500byte 包已发一半，还有 4 包等待，均 1500byte，FIFO，R=2Mbps。求新包排队时延，再推广为包长 L bit、当前包已发送 x bit、另有 n 包排队。/ A new packet finds one equal packet half transmitted and four equal packets queued, FIFO, L=1500 bytes and R=2 Mbps. Find queueing delay, then generalize to x bits already transmitted and n waiting packets.

前面剩 4.5 包，$4.5 \times 12000/(2 \times 10^6) = 0.027$s=**27ms，选 A**。一般式为

$$d_{queue} = \frac{(L - x) + nL}{R}. \tag{T2.2}$$

English answer: Queueing delay is 27 ms, or ((L−x)+nL)/R. It excludes transmission of the arriving packet itself.

若问它**完成发送**的时间，则另加 L/R；若问到目的地还要加传播及后续节点条件。若出口本来空闲，就不是这条“有一个正在发送包”的场景，等待为 0。

变式 / Transfer: 同速同包长，当前包已发 75%，队列前面还有 2 包，新包排队多久？/ The current packet is 75% transmitted and two packets are queued ahead. Find the new packet's queueing delay.

答案 / Answer

2.25×6=13.5ms。/ 13.5 ms. 不是加上新包自己的 6ms 变成 19.5ms。

Q5. Traceroute ｜把真实测量和推断分开写

原任务 / Task: 对 cityu.edu.hk 运行 traceroute，解释每列、星号和后跳 RTT 较小的原因。/ Run traceroute to cityu.edu.hk and explain its columns, asterisks, and non-monotonic RTTs.

macOS 可运行原命令`traceroute cityu.edu.hk`。下方 2026-09-23 的观测使用：

```
traceroute -n -m 12 -q 3 -w 1 cityu.edu.hk
```

`-n`保留数值 IP，`-m 12`最多 12 跳，`-q 3`每跳 3 次探测，`-w 1`每次最多等 1 秒。这些参数用于限制本次探测的长度与等待时间，并非题目指定值。输出与时间见 2026-09-23 观察，采集脚本为 observe_network.py。

每行第一项为 TTL/hop 编号，随后为响应地址和三次 RTT（ms）；某次无及时响应显示*。地址识别的是响应接口，复杂网络里不保证完整暴露每一台中间设备。macOS 常用 UDP 探测和 ICMP 响应；Windows `tracert`的实现细节不同，不应机械照搬命令选项。

星号可以来自过滤、应答限速、探测/响应丢失、超时等。Tutorial 2 教师解答，第 12 张幻灯片强调防火墙是常见原因，但仅凭星号不能确定具体根因。后跳 RTT 小于前跳，可能因为不同时刻队列变化、回程路由不同、路由器处理探测优先级不同；RTT 也不是沿途正向传播的逐段累加测量。

English answer: Columns identify probe hop, responding address and round-trip times. An asterisk means no response within the timeout. Different probes and return paths can produce non-monotonic RTTs.

已保存的实际观察（2026-09-23 01:19，UTC+8）：域名解析有两个地址，traceroute 选用 45.60.199.218；第 1 跳三个 RTT 约 4.352/3.957/4.301ms，第 7 跳有星号，第 11 跳第三次升到 192.365ms。在最多 12 跳内没有看到目的地址响应，故这不是完整路径。进程正常结束也不表示探测已到终点。同一跳有不同地址响应，不足以断言一条固定唯一线路。

你自己的观测应另记时间、探测参数和实际输出；换时间或网络，路径和 RTT 可能变化。

回查与巩固

原题 p1=Q1/Q2，p2=Q3/Q4，p3=Q5。Tutorial 2 教师解答，第 1–4 张幻灯片为四类时延/车队背景（见 Chapter1），5=Q1，6–7=Q2，8=Q3，9=Q4，10–11=traceroute 课堂示例，12–13=Q5 解释。全部已对应，原教师结论与本文补充条件分开。

计算复核：network-example-results.json。复习可用现有时延卡 N025 与原题卡 N080，根据本题错误位置，复习单位、时序、转发或排队的相关概念。

Tutorial 3 · HTTP and DNS ｜从报文证据走到时间计算

Chapter 2（本册 p.24）· 课程目录 · Assignment 1（本册 p.82）

依据当前原题 PDF（3 页），按 Problem1–4 展开。教师解答共 13 页：Tutorial 3 教师解答，第 3–4 张幻灯片对应 Q1，Tutorial 3 教师解答，第 8 张幻灯片对应 Q2，Tutorial 3 教师解答，第 11–13 张幻灯片对应 Q3；Q4 未给当前测量答案。以下保留教师结论与整理补充的区别。Q1/Q2/Q3 为核心必会（中等难度，卡在字段证据与 RTT 条件），Q4 为常规掌握的实际操作任务（需区别观察和推断）。

可选先修：ASCII、CRLF 与报文（基础册 p.39）、时间轴（基础册 p.40）。

Problem 1. Read an HTTP request ｜不知道的就明确说不知道

Tutorial 3, CS5222, Sem A, 2026-27

Problem 1. Consider the following string of ASCII characters that were captured by Wireshark when the browser sent an **HTTP GET** message (i.e., this is the actual content of an **HTTP GET** message). The characters **<cr><lf>** are carriage return and line-feed characters (that is, the italized character string **<cr>** in the text below represents the single carriage-return character that was contained at that point in the HTTP header). Answer the following questions, indicating where in the **HTTP GET** message below you find the answer.

```
GET /cs453/index.html HTTP/1.1<cr><lf>Host:gaia.cs.umass.edu<cr><lf>
User-Agent: Mozilla/5.0 ( Windows;U; Windows NT 5.1; en-US; rv:1.7.2)
Gecko/20040804 Netscape/7.2 (ax) <cr><lf> Accept:ext/xml,
application/xml, application/xhtml+xml, text /html;q=0.9,
text/plain;q=0.8,image/png,*/*;q=0.5
<cr><lf>Accept-Language: en-us,en;q=0.5<cr><lf>Accept-Encoding:
zip,deflate<cr><lf>Accept-Charset: ISO-8859-
1,utf8;q=0.7,*;q=0.7<cr><lf>Keep-Alive: 300<cr><lf>
Connection:keepalive<cr><lf><cr><lf>
```

a. What is the URL of the document requested by the browser?
b. What version of HTTP is the browser running?
c. Does the browser request a non-persistent or a persistent connection?
d. What is the IP address of the host on which the browser is running?
e. What type of browser initiates this message? Why is the browser type needed in an HTTP request message?

任务 / Task: 原题给出 Wireshark 看到的一段 HTTP GET 的 ASCII 内容，`<cr><lf>`代表实际单字节 CR、LF。指出 URL、HTTP 版本、是否持久、浏览器主机 IP、浏览器类型及该字段的用途。/ From the supplied HTTP GET bytes, identify the URL, version, connection preference, client IP, and user-agent type and purpose, citing the relevant fields.

为便于定位，下面仅摘录与答案直接相关的原字段；完整报文（包括 Accept 各项及原拼写）见上图：

```
GET /cs453/index.html HTTP/1.1<cr><lf>
Host:gaia.cs.umass.edu<cr><lf>
User-Agent: Mozilla/5.0 ( Windows;U; Windows NT 5.1; en-US; rv:1.7.2)
Gecko/20040804 Netscape/7.2 (ax) <cr><lf>
Keep-Alive: 300<cr><lf>
Connection:keepalive<cr><lf><cr><lf>
```

a. **URL / 请求地址：** 按题目传统明文 HTTP 情境，`http://gaia.cs.umass.edu/cs453/index.html`；Host 给主机，GET 行给路径。若孤立看这些应用字节，并不能独立证明外层是否经过 TLS，方案 http 来自题目情境。/ Under the exercise's plain-HTTP context, combine Host and the request target to obtain this URL.

b. **Version / 版本：** 请求行明确`HTTP/1.1`。/ The request line states HTTP/1.1.

c. **Connection / 连接：** 题意为持久连接。原`keepalive`不是规范常见的`keep-alive`拼写，但 HTTP/1.1 默认可持久，且这里未见`Connection: close`；不要把拼写错误当新标准 token。头表达客户端意图，不单独证明服务器实际维持了多久。RFC9112 §9.3 / The intended answer is persistent HTTP, consistent with HTTP/1.1 defaults; the header alone does not prove the connection's observed lifetime.

d. **Client IP / 浏览器主机 IP：** 给出的 HTTP 头没有源 IP，无法仅据此确定。`gaia.cs.umass.edu`是请求目标，不能当客户端。需原 IP 包头/捕获上下文。/ The client IP cannot be determined from these HTTP fields; inspect the enclosing IP header or capture metadata.

e. **Browser / 浏览器：** Tutorial 3 教师解答，第 4 张幻灯片简答写 Mozilla/5.0；完整 User-Agent 还自称 Netscape/7.2，并带 Mozilla/Gecko 兼容信息。课堂作答可以指出教师采用的 token，同时说明完整字段不只一个产品名。服务器可能据此作兼容处理、统计或内容选择；不要求每种浏览器都必须得到不同页面，也不是身份认证证据。/ The field claims Netscape 7.2 with Mozilla/Gecko compatibility tokens. It can support compatibility handling or analytics, but is self-reported and spoofable.

独立变式 / Transfer: `GET /a.png HTTP/1.1`、`Host: example.com`、`Connection: close`，能判断路径、连接意图及客户端 IP 吗？/ Identify the path and connection preference, and determine whether these fields reveal the client IP.

答案 / Answer

路径/a.png；请求关闭连接；不能确定客户端 IP。/ Path /a.png, requested connection closure, client IP unavailable from these fields.

Problem 2. DNS resolution ｜一对箭头才是一次往返

Tutorial 3, CS5222, Sem A, 2026-27

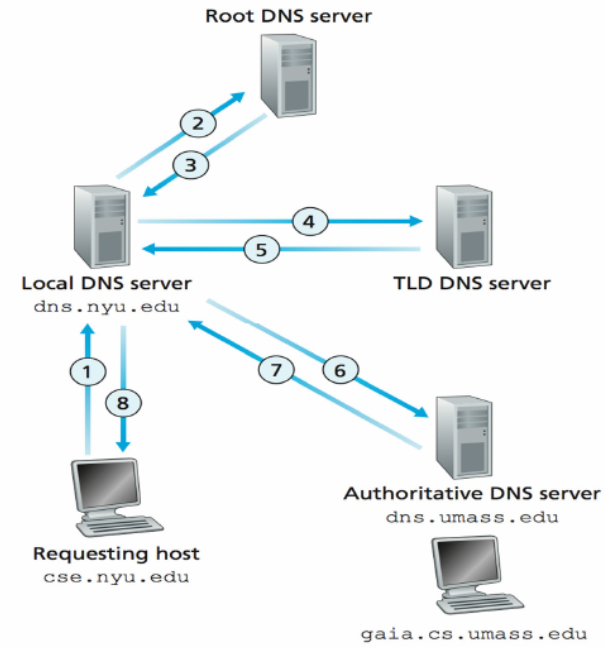
Problem 2. Assume that the RTT between a client and the local DNS server is $\mathbf{RTT_L}$, while the RTT between the local DNS server and other DNS servers is $\mathbf{RTT_r}$. Assume that no DNS server performs caching.

a. What is the total response time for the scenario illustrated in the following Figure?

A) RTT_L
B) RTT_L + RTT_r
C) RTT_L + $2RTT_r$
D) RTT_L + $3RTT_r$

b. Assume now that the DNS record for the requested name is cached at the local DNS server. What is the total response time?

A) RTT_L
B) RTT_L + RTT_r
C) RTT_L + $2RTT_r$
D) RTT_L + $3RTT_r$



条件 / Conditions: client 到 local DNS 的 RTT 为 RTT_L，local 到其他 DNS 每次 RTT 为 RTT_r。a 无任何缓存，按图依次访问 root、TLD、authoritative；b 所需记录已在 local 缓存。忽略额外处理与传输大小。/ The client-local RTT is RTT_L; each local-remote DNS RTT is RTT_r. Follow the figure's sequential root, TLD and authoritative queries without caches, then consider a local cache hit.

a 1/8 合成一次 client-local 往返，2/3、4/5、6/7 分别三次 local-remote 往返。总 **RTT_L+3RTT_r，选 D**。不是 8RTT，因为图上 8 条单向消息不能各算一次来回。

b local 直接回答，有效缓存无需再查询上级，所以 **RTT_L，选 A**。本题缓存只在 local；若题设改为客户端本身已有可用缓存，网络 DNS 往返又不同。

English answer: (a)RTT_L+3RTT_r; (b)RTT_L. Count round trips, not individual arrows, and locate the cache precisely.

变式 / Transfer: RTT_L=4ms、RTT_r=20ms，无缓存与 local 命中分别多久？/ Compute both cases for these RTTs.

答案 / Answer

64ms 与 4ms。/ 64 ms and 4 ms. 缓存减少的是后续查询，不让 client 到 local 的往返也凭空消失。

Problem 3. A page with eight objects ｜先取目录，再知道要取哪八件东西

Tutorial 3, CS5222, Sem A, 2026-27

Problem 3. Suppose within your browser you type in a URL to obtain a webpage. The IP address for the associated URL is not cached in your local host, so as DNS lookup is necessary to obtain the IP address. Suppose that n DNS servers are visited before your host receives the IP address from DNS; Visiting the i-th of the them incurs an RTT_i per DNS. Further suppose the HTML file references eight very small objects on the same server. Let RTT_0 denote the RTT between the local host and the server containing an object. Neglecting transmission times, how much time elapses with

a. Non-persistent HTTP with no parallel TCP connections?

A) $RTT_1 + \ldots + RTT_n + 8RTT_0$
B) $RTT_1 + \ldots + RTT_n + 9RTT_0$
C) $RTT_1 + \ldots + RTT_n + 16RTT_0$
D) $RTT_1 + \ldots + RTT_n + 18RTT_0$

b. Non-persistent HTTP with the browser configured for 5 parallel connections?

A) $RTT_1 + \ldots + RTT_n + 4RTT_0$
B) $RTT_1 + \ldots + RTT_n + 5RTT_0$
C) $RTT_1 + \ldots + RTT_n + 6RTT_0$
D) $RTT_1 + \ldots + RTT_n + 7RTT_0$

c. Persistent HTTP?

A) $RTT_1 + \ldots + RTT_n + 8RTT_0$
B) $RTT_1 + \ldots + RTT_n + 10RTT_0$
C) $RTT_1 + \ldots + RTT_n + 12RTT_0$
D) $RTT_1 + \ldots + RTT_n + 14RTT_0$

Problem 4. (Looking up an IP address using nslookup) You can type nslookup followed by the hostname you want to look up (e.g., type “nslookup cityu.edu.hk” in the command prompt), and nslookup will issues a DNS query to find out the IP address.

a. What is the IP address of the hostname?
b. Can you type the IP address in your browser’s address bar and get to the webpage?

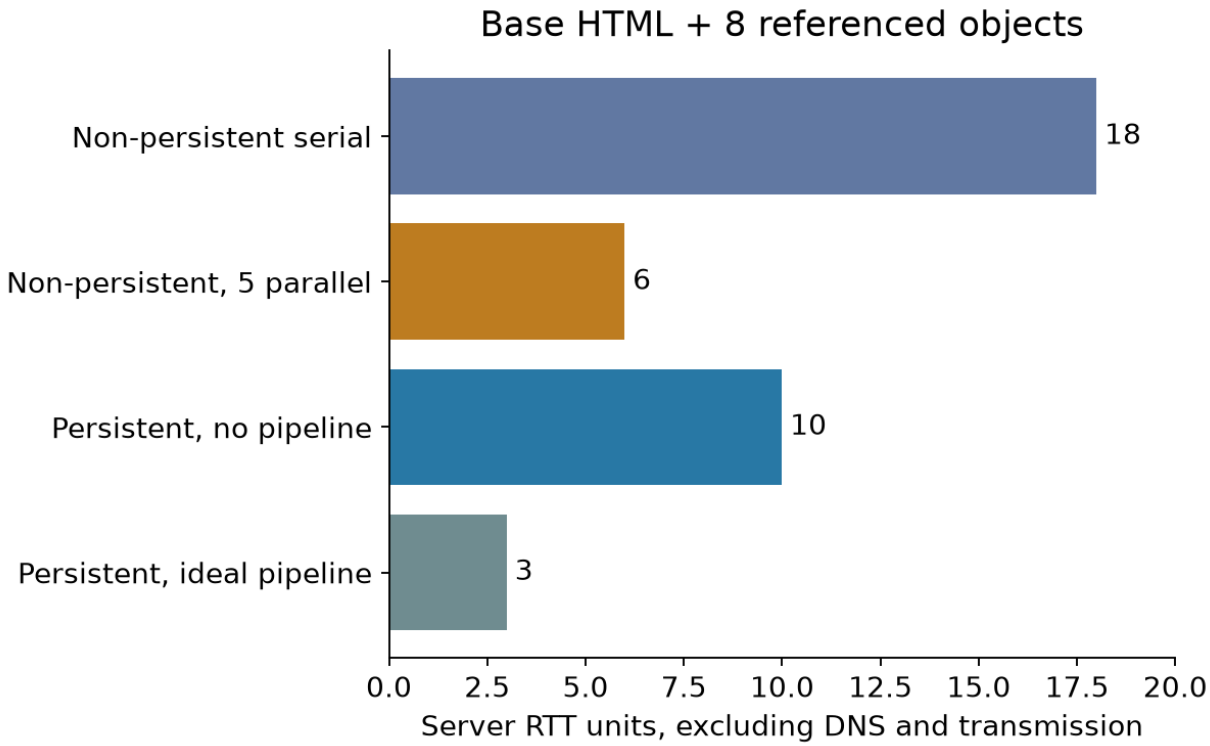
条件 / Conditions: 域名未缓存，顺序 DNS 查找代价 $D = \sum_{i=1}^{n} RTT_i$。base HTML 引用同一服务器的 8 个很小对象；client-server RTT 为 RTT_0，忽略发送时间。先得到 HTML 才能知道对象地址。/ Sequential DNS costs D=ΣRTT_i. One base HTML file references eight small objects on the same server; server RTT is RTT_0 and transmission times are negligible. Object requests start after the base HTML is received.

a **Non-persistent, serial / 非持久且无并行：**共 9 个对象，每个 1RTT 握手 +1RTT 请求返回，所以 **D+18RTT_0，选 D**。

b **Non-persistent, five parallel connections / 非持久最多 5 并行：** HTML 先用 2RTT；8 对象分两批（5+3），每批各 2RTT，合计 **D+6RTT_0，选 C**。不能把 HTML 也塞入尚未解析出来的 8 对象同一批。

c **Persistent / 持久：**原题未明说是否 pipelining；新到 Tutorial 3 教师解答，第 13 张幻灯片明确按逐个请求计算，确认采用单连接**无流水线**。结合给出的选项，一次握手 +HTML 一次往返 +8 对象各一次，共 **D+10RTT_0，选 B**。若另加“拿到 HTML 后流水线一次发完 8 请求”的理想条件，则 D+3RTT_0；这不在原选项中。因此，答案需注明采用单连接、无流水线的约定。

English answer: (a)D+18RTT_0; (b)D+6RTT_0; (c)D+10RTT_0 for non-pipelined persistent HTTP. Pipelining would change part c to D+3RTT_0 under the idealized model.



图按握手/请求轮次计算，仅表示题目简化模型；不包括 TLS、服务端处理时间、连接拥塞控制与大文件传输。

变式 / Transfer: 改为 base HTML+6 对象，非持久且最多 4 并行、其他条件不变，结果？/ Use six referenced objects and at most four parallel non-persistent connections.

答案 / Answer

D+2RTT_0+2×ceil(6/4)RTT_0=D+6RTT_0。/ D+6RTT_0. 物件少两个仍需两批，所以本模型总时间没有减少。

Problem 4. Nslookup and direct-IP browsing ｜解析成功，不保证 IP 网址等于原站

原任务 / Task: 用`nslookup cityu.edu.hk`找地址，再判断把 IP 直接输入浏览器能否得到网页。/ Resolve cityu.edu.hk and investigate whether navigating directly to its IP retrieves the site.

本机 macOS 于 **2026-09-23 01:19（UTC+8）**实际查询，解析器 223.5.5.5 返回 non-authoritative answer，两个 A 地址为 **45.60.199.218**、**45.60.197.218**。“非权威”表示答案来自递归/缓存服务器，不等于错误；多地址也不是异常。地点、时间、DNS 策略变化可返回不同集合，不能把这两个 IP 当永久常量背下来。

直接输入 IP 可能打开默认页面、错误页、重定向，也可能某些服务可用；**不保证与域名访问相同**。HTTP 虚拟主机依赖 Host，HTTPS 还涉及 TLS 服务器名 SNI 和证书名称，CDN/WAF 可能按域名路由。不能从“DNS 返回 IP”推断“浏览器写这个 IP 必定显示该站”。这份 2026-09-23 的记录只包含 DNS 查询，未包含直接 IP 访问的结果。

English answer: The dated lookup returned two A records. Direct-IP navigation is not guaranteed to reproduce the hostname-based website because virtual hosting, TLS name validation and routing can depend on the hostname.

来源与复习

原题 p1=Problem1 全部五问，p2=Problem2 原图及两问，p3=Problem3 三种连接模型与 Problem4 实际查询。理论对应 Chapter2 p19–31（HTTP）、p53–67（DNS）；教师解答与本稿 DNS/HTTP 时间计算一致；未给出的实际网络测量与协议边界仍标为整理补充。HTTP 补充标准查阅于 2026-09-23。

复算脚本见 network_examples.py。关联现有原题卡 N231、N232、N233 时，先能解释字段证据和计时条件，再用 Markji 巩固。

Tutorial 4 · File Distribution, BitTorrent and DASH

先找谁供不动，再算怎样分得完

Chapter 2（本册 p.24）· 课程目录 · 下一份：Tutorial 5（本册 p.77）

依据原题 2 页和教师解答 v2 共 15 页，按 Problem1–4。本 Tutorial 配套 Chapter2 应用层。题目单位用十进制 Gbit/Mbit/kbit。

按需基础： 单位换算（基础册 p.37）、速率与完成时间（基础册 p.40）。先会用“任务量 ÷ 每秒处理量”算时间。

对应讲解：分发下界（本册 p.32）解释三类容量约束；BitTorrent 请求与供给（本册 p.33）区分稀缺块选择和上传机会；DASH 与 CDN（本册 p.34）把表示、时间块、服务节点及播放缓冲接起来。

题目	优先级	难度与卡点	掌握要求 / 依据
Q1 分配方案	核心必会	中，下界还需构造达到它的方案	按两种瓶颈给实际速率；Tutorial 4 教师解答 v2，第 1–5 张幻灯片
Q2 时间计算	核心必会	中，总供给与最慢下载	统一单位、比较每个约束；Tutorial 4 教师解答 v2，第 6–8 张幻灯片
Q3 free-riding	常规掌握	中，可能与保证不同	解释 optimistic unchoking 条件；Tutorial 4 教师解答 v2，第 9–11 张幻灯片
Q4 DASH	核心必会	中，配对假设	区分 N、N^2 与 2N；Tutorial 4 教师解答 v2，第 12–15 张幻灯片

优先级依据当前原题；额外练习标为变式。

Problem 1. Construct a distribution scheme

原条件 / Conditions： 服务器初始持有 F bit 文件，分给 N 个客户端，服务器总上传率 u_s，客户端下载率 d_i，$d_{min} = \min_i d_i$。流体模型允许同时以不同速率发送，总发送率不超过 u_s，忽略其他瓶颈。/ The server simultaneously allocates divisible traffic to N clients, subject to its total upload and each client's download limit.

服务器必须上传 NF bit，所以时间至少 NF/u_s；最慢客户端必须收 F bit，所以至少 F/d_{min}。但“至少这么久”还不是“确实做得到”，题目要求给方案。

a. $u_s/N < d_{min}$：平均给每个客户端 u_s/N。总速率正好 u_s，每人都不超过其下载上限；每人同时在 $F/(u_s/N) = NF/u_s$ 完成。瓶颈是服务器供给。

b. $u_s/N > d_{min}$：每人按 d_{min} 发送。总量 $Nd_{min} < u_s$，所有客户端都收得下；同在 F/d_{min} 完成。快客户端没有跑满也不妨碍整体最优，因为慢客户端仍决定结束时刻。

合起来，每人分配 $\min(u_s/N, d_{min})$，达到两项下界的最大值。等号边界两种表达相同。Tutorial 4 教师解答 v2，第 2 张幻灯片/Tutorial 4 教师解答 v2，第 4 张幻灯片的反证是在同一流体约束下说明方案，不是现实 TCP 性能保证。

English answer: Allocate min(us/N,dmin) to every client. The aggregate rate is feasible and every client completes in max(NF/us,F/dmin).

独立题 / Transfer： F=100Mbit、N=4、u_s=20Mbps、每人下载 8Mbps，多久？/ For F=100 Mbit, N=4, server upload 20 Mbps and each client download 8 Mbps, give a feasible allocation and completion time.

答案 / Answer

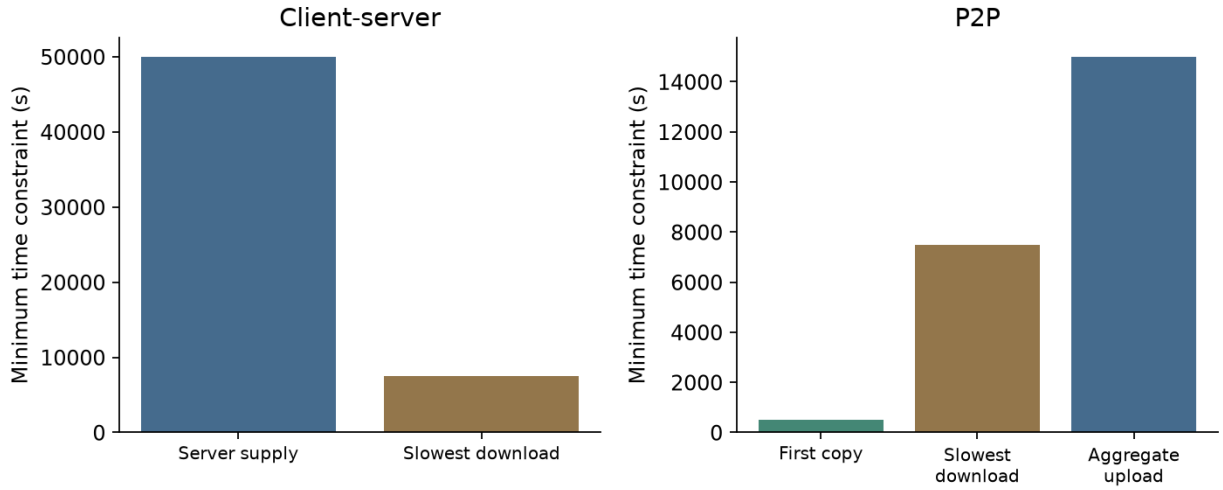
每人 5Mbps，20 秒。最慢下载约束 12.5 秒不是最终答案，要与总上传约束 20 秒取最大。/ 5 Mbps each, finishing in 20 seconds.

Problem 2. 15 Gbit 分给 100 人

完整条件 / Conditions： F=15Gbit, N=100, 服务器 30Mbps, 每人下载 2Mbps、上传 700kbps=0.7Mbps。问 client-server 与 P2P 理想最短分发时间。/ A 15-Gbit file is distributed to 100 peers. Server upload is 30 Mbps; each peer downloads at 2 Mbps and uploads at 700 kbps. Using decimal units and the ideal model, compare client-server and P2P completion times.

先把文件写成 15000Mbit。Client-server 两项为 100×15000/30=50000 秒，以及 15000/2=7500 秒，因此选 **50000 秒，C**。

P2P 仍需要服务器发出至少一份文件，至少 500 秒；每人下载至少 7500 秒；全体上传总能力 30+100×0.7=100Mbps，供给 NF 需要 15000 秒。三项取最大，得 **15000 秒，B**。



图中每根柱都是一个必要时间约束，最大者决定该理想模型下的瓶颈。不是把三根柱相加，因为不同资源可同时工作。原教师页的下界符号视觉上写成“>”，最短时间公式应理解为“至少”，流体理想条件下可以达到；严格大于会排除等号而与 Q1 的构造冲突。

English answer: Client-server: max(50000,7500)=50000 s. P2P: max(500,7500,15000)=15000 s. Upload assistance reduces the aggregate bottleneck but does not remove the other constraints.

变式 / Transfer： N 改为 10，其他条件不变，P2P 会比 client-server 快吗？/ Recompute both bounds for ten clients.

答案 / Answer

client-server=max(5000,7500)=7500 秒；P2P=max(500,7500,150000/37)=7500 秒。此时每人的 2Mbps 下载是瓶颈，增加上传互助不会突破它。/ Both are 7500 s, limited by the slowest download rate.

Problem 3. 不上传的人，是否仍可能下载完成？

原题 / Task： Bob 加入 BitTorrent 但不上传；a 能否完整下载？b 多台不同 IP 电脑是否可能让这种 free-riding 更高效？/ Can a non-uploading participant finish, and can multiple hosts improve that possibility?

Tutorial 4 教师解答 v2，第 9–11 张幻灯片答两个问题均**可能，B**。除了优先服务回馈较好的 peer，课堂 BitTorrent 模型还有 optimistic unchoking：周期性试着给某个新 peer 机会。如果有足够 peer 停留足够久，Bob 可能逐渐收齐；参与者过早离开则未必能完成。

多主机可各自获得机会并收集不同片段后合并，教师称为一种 Sybil 式行为。这是机制分析；实际速率还受 peer 数量、停留时间、网络资源等影响，不能把主机数量乘上去就断言下载速度同比增加。

English answer: Both claims are possible under the stated classroom assumptions. Optimistic unchoking can supply a free-rider; multiple identities may collect complementary chunks. Neither completion nor proportional speedup is guaranteed.

自测 / Check：如果所有 peer 在 Bob 收齐之前离开，前面的结论是否矛盾？/ Does early peer departure contradict the possibility claim?

答案 / Answer

不矛盾，原答案有停留时间和供给条件。“可能”不等于“必然”。/ No; the sufficient opportunity may not exist in that swarm.

Problem 4. DASH 到底存 N 份，还是 N^2 份？

原题有 N 个视频质量版本和 N 个音频质量版本。开头说任意选择音视频，a 的括号又明确要求按质量一一配对；教师解答 v2 第 14 张幻灯片按**一一配对**作答，因此本题 a 为 **N，A**。b 分离保存音频和视频，需 **2N，B**，客户端负责同步。

以 N=3 看清：固定高配高、中配中、低配低，预合并文件只有 3 份。若真正允许任意组合，要把 3×3 种配对都提前合并，则是 9 份。若分轨，保存 3 视频 +3 音频共 6 份，播放时组合即可。这三种情境不是互相推翻，差别在配对约束。

English answer: Under part a's explicit one-to-one matching assumption, N combined files are needed. Separate audio and video require 2N files. N^2 applies only when every pair is precombined. These counts concern representations, ignoring segment multiplication.

独立题 / Transfer：视频 4 种、音频 2 种，全部预组合与分轨分别几份？/ Four video versions and two audio versions: all pairs versus separate tracks.

答案 / Answer

8 与 6。不是 4^2，因为两边版本数量不同。/ Eight combined files or six separate tracks.

复习与来源索引

原题 p1 含 Q1–3，p2 含 Q4。Tutorial 4 教师解答 v2，第 1–5 张幻灯片对应 Q1，Tutorial 4 教师解答 v2，第 6–8 张幻灯片对应 Q2，Tutorial 4 教师解答 v2，第 9–11 张幻灯片对应 Q3，Tutorial 4 教师解答 v2，第 12–15 张幻灯片对应 Q4。原题数值、单位及括号条件均保留；图为按原数值生成的教学图。

复习沿用 N234、N235、N236。Q3 可结合 optimistic unchoking，复述为什么答案是“可能”。

Tutorial 5 · Demultiplexing, Checksums and Reliable Transfer

写清端点，算清位，再用反例检查协议假设

Chapter 3（已下载部分）（本册 p.41）· 课程目录 · 上一份：Tutorial 4（本册 p.73）

依据当前原题 2 页，共四题；当前未下载教师解答。以下答案根据 Chapter3 推导，并对照历史同题核查。

基础薄弱时先补二进制与字节（基础册 p.38）、时间轴（基础册 p.40）。Q1–3 为核心必会、中等难度，检查端点方向与反码；Q4 也是核心必会，但难度较高，需要构造“旧包被当新包”的具体事件顺序。优先级直接来自当前原题。

做 Q4 前可返回 rdt 双方状态与编号（本册 p.43）、rdt3.0 定时器（本册 p.46）。每一步分别写发送者正在等哪个 ACK、接收者正在等哪个数据编号；序号重用后才出现本题需要的歧义。这里编号标识数据包，和 TCP 按字节编号（本册 p.50）的练习不同。

Q1. How is a socket identified?

题目 / Question： UDP socket 怎样完整标识？TCP 呢？区别是什么？/ Compare identification in the instructor's UDP and established-TCP demultiplexing models.

按 Chapter 3 part 1，第 11、13 张幻灯片的普通模型：UDP 接收 socket 看目的 IP、目的端口；已建立 TCP 连接看源 IP、源端口、目的 IP、目的端口。同一个 UDP 接收 socket 可以接收多个来源的数据报，TCP 按不同四元组区分连接。

术语“完整标识”这里按课件抽象回答；监听 socket 和已建立连接不同，真实系统也允许 connected UDP。本题按上述课堂模型作答。

English answer: The classroom UDP model uses the destination IP and destination port. An established TCP connection is identified by a four-tuple containing both endpoints. Different TCP clients can therefore share the same server port without sharing a connection.

Q2. Server replies ｜回程要把两端一起交换

Tutorial 5, CS5222, Sem A, 2026-27

1. How is a UDP socket fully identified? What about a TCP socket? What is the difference between the full identification of both sockets?

2. Consider the figure below. What are the source and destination port values in the segments flowing from the server back to the clients' processes? What are the IP addresses in the network-layer datagrams carrying the transport-layer segments?

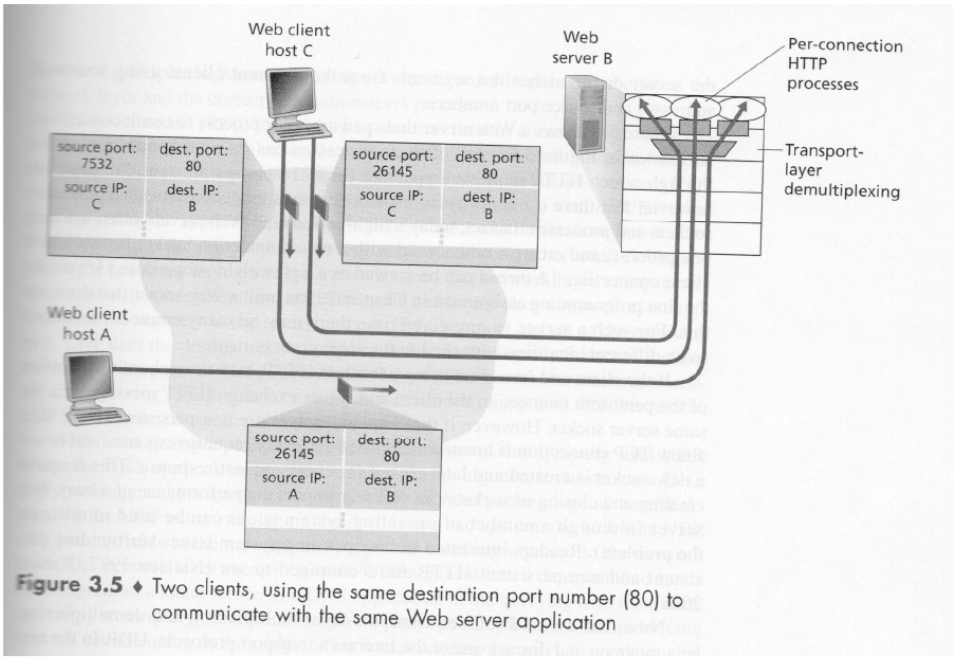


Figure 3.5 ♦ Two clients, using the same destination port number (80) to communicate with the same Web server application

3. UDP and TCP use 1s complement for their checksums. Suppose you have the following three 8-bit bytes: 00100011, 01001110, 01010100. What is the 1s complement of the sum of these 8-bit bytes? (Note that although UDP and TCP use 16-bit words in computing the checksum, for this problem you are being asked to consider 8-bit sums.) Show all work. With the 1s complement scheme, how does the receiver detect errors? Is it possible that a 1-bit error will go undetected? How about a 2-bit error?

完整条件 / Conditions：原图有服务器 B:80；客户端 C 两条连接分别从 7532 和 26145 发起，客户端 A 一条连接从 26145 发起。问服务器回程 segment 的源/目的端口及外层 IP 源/目的地址。/ Read all three return paths from the server to the client processes.

对应请求	回程源端口	回程目的端口	IP 源 → 目的
C:7532→B:80	80	7532	B→C
C:26145→B:80	80	26145	B→C
A:26145→B:80	80	26145	B→A

端口写在传输层首部，IP 地址写在网络层首部。图中的 A/B/C 代表主机 IP，答案沿用这些符号。

English answer: The server's source port is 80 on all three replies. Destination ports are 7532, 26145 and 26145, with IP destinations C, C and A. The last two connections differ in client IP.

变式 / Transfer：服务器改为 8443，客户端 C 的 7532 仍不变，回程端口怎样写？/ Change only the server port to 8443.

答案 / Answer

8443→7532；IP 仍 B→C。/ Source 8443, destination 7532; IP endpoints remain B and C.

Q3. One's-complement checksum ｜八位是题目的简化

原题给三个 8bit 字节：00100011、01001110、01010100，要求逐步求和、取反，说明接收端检查及 1bit/2bit 错误检测。真实 UDP/TCP 使用 16bit 字；此处按 8bit 演算。

```
  00100011   = 35
+ 01001110   = 78
-----------
  01110001   = 113
+ 01010100   = 84
-----------
  11000101   = 197
```

没有最高位进位，按位取反得到 checksum **00111010**（58）。接收端对数据与 checksum 采用同样反码加法：197+58=255，即 11111111。若有超出 8bit 的进位，必须先卷回低位，再继续检查；本题恰好没有，不代表一般情况下可以省略。

一个 bit 翻转会改变加权和而被检测。两个 bit 可能相消：把第一个字的最低位从 1 改 0（35 变 34），第二个字的最低位从 0 改 1（78 变 79），总和仍 197，checksum 不变，错误可能漏检。这是具体反例，不是“两个 bit 一定漏检”。

English answer: Sum 35+78+84=197 (11000101), then complement to 00111010. The receiver expects an all-one one's-complement sum including the checksum. Single-bit corruption is detected; two compensating changes can be missed.

独立题 / Transfer：数据只有 11111111 和 00000010，8bit 校验和是什么？/ Compute the toy 8-bit checksum for 11111111 and 00000010, including end-around carry.

答案 / Answer

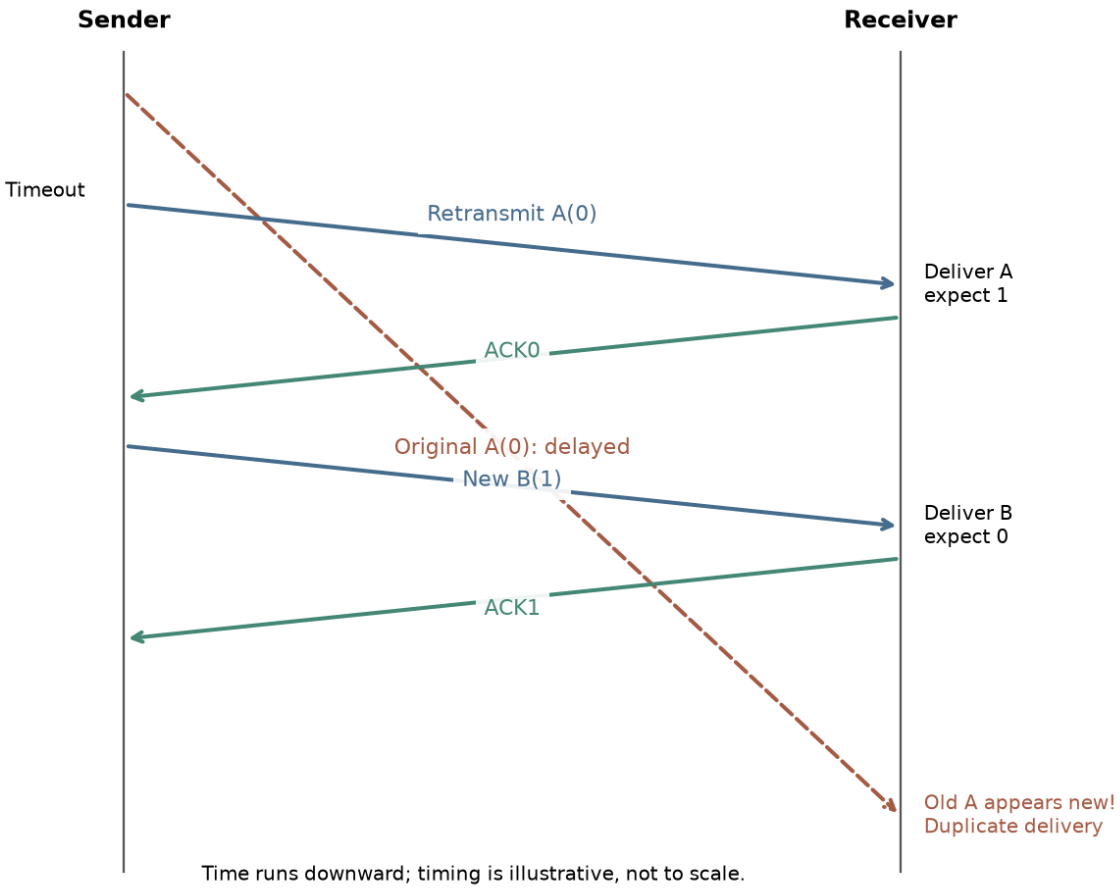
普通和 257=1 00000001，卷回后 00000010，再取反 11111101。/ Fold the carry to get 2, then complement to 253.

Q4. Alternating bit ｜编号又回到 0，旧包也可能叫 0

原任务 / Task： rdt3.0 的信道允许消息重排时，只有 0/1 序号的交替比特协议是否总能正确？若不能，画发送者左、接收者右、时间向下的数据/ACK 交换，标明序号。/ Give a reordering counterexample with explicit packet and ACK numbers.

只说“乱序会出错”不够，需要让接收端在错误时刻恰好期待旧包的编号。构造如下：

1. 发送数据 A，编号 0；它在网络中很慢，暂未到。
2. 发送端超时，重传 A(0)。重传先到，接收端交付 A 并回 ACK0。
3. 发送端收到 ACK0，发送 B(1)。接收端交付 B、回 ACK1，重新等待 0。
4. 最早那份 A(0) 此时才到。接收方期望 0 且校验通过，于是误把 A 当下一条新数据，再次交付。



此例只用超时造成的重传和允许的重排，不要求把随机损坏伪装成有效包。收到第三个数据时接收方没法仅凭“0”知道它是老 A 还是新 C。说明可靠性结论依赖对信道和旧副本寿命的假设。

English answer: A delayed original packet 0 can arrive after its retransmission and packet 1 have already been accepted. The receiver expects 0 again and can deliver the stale data as new. Thus alternating-bit correctness does not extend to arbitrary reordering and delayed duplicates without additional assumptions.

迁移 / Transfer： 只把序号从 2 种改为 4 种，是否能无条件解决“任意长延迟”? / Does a larger finite sequence space eliminate arbitrarily delayed old duplicates?

答案 / Answer

不能无条件保证；编号最终仍会重用，还需要寿命、窗口或其他协议约束。更多序号可以扩大安全范围，但不替代假设。/ No finite sequence space alone prevents a sufficiently old duplicate from surviving until number reuse.

来源与复习

原题 p1 为 Q1–3 及完整端点图，p2 为 Q4。Chapter 3 part 1，第 9–15 张幻灯片对应端点，Chapter 3 part 1，第 19–20 张幻灯片对应校验，Chapter 3 part 1，第 35–44 张幻灯片对应编号、超时与重复处理；Q4 反例为按原条件整理的推导。

沿用 N150、N237、N238。Q2 可在表格里换一组端口，自行写出回程。

Assignment 1 · Chapters 1 and 2 ｜计算、事务与技术证据

课程目录 · Chapter1（本册 p.5）· Chapter2（本册 p.24）· 原任务（4 页）

本讲义提供 P6、P31、P17 的纸笔推导及 Zoom 资料核查方法。当前未提供教师解答，以下推导与资料分析供学习参考。截止日期、提交格式和手续回查原任务及课程信息栏。

基础可选：单位与时间轴（基础册 p.37）、报文字段（基础册 p.39）。当前四题都属于核心任务；P6 难点是首/末位事件，P31 是跨链路重叠，P17 是跨会话状态，Zoom 是产品情境与来源边界。理解难度均为中等。

P6. Transmission and propagation ｜最后一位刚出发，第一位可能已经到达

CS5222 Assignment 1 (on Chapter 1 and 2)

Chapter 1

P6. This elementary problem begins to explore propagation delay and transmission delay, two central concepts in data networking. Consider two hosts, A and B, connected by a single link of rate R bps. Suppose that the two hosts are separated by m meters, and suppose the propagation speed along the link is s meters/sec. Host A is to send a packet of size L bits to Host B.

a. Express the propagation delay, d_{prop}, in terms of m and s.

b. Determine the transmission time of the packet, d_{trans}, in terms of L and R.

c. Ignoring processing and queuing delays, obtain an expression for the end-to-end delay.

d. Suppose Host A begins to transmit the packet at time $t = 0$. At time $t = d_{trans}$, where is the last bit of the packet?

e. Suppose d_{prop} is greater than d_{trans}. At time $t = d_{trans}$, where is the first bit of the packet?

f. Suppose d_{prop} is less than d_{trans}. At time $t = d_{trans}$, where is the first bit of the packet?

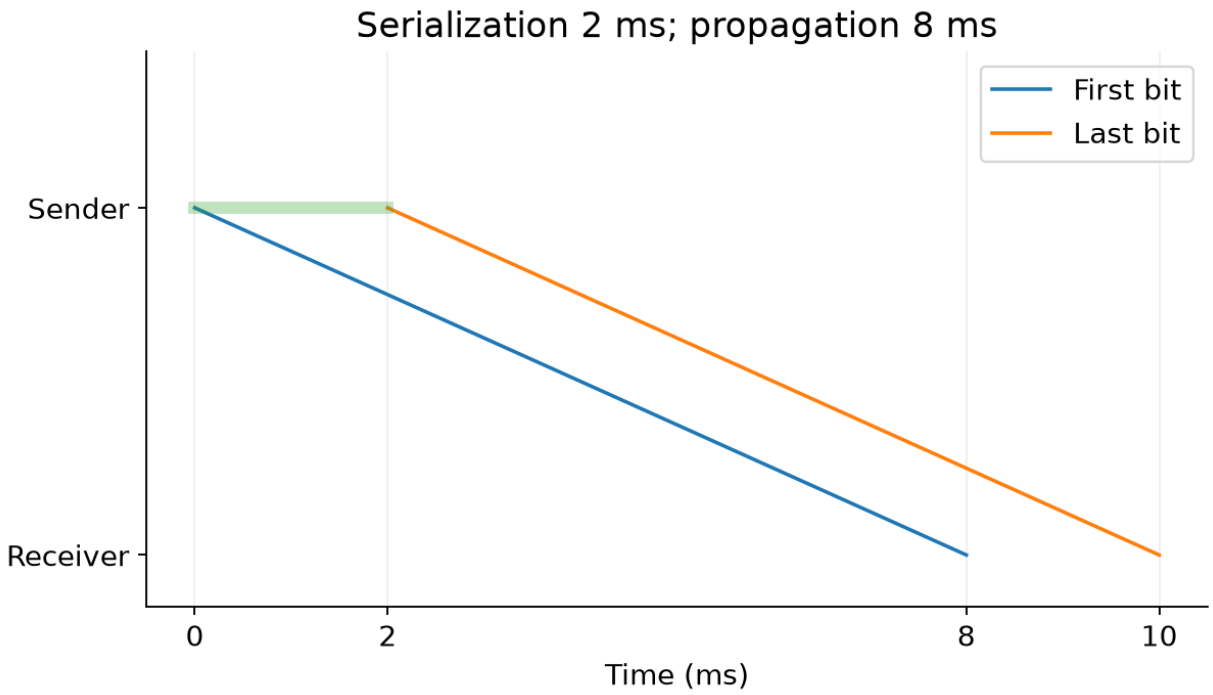
g. Suppose $s = 2.5 \cdot 10^8$, $L = 120$ bits, and $R = 56$ kbps. Find the distance m so that d_{prop} equals d_{trans}.

条件 / Conditions: 主机 A、B 相距 m 米，链路发送速率 R bit/s，传播速度 s m/s，包长 L bit。忽略处理、排队，从 A 开始发送首位计时 t=0。/ Hosts A and B are m metres apart; link rate is R bit/s, propagation speed s m/s, and packet length L bits. Ignore processing and queueing and start timing with the first bit.

原小问	中文推导与答案	English answer
a 传播	$d_{prop} = m/s$，距离除速度	Propagation delay is m/s.
b 发送	$d_{trans} = L/R$，数据量除速率	Serialization time is L/R.
c 整包端到端	$L/R + m/s$，末位先被送入链路再传播	Complete-packet delay is L/R+m/s.
d t=dtrans，末位位置	在连续比特模型的边界时刻刚离开 A 进入链路，还没有走完整条路	The last bit is just entering the link at A.
e dprop>dtrans，首位位置	仍在线路内，离 A 的距离为 $sL/R < m$	The first bit remains on the link, sL/R metres from A.
f dprop<dtrans，首位位置	已到达 B；不能继续说还在链路上	The first bit has already reached B.
g 两时延相等	$m/s = L/R$，故 $m = sL/R$	Set m=sL/R.

g 给 s=2.5×10^8m/s、L=120bit、R=56kbps=56000bit/s：

$$d_{trans} = 120/56000 \approx 0.002142857 \text{ s}, \quad m \approx 535714.29 \text{ m} = 535.714 \text{ km}. \tag{A1.1}$$



图是帮助读事件的教学示意（参数 2ms/8ms），不是 g 题的参数图。g 的数值单独复算，避免把示意图标注抄进答案。

变式 / Transfer: L=240bit、R 不变，要保持两时延相等，m 如何变化？/ Double packet length while keeping R and s fixed; how must m change?

答案 / Answer

m 翻倍，约 1071.429km。/ m doubles to about 1071.429 km. 改变的是等时延所需距离，不是信号速度。

P31. Message segmentation ｜像流水线一样，让不同链路同时工作

P31. In modern packet-switched networks, including the Internet, the source host segments long, application-layer messages (for example, an image or a music file) into smaller packets and sends the packets into the network. The receiver then reassembles the packets back into the original message. We refer to this process as *message segmentation*. Figure 1.27 illustrates the end-to-end transport of a message with and without message segmentation. Consider a message that is $8 \cdot 10^6$ bits long that is to be sent from source to destination in Figure 1.27. Suppose each link in the figure is 2 Mbps. Ignore propagation, queuing, and processing delays.

a. Consider sending the message from source to destination *without* message segmentation. How long does it take to move the message from the source host to the first packet switch? Keeping in mind that each switch uses store-and-forward packet switching, what is the total time to move the message from source host to destination host?

b. Now suppose that the message is segmented into 800 packets, with each packet being 10,000 bits long. How long does it take to move the first packet from source host to the first switch? When the first packet is being sent from the first switch to the second switch, the second packet is being sent from the source host to the first switch. At what time will the second packet be fully received at the first switch?

c. How long does it take to move the file from source host to destination host when message segmentation is used? Compare this result with your answer in part (a) and comment.

d. In addition to reducing delay, what are reasons to use message segmentation?

e. Discuss the drawbacks of message segmentation.

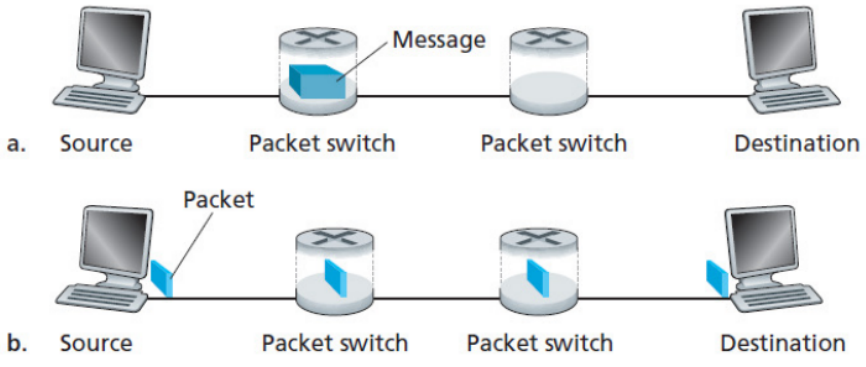


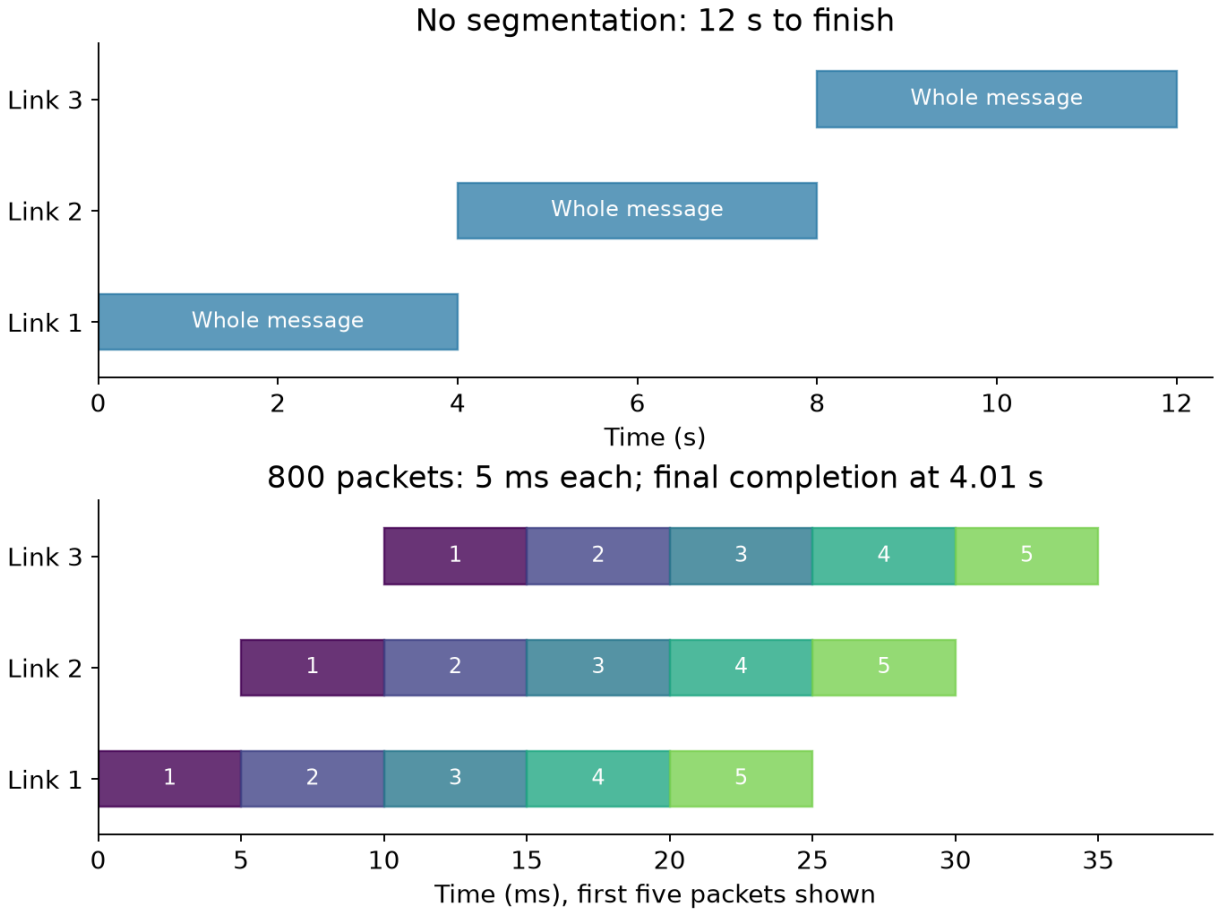
Figure 1.27 ♦ End-to-end message transport: (a) without message segmentation; (b) with message segmentation

条件 / Conditions: 原图源 → 交换机 1→ 交换机 2→ 目的，共 3 条 2Mbps 链路；消息 8×10^6bit，存储转发，忽略传播、排队和处理。分段时为 800 个 10000bit 包，题目未另加包头。/ An 8-million-bit message crosses three 2-Mbps store-and-forward links. Ignore propagation, queueing and processing. Segmentation creates 800 packets of 10000 bits each, with no additional header overhead specified.

a 不分段，第一链路 $8\times10^6/(2\times10^6) = 4$s；交换机必须等整条消息，3 跳总 **12s**。/ Without segmentation: 4 s to the first switch and 12 s to the destination.

b 单包发送 $10000/(2\times10^6)$=**0.005s=5ms**。第一包到第一交换机 t=5ms；它继续去第二交换机时，源发送第二包，第二包在 t=**10ms** 完全到第一交换机。/ First packet reaches switch1 at 5 ms; packet2 is completely received there at 10 ms.

c 第一包到目的需 3×5ms，后续 799 包每隔 5ms 到，总 $(800+3-1)\times0.005 =$ **4.01s**。/ Completion time is 4.01 s. 链路本身没变快，减少的是“大消息在每一跳全等完”的串行等待。



图中展开前几个包，末尾标出全部 800 包的完成时刻。

d 其他好处：较短包给其他流量穿插机会；出现丢失时可只重传受影响部分（若上层协议支持）；较小存储/处理单位便于逐包转发。/ Segmentation enables interleaving, selective retransmission when supported, and manageable forwarding units.

e 代价：每包头部和处理开销，更多调度；可能乱序，需要识别与重组；重传/超时机制增加复杂度。/ Costs include headers, per-packet processing, possible reordering, reassembly and recovery complexity. 不把“越小永远越快”当结论；本题忽略头部是有意简化。

变式 / Transfer: 同消息改成 400 包，每包 20000bit，其他不变，总多久？/ Use 400 packets of 20000 bits with the same path.

答案 / Answer

每包 10ms，总 (400+3−1)×0.01=4.02s。/ 4.02 s. 仍近似受源端总上传 4s 约束，只是流水线填充稍慢。

P17. POP3 ｜完成整次事务，不只补两行

Chapter 2

P17. Consider accessing your e-mail with POP3.

a. Suppose you have configured your POP mail client to operate in the download-and-delete mode. Complete the following transaction:

```
C: list
S: 1 498
S: 2 912
S: .
C: retr 1
S: blah blah ...
S: .........blah
S: .
?
?
```

b. Suppose you have configured your POP mail client to operate in the download-and-keep mode. Complete the following transaction:

```
C: list
S: 1 498
S: 2 912
S: .
C: retr 1
S: blah blah ...
S: .........blah
S: .
?
?
```

(Note: For parts a and b above, you should continue writing the transaction until the service is finished. It is not limited to only the next two lines.)

c. Suppose you have configured your POP mail client to operate in the download-and-keep mode. Using your transcript in part (b), suppose you retrieve messages 1 and 2, exit POP, and then five minutes later you again access POP to retrieve new e-mail. Suppose that in the five-minute interval no new messages have been sent to you. Provide a transcript of this second POP session.

完整条件 / Conditions: 已认证进入 transaction，LIST 给两封邮件，编号 1/2、大小 498/912byte；原片段已经 RETR1。a 为 download-and-delete，b 为 download-and-keep，均继续到服务完成。c 在 b 后退出，5 分钟后再次登录，无新邮件，给出第二会话。/ Two messages of 498 and 912 octets are present. Finish

the full download-and-delete and download-and-keep transactions, then show a second keep-mode session five minutes later with no new mail.

原题略去若干+OK确认；下面补齐规范响应。[message ...]只是未知邮件内容的说明占位，**不是实际发送的协议文本**。每行以 CRLF 结束，单独.结束多行结果。命令字大小写不影响本题含义。

a. Download and delete / 下载并删除

```
C: LIST
S: +OK 2 messages (1410 octets)
S: 1 498
S: 2 912
S: .
C: RETR 1
S: +OK 498 octets
S: [message 1 content]
S: .
C: DELE 1
S: +OK message 1 marked for deletion
C: RETR 2
S: +OK 912 octets
S: [message 2 content]
S: .
C: DELE 2
S: +OK message 2 marked for deletion
C: QUIT
S: +OK signing off
```

DELE 先标记，成功 QUIT 使会话进入 UPDATE 并提交删除；本例假设操作成功且没有异常退出。只写 DELE1 然后 QUIT 会漏掉第二封的下载与删除。/ Retrieve both messages, mark both for deletion, then successfully quit to commit the update.

b. Download and keep / 下载并保留

```
C: LIST
S: +OK 2 messages (1410 octets)
S: 1 498
S: 2 912
S: .
C: RETR 1
S: +OK 498 octets
S: [message 1 content]
S: .
C: RETR 2
S: +OK 912 octets
S: [message 2 content]
S: .
```

```
C: QUIT
S: +OK signing off
```

没有 DELE，两封保留。RETR 不会自动删除；“keep”不是另加一个名叫 KEEP 的 POP3 命令。/ No messages are marked for deletion, so both remain on the server.

c. Second keep-mode session / 五分钟后的第二次会话

假设没有额外客户端 UIDL 去重、服务器未改变这两封邮件的编号，重新认证后仍 LIST 两封；可以再次 RETR1/2。原问题仅说“无新邮件”，没有说服务器将已读邮件隐藏。

```
S: +OK POP3 server ready
C: USER <username>
S: +OK
C: PASS <password>
S: +OK authenticated
C: LIST
S: +OK 2 messages (1410 octets)
S: 1 498
S: 2 912
S: .
C: RETR 1
S: +OK 498 octets
S: [message 1 content]
S: .
C: RETR 2
S: +OK 912 octets
S: [message 2 content]
S: .
C: QUIT
S: +OK signing off
```

尖括号内为用户名和密码的占位。实际客户端可保存 UIDL 跳过已下载邮件；若采用这种扩展约定，可能 LIST/UIDL 核对后不 RETR 旧邮件。应明确这是客户端保存状态的行为，而不是“POP3 服务器因为上次读过就删掉”。RFC1939（查阅于 2026-09-23）

English answer: With no deletion and no client-side duplicate suppression, both messages are still available and can be downloaded again. State any UIDL-based client behavior explicitly.

变式 / Transfer: b 中仅 DELE1 并成功 QUIT，之后无新邮件，剩多少总字节？/ Delete only message1 in part b and successfully quit. How many message octets remain?

答案 / Answer

只剩原邮件 2，912byte；新会话编号不应依赖旧编号恒定。/ 912 octets remain; message numbers need not remain identical across sessions.

Zoom. Apply protocol ideas with sources ｜先说研究的是哪一种 Zoom 连接

Question. Zoom (https://zoom.us/) is a videoconferencing application. After doing some research about its communication protocols, answer the following questions:

a) What kind of application-layer protocols do Zoom use? Please state three of them and briefly discuss their functions.
b) Is Zoom primarily based on a client-server or P2P architecture? Why?
c) Which transport layer protocol is Zoom primarily using? Why?
d) During our CS5222 lecture using Zoom, does a student's computer need to obtain the IP address of the lecturer's computer? If so, how? If not, why?

原问题 / Questions: a 列举三种应用层协议并解释作用；b 主要 client-server 还是 P2P；c 主要传输协议及原因；d 学生是否需要老师电脑 IP 及原因。/ Identify three application-layer protocols and their roles, explain the main architecture and transport choice, and determine whether a student needs the lecturer's host IP.

核查情境：2026-09-23，普通多人 Zoom Meetings、原生 Zoom Workplace 客户端、经 Zoom 云人会的默认讨论范围。没有实际抓取本校课堂流量，也未确认本校启用哪些企业功能。因此下述为官方资料支撑的典型架构分析，不是对某一次课堂连接的实测报告。

a. 应用协议：产品生态的三项，与原生媒体路径分开

题目未限定客户端类型，可用下表列出 Zoom 产品生态明确支持的三项，并把各自情境写进答案：

协议	功能（中文）	English explanation	证据/适用情境
HTTPS	网页/客户端与 Zoom Web 服务的请求、资源获取等	Secure HTTP access to web services and content	官方网络配置明确 HTTP/HTTPS 访问与 CDN；不据端口猜全部音视频都经 HTTP
SIP	让标准会议室端点进行会话建立/互通	Session signaling for supported standards-based room endpoints	Conference Room Connector；不是所有原生客户端必经路径
H.323（协议族）	另一类标准音视频会议端点经连接器接入	A conferencing protocol suite for supported room-system interoperability	同一官方连接器资料；需注明它是协议族而非单一报文协议

表中 HTTPS 用于 Web 服务，SIP/H.323 用于会议室端点互通，作答时分别注明情境。若教师将题意限定为**原生客户端自身的媒体/控制三种协议**，应按该范围补充特定版本的技术资料，而不能把 SIP/H.323 互通项代替原生客户端路径的说明。公开端口表只能证明传输端口用途，不能单独证明 RTP/SRTP 等内部封装；作答时应注明所选客户端与部署情境。

b. 架构 / Architecture

普通多人云会议主要由云服务协调与转发，按课程分类主要是 client-server：各客户端向服务基础设施连接，云处理参与者发现、会话与媒体分发。**这是基于官方云连接/回退说明的架构归纳**，具体部署也可能包含下面介绍的 P2P 或 Mesh 连接。Zoom Mesh 官方说明

English: Ordinary multi-participant cloud meetings are primarily client-server in the course's sense; clients rely on Zoom infrastructure for coordination and media distribution.

明确例外：官方允许同一局域网的两人会议在符合设置条件时用 peer-to-peer；加入第三人等情况会使用云。两人 P2P 说明企业启用 Zoom Mesh 时，局域网内可由 parent 客户端给 child 分发视频；这是有云协调的混合机制，不能拿来证明普通多人课堂始终直接连接老师。

c. 传输 / Transport

实时媒体通常优先 UDP，官方 DSCP 文档说明默认媒体在 UDP8801 等端口，特定管理设置可拆分音频/视频/共享端口。官方媒体端口说明控制/Web 访问及媒体回退可以使用 TCP；"Zoom 只用 UDP"错误。课程层面的原因是实时音视频更在意及时性，可由应用处理丢失与适应码率，而 TCP 有序交付可能让后续数据等重传；这解释选择动机，不意味 UDP 本身自动更可靠或保证低延迟。

English: Real-time media commonly prefers UDP, while control/web traffic and fallback paths may use TCP. Low delay and application-level adaptation matter more than waiting indefinitely for every media byte.

d. 老师电脑的 IP / Lecturer's host IP

在普通云课堂情境，学生连接的是 Zoom 提供的服务端点，不需要自己先解析或输入老师个人电脑的 IP。老师和学生各自向基础设施建立通信，服务负责转发；知道 meeting ID 也不是把它翻译成老师 IP。

English: In an ordinary cloud meeting, the student does not need to obtain the lecturer's host IP; Zoom infrastructure routes the session. Peer/mesh configurations may use other local endpoints and require separate analysis.

怎样整理成自己的作答

每个判断写成"结论 → 明确情境 → 机制 → 官方来源及查阅日期"。区分产品支持、当前默认与本机实测三种证据。原题是应用课程知识的调查题，不应把一段不知年份的博客当软件永久架构。

覆盖、复算与验收

原 PDF p1 的 P6(a–g)、p2 的 P31(a–e) 及原拓扑、p3 的 P17(a–c) 及"写到事务结束"要求、p4 的 Zoom(a–d) 全部对应。数值和分包图由 network_examples.py 复算；原题未给邮件正文，事务中使用占位说明。

完成前检查：每个速率/长度有单位，每段延迟不多算漏算；POP3 包含两封邮件与成功结束；Zoom 区分客户端/部署并注明来源时间。再回到已有 N260–N263 复习对应知识。

Research Report Guide ｜把一个真实问题解释清楚

课程目录 · Chapter1（本册 p.5）· Chapter2（本册 p.24）· 课程信息栏

这份指南帮助你选题、解释机制、核查资料和整理反思。原任务要求研究你在日常生活、新闻或工作中遇到的新网络技术，联系教材 Chapter1–7，排除 Chapter8 安全主题；用自己的解释和图说明机制，并反思学习过程。格式、页数、评分与提交要求以 Canvas 中的当前原任务与 Rubric 为准。

按选题补基础：涉及码率/吞吐就回网络单位（基础册 p.37），涉及状态/时间就回报文与时间轴（基础册 p.39）。没有实际卡点便直接开始。

能力	优先级	难度与卡点	达标表现；依据
明确问题、解释机制	核心必会	中至高；不能只有名词	不看原文也能画出数据/控制流；原任务 A
来源核查与边界	核心必会	中；资料版本/适用场景	每个关键结论有对应证据；原任务 p2
图文与英语表达	核心必会	中；精简与可读	图能帮助解释一个具体难点；原 Rubric
真实反思	核心必会	中；具体变化而非套话	记录自己确实做过的核查与困难；原任务 B
复杂数学/未用机制	按需拓展	可能高	只有支撑论点时补；整理者建议，不暗示不考

1. 选题从一件事开始，不从一串热门缩写开始

先记录你**确实遇到**的情况：某个应用在带宽变化时切画质、同一内容从不同地点下载速度不同、多人共享网络时服务突然卡顿。题目应围绕一个能解释的机制问题，例如“客户端怎样在吞吐波动时选择下一视频块的编码版本”。

用三个问题缩小范围：这个问题属于网络吗？能对应教材哪一层/哪章？在本次篇幅内能用一幅图、一个小场景说明主要机制吗？“所有 5G 技术”往往太宽；“某种特定机制在什么条件下改善什么问题”更容易论证。选题的新意应落在具体变化、部署或问题上。若选 DASH 等课内基础，需说明你研究的具体新变化、部署或问题，不能只复述课件。

English framing (整理表达): *This report examines how [a specific mechanism] addresses [a concrete networking problem] under [a defined setting].* 方括号中填写你的具体问题、机制和调查情境。

2. 把问题、机制、效果连成一条可核查的解释

先写旧方法遇到什么限制，再写新机制改了哪一步，最后讨论收益与代价。不要“协议很先进，所以性能很好”这样跳过中间因果链。

以分块视频为**教学例**：播放每秒消耗编码数据 → 网络吞吐会变动 → 缓冲可能耗尽 → 客户端按估计吞吐/缓冲选择下一块 → 可能减少停顿，但画质也会变化。这里每条箭头都能被课程概念支持，而不是把“自适应”三个字当解释。

完整补算 / Worked example: 4 秒片段编码 2Mbps，数据量 8Mbit；若稳定吞吐 4Mbps，下载 2 秒。在缓冲足够、同时播放、忽略请求与解码开销时，缓冲净增加 4−2=2 秒。/ A four-second segment encoded at 2 Mbps contains 8 Mbit. At 4 Mbps throughput it downloads in two seconds, increasing the playable buffer by two seconds under the stated assumptions.

变式 / Transfer: 网络降至 1Mbps，同一块需几秒？若开始时只缓冲 3 秒会发生什么？/ At 1 Mbps, how long does the same chunk take, and what if only three seconds are buffered initially?

答案 / Answer

下载 8 秒，已有缓冲只能撑 3 秒，理想模型中会停顿约 5 秒，等块到后才续播。/ Download takes eight seconds; under the simplified model, playback stalls for about five seconds. 缓冲余量和吞吐估计误差都会影响是否停顿。

上面是理想模型的教学算例；实际停顿还取决于播放器策略和网络变化。

3. 找来源时，为每一句重要话准备证据

优先找协议标准、技术提供方文档、原始研究论文或可复核的测量报告。二手讲解和 AI 适合帮助形成问题，但需要回到原资料核对。阅读来源时，找到直接支持该结论的段落或数据。

写作中的主张	应找的证据	必须检查
协议怎样工作	规范/原设计文档	版本、消息顺序、可选项、条件
某产品使用此机制	官方当前产品文档	原生/浏览器/企业部署是否相同
性能提高多少	原实验/可重复测量	对比方法、数据、指标、环境
有哪些限制	规范限制、实验失败或设计分析	不把推断写成已测事实
为什么和课程有关	教材/当前课件定位	是当前已学，还是教材后续章节

可维护一张很小的工作表，放在你自己的研究草稿里：`Claim | Source/section | Version/date | Supports what | Does not support what`。用它检查每条主张与来源是否对应。

如引用 AI 输出，原任务要求说明使用并自行负责核查；图、引用也要查，不能只给一个模型名字就当技术来源。对论文核对题名、作者、年份、DOI/官方页面，打开原文确认所引结论和实验设置。查不到原文的来源标为待核查，确认后再引用。

4. 图要解释困难，不是给文字化妆

选择一种与你的核心问题匹配的图：

- 交互问题画时序图，标清参与者、请求、响应、时间方向；
- 架构问题画节点与数据/控制流，说明图中箭头是什么；
- 权衡问题画有单位的曲线，说明数据来自实测、论文还是教学计算。

画完问自己：读者通过这张图能更容易答出哪个问题？节点之间的箭头应说明数据从哪里来、到哪里去，以及每一步做什么。引用原图标来源，改编图标“Adapted from…”，自己计算的图保存脚本和输入。AI 图同样要署明并逐条检查技术结构，不能因为好看就默认网络方向正确。

5. 英文报告怎样组织才连贯

建议采用内容驱动的小标题，而非每段中英对照：英文完成正式论证，中文草稿帮助自己先讲懂。术语第一次出现给全称与缩写，随后保持一致。可以按以下结构组织：

1. **Problem and context**：你实际怎样接触主题？问题是什么、研究范围到哪里？
2. **Background from the course**：挑真正用到的 1–3 个概念，解释它们为何必要。
3. **How the mechanism works**：沿一个具体请求/数据流走完步骤，配关键图。
4. **Benefits, limits and evidence**：哪项收益有证据，代价是什么，什么条件下不适用？
5. **Discussion**：你基于证据形成的判断、未解决问题与合理后续方向。

每段先给主张，再解释机制，再放证据与条件。比如 *The mechanism reduces repeated origin fetches when reusable content is available in cache* 比 *It greatly improves everything* 更可核查。不要把“可能”藏起来，也不用给每句话堆三个形容词。

英文自测 / English self-check: *What changes, why does it help, and under what conditions might it not help? / 改变了什么，为何有效，何时可能无效？*若不能脱稿回答，正文还需要补机制，而非继续增加术语。

6. Reflection log ｜写你真的经历过的理解变化

原任务的五个问题，可以分别对应一条具体记录：

原问题	真实记录的抓手
What did you learn that you did not know before?	原来怎么理解，哪个具体事实改变了它
What was the most challenging part?	一个真正卡住的机制/推导/资料冲突
How did GenAI help or hinder?	它帮你拆了哪一步，或给过什么误导
How did you verify AI-generated information?	打开了哪个原来源、比对哪句、做了什么检查

What would you do differently next time?	一个具体可实行的改进

可从现在开始记两三条带日期的研究记录，报告结束时据实整理。写清你做过什么、哪里卡住、怎样查证，以及理解发生了什么变化；工具使用情况按实际记录。

7. 交付前的自查与来源覆盖

读者应能复述你的问题与机制，认出它对应课程哪部分；一张图能独立读懂；数字都有单位、来源和条件；观点与已验证事实分开。英语表达、引用、AI 使用说明、格式与评分要求逐项回查原任务及课程信息。

原任务位置	本指南对应
p1 Goal、题目范围	开篇、§1
p1 Format、提交构成	原任务/信息栏链接；不生成正式提交件
p1 A：接触途径、机制、联系、动机/创新/挑战/观点	§1–2、§5
p1 自己的表达、图与清晰度	§4–5
p2 GenAI 引用、图/参考核查责任	§3–4
p2 B：五个 Reflection 问题	§6 逐项对应
p2 Evaluation、p3 Rubric	开篇能力表、§7；具体权重以原页为准

确定主题后，再围绕核心问题补充必要基础、原始来源和图示。